Deep Dive into Microsoft Fabric Capacities: Fundamentals, Concepts, and Advanced Strategies

Benni De Jagere

Slides

CAT



Thank you sponsors!

INSPARK

twintos

ilionx

InfoSupport
Solid Innovator

redgate

sopra steria

DATA GIANTS

BRANDER
company

SQL brander.nl
DATA brander.nl
CRM brander.nl
ERP brander.nl

Bull
Master your AI destiny

dbWatch

BBTG

Data Saturdays

Microsoft

Benni De Jagere

Principal Program Manager | Fabric Customer Advisory Team (Fabric CAT)

Fabric CAT

dataMinds .be Member

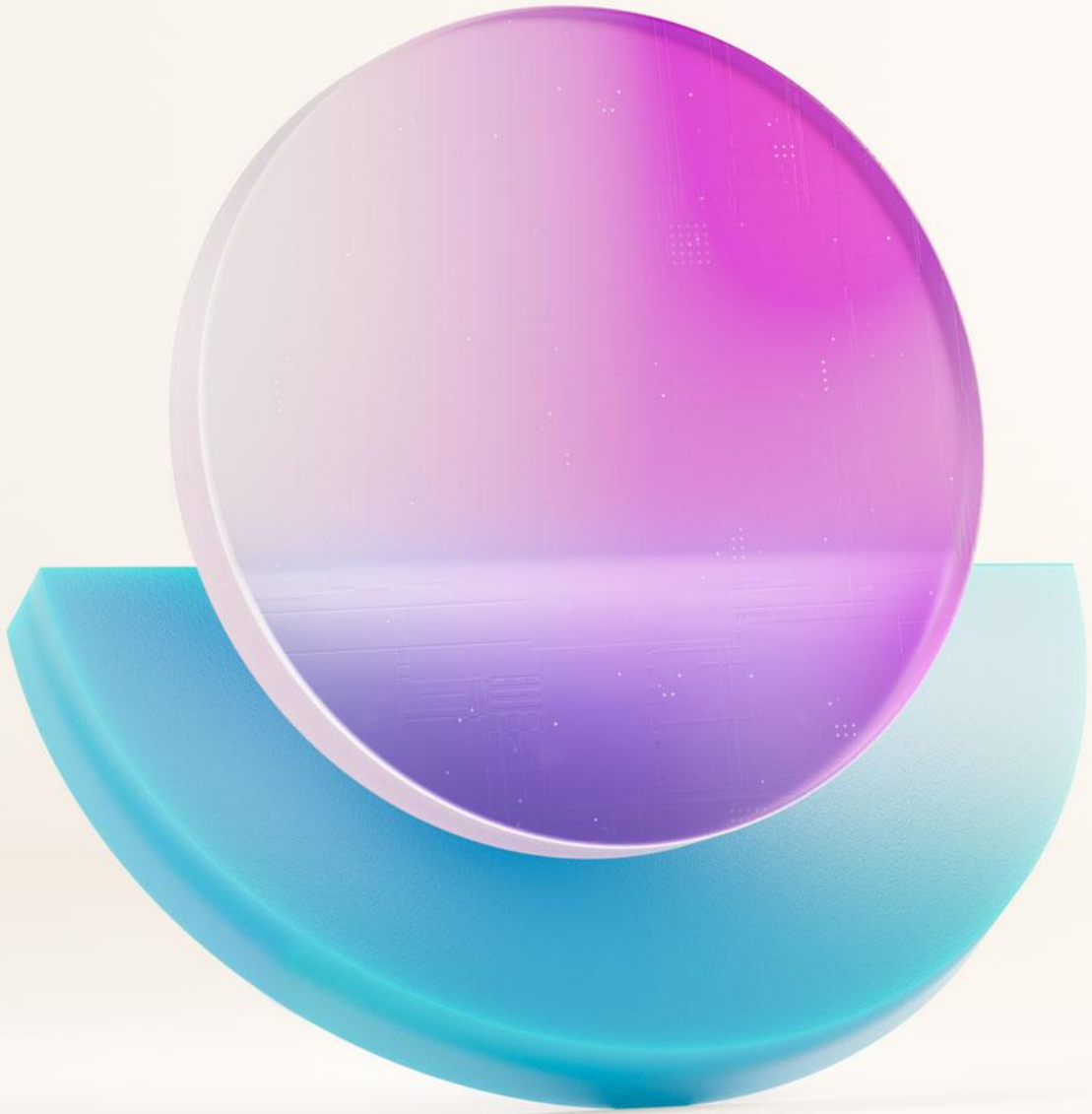
/bennidejagere

sessionize /bennidejagere

/bennidejagere



Microsoft Fabric
Community Conference



Assumptions

You read the session abstract ..

So you know about bursting, smoothing, and throttling ..

Right? 😼

Fabric Capacity Core Concepts

Scalable Compute

- **Capacity Unit Seconds (CUs)** are the base compute unit for all Capacities
- Your SKU Size determines the number of CUs you have available
- **Multiple workloads** can use the same capacity at the same time

Resizeable, Pausable

- Fabric SKUs (F SKUs) enable **added flexibility**
- **Resize** to increase or decrease the SKU to meet your compute needs
- **Pause** and **Resume** the capacity

Self-Managing

- **Self-Managing** with Bursting and Smoothing
- **Bursting** allows jobs to run at peak performance
- **Smoothing** reduces the impact of spikes in compute

But as with any resource, you still can push them too far (Throttling)

- Capacities offer **built-in resource governance**
- When there's too much smoothed usage, **throttling is applied**
  - **Interactive jobs Delay –** 20s delay, when 10 min > Usage <=60 min
  - **Interactive jobs Rejections –** Rejection, when 60 min > Usage <= 24 hours
  - **Background Rejections –** Rejection, when Usage > 24hrs

Capacities are a shared resource

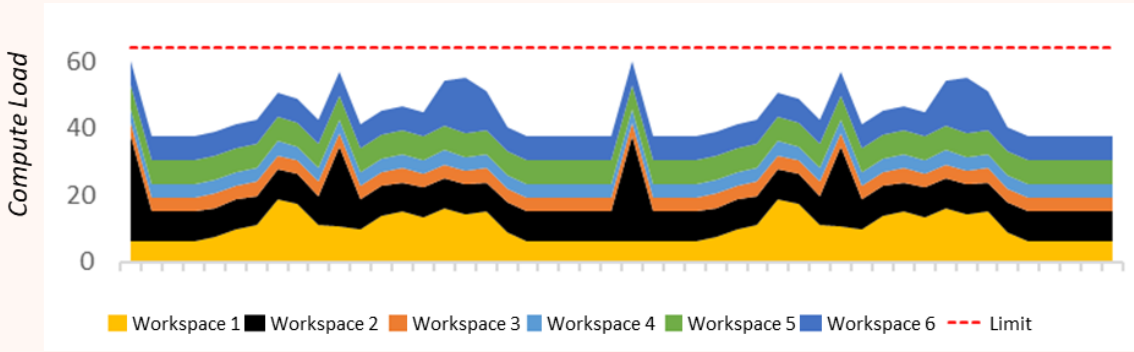
Shared across workloads
A single capacity is providing the compute power for all Fabric workloads.

There is no need to allocate compute for each workload separately.



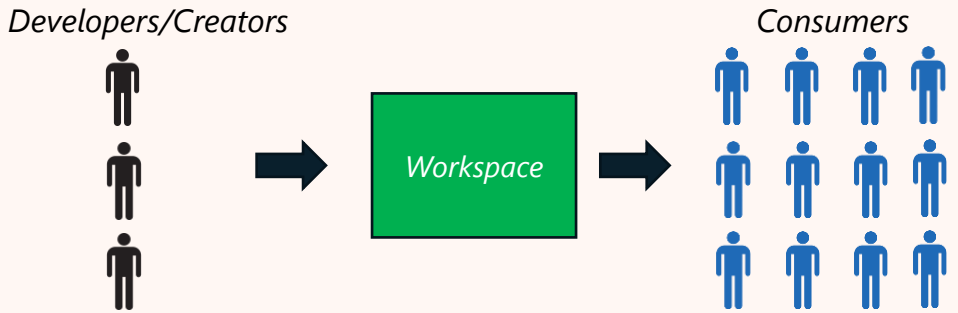
Shared Across Projects
A single capacity typically supports dozens of separate projects simultaneously, each managed in its own workspace.

It is rare to have a capacity dedicated to a single project



Shared across users
For each project, many developers will share a workspace where collaborative development and consumption at scale is managed.

Each creator can provision any artifact and run any job without the need for any pre-approval or planning

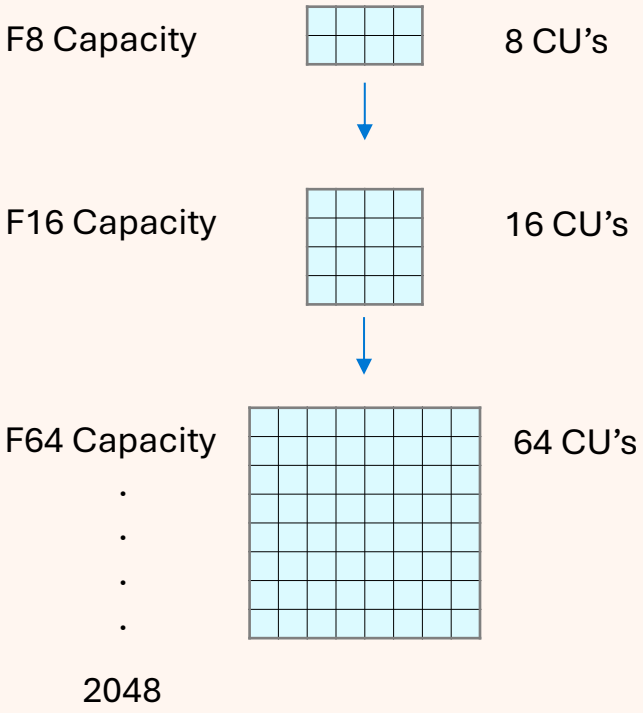


Capacities are flexible building blocks for growth

Capacities can be configured in endless ways to meet scale, usage and governance requirements while tuning to minimize TCO and performance goals

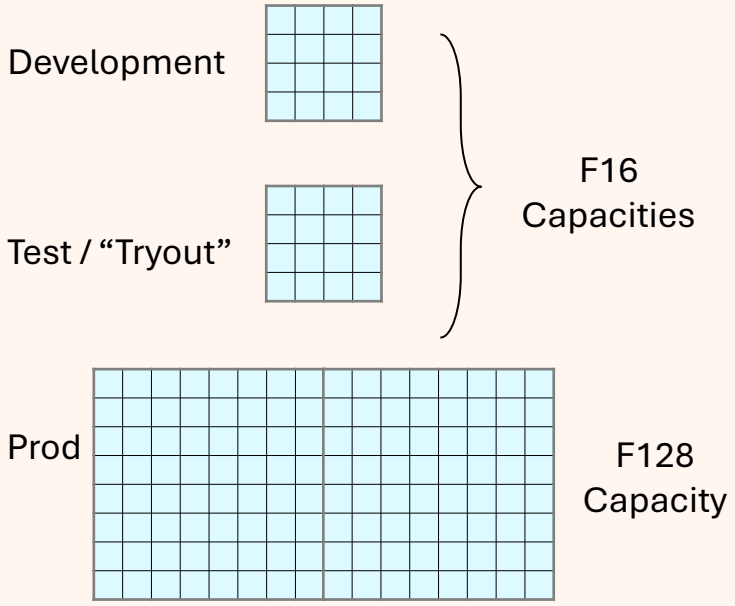
Scale Vertically

Increased capacity size provides more throughput



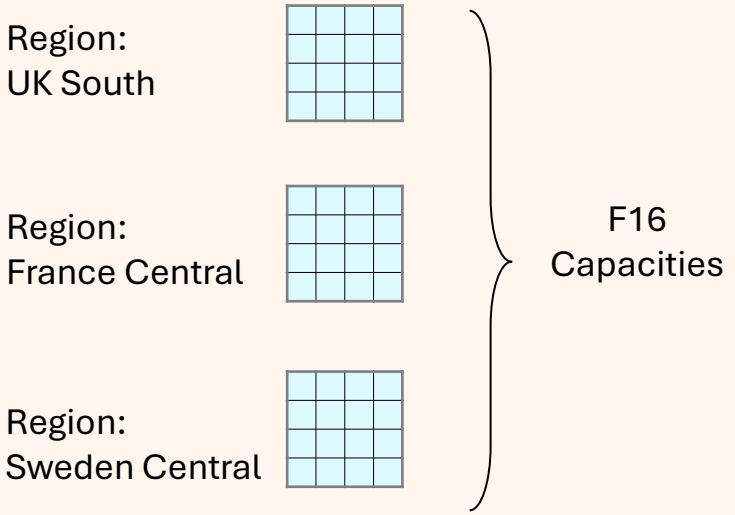
Scale Horizontally

Scale horizontally using the benefits of modular design for hardened isolation and governance



Regional Availability

Use different capacities for different regions to support GDPR / Data residency requirements



Provisioning and Deploying Capacities

Purchased in Azure

- **Purchased** either as a PAYG or RI resource
- **Provisioned with a certain amount of compute** units, analogous to CPU cores.
- The **more capacity units are provisioned, the more load** the capacity can support
  - Multiply SKU size by 30s to match platform evaluation in metrics app
- Capacities are **priced at a fixed hourly rate**, based on capacity units provisioned
- The RI commitment (1-year reserved instance) enjoys a **41% discount**

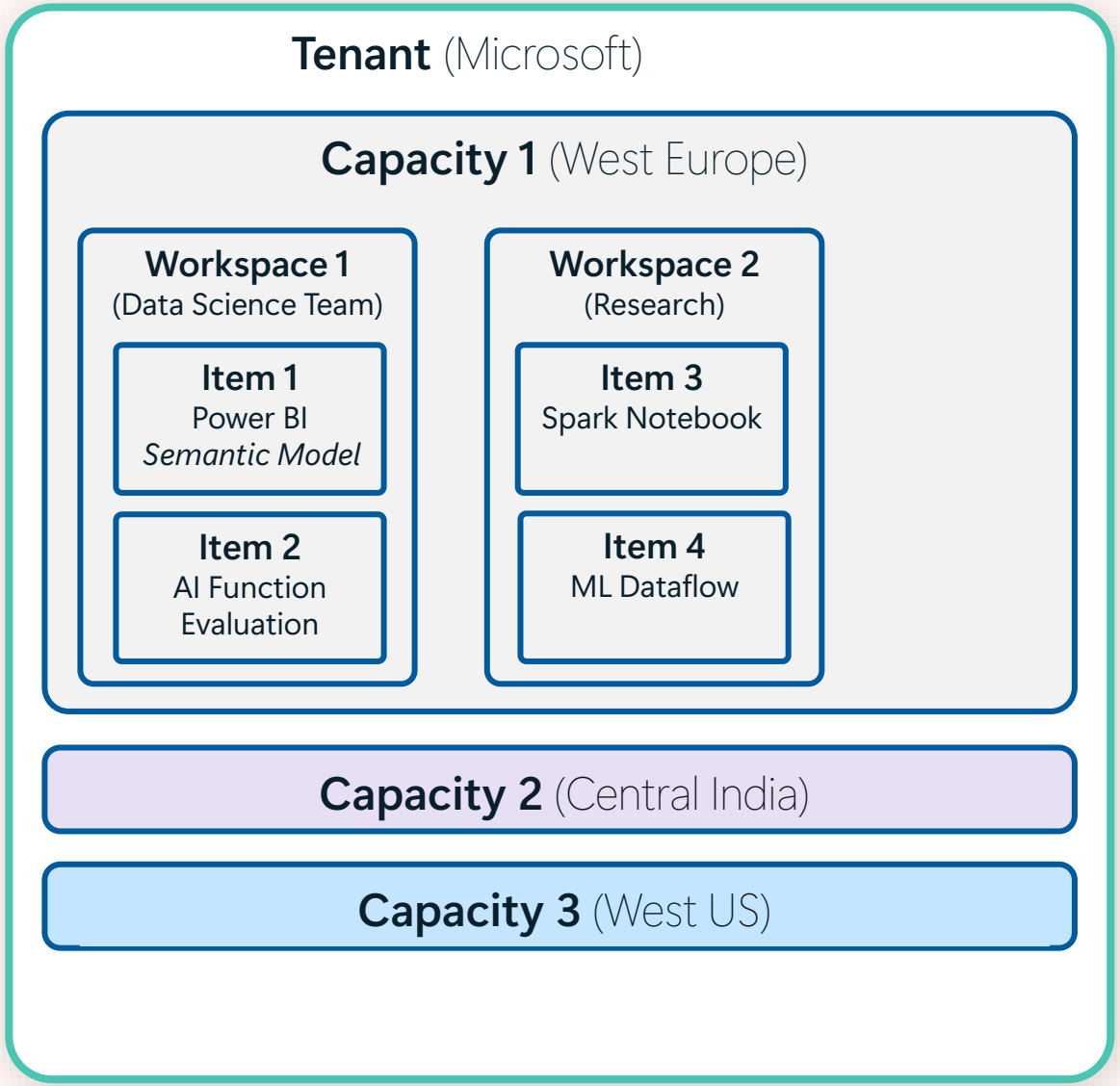
Universal Compute Capacities SKU Sizing

SKU	Capacity Units (CU)	CU's (per 30s)	Power BI SKU	Power BI V-cores
F2	2	60	-	0.25
F4	4	120	-	0.5
F8	8	240	A1	1
F16	16	480	A2	2
F32	32	960	A3	4
F64	64	1920	P1	8
F128	128	3840	P2	16
F256	256	7680	P3	32
F512	512	15360	P4	64
F1024	1024	30720	P5	128
F2048	2048	61440	-	256

Provisioning and Deploying Capacities

Deployed to Regions

- Each capacity **resides in a specific region of the buyers' choice** where both the data & compute reside
- **Workspaces are assigned to a capacity** that provides the compute and storage for all the workspace artifacts
- Multiple capacities can be purchased, deployed and managed by **different owners** residing in a single tenant allowing each business unit to pay for their own consumption



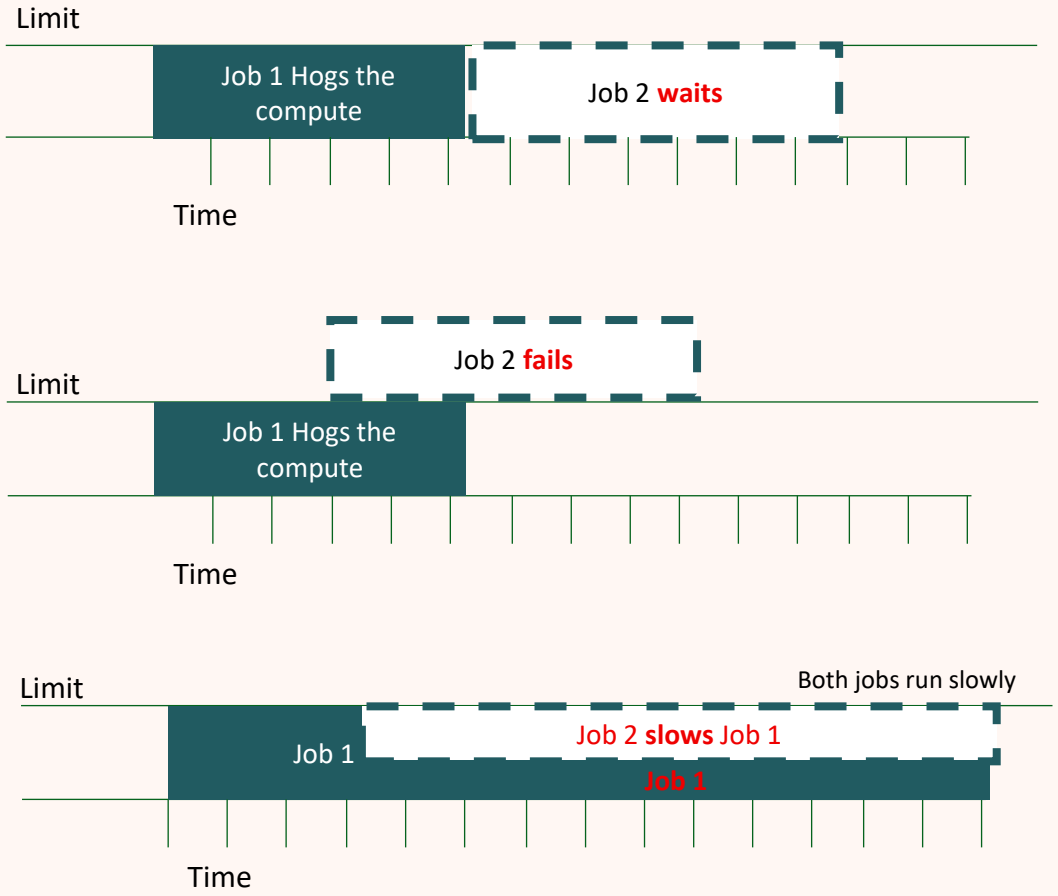
Microsoft Fabric
COMMUNITY CONFERENCE



Bursting and Smoothing

Bursting and smoothing | before and after

‘Traditional’ Systems stay within a hard compute limit



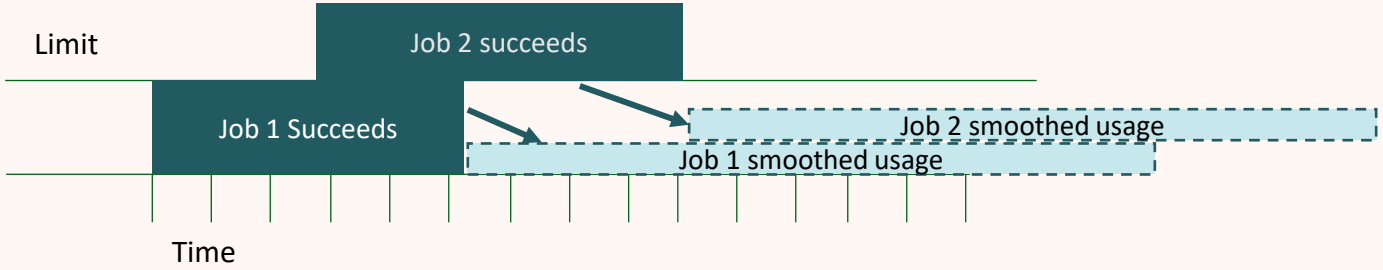
Bursting and smoothing | before and after

Bursting and smoothing help to solve these problems

Bursting lets both jobs **succeed**, running at **peak performance** to finish fast.

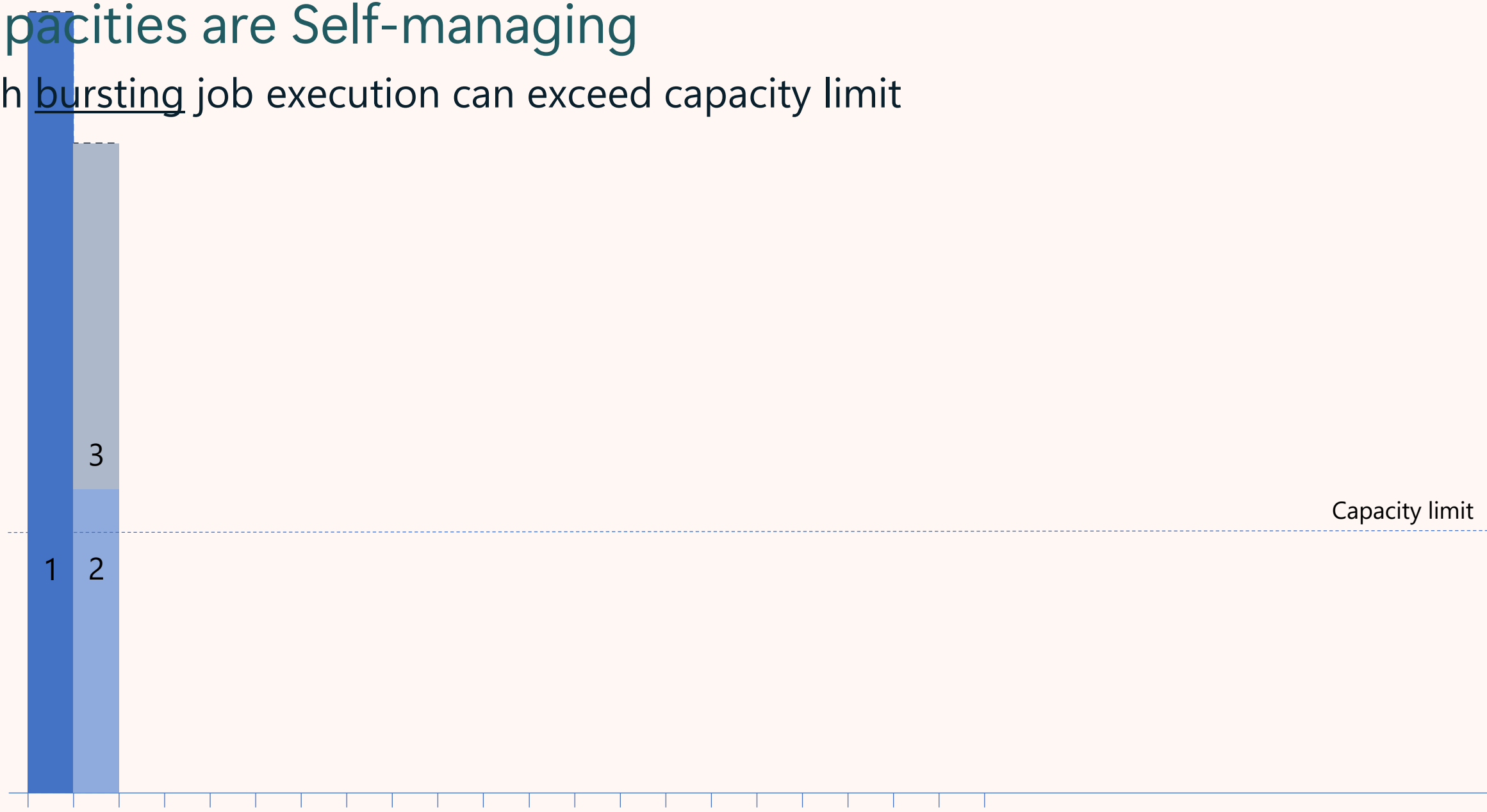


Smoothing distributes the **cost** of jobs over a longer time-period preventing scheduling issues.



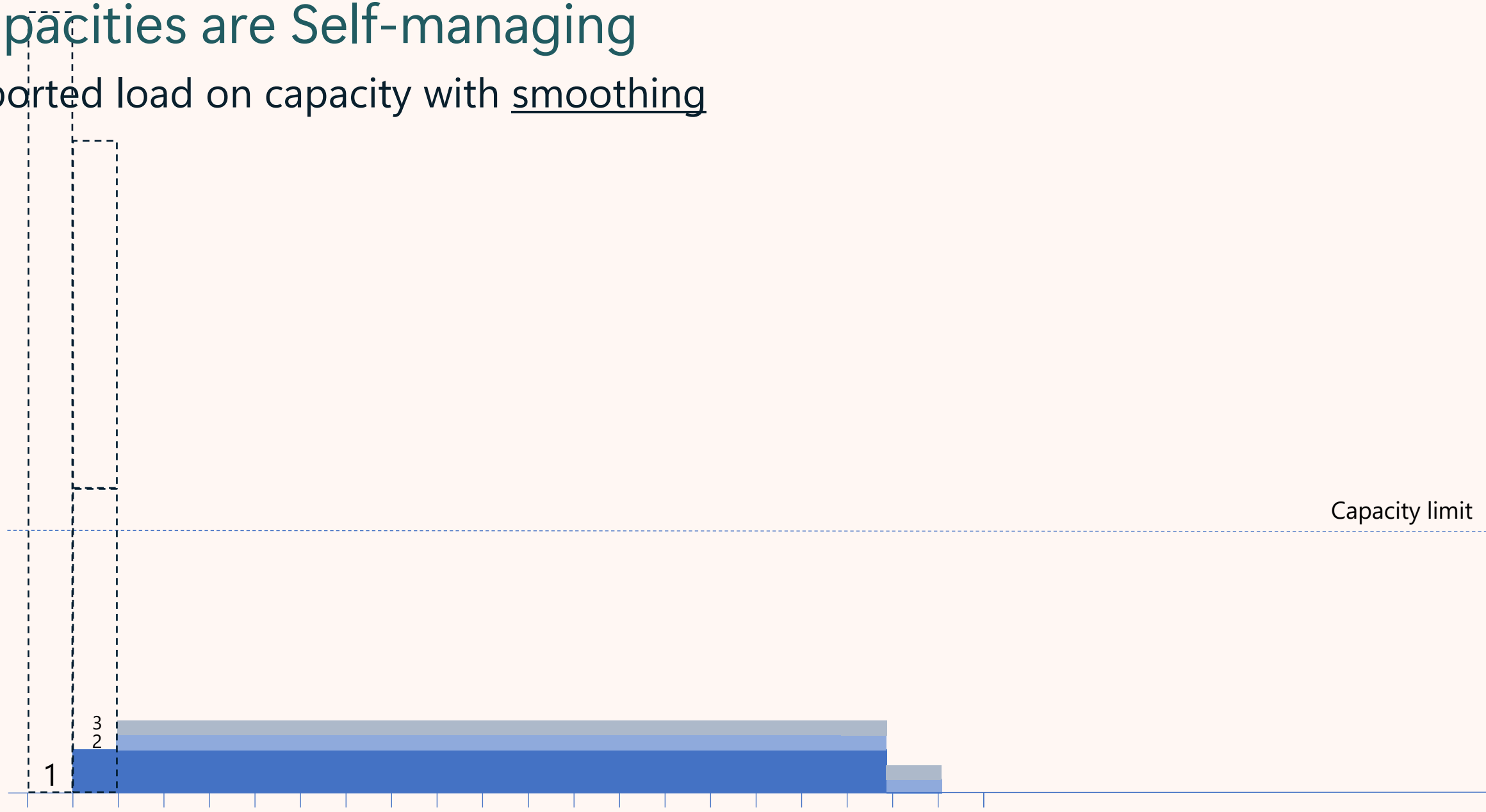
Capacities are Self-managing

With bursting job execution can exceed capacity limit



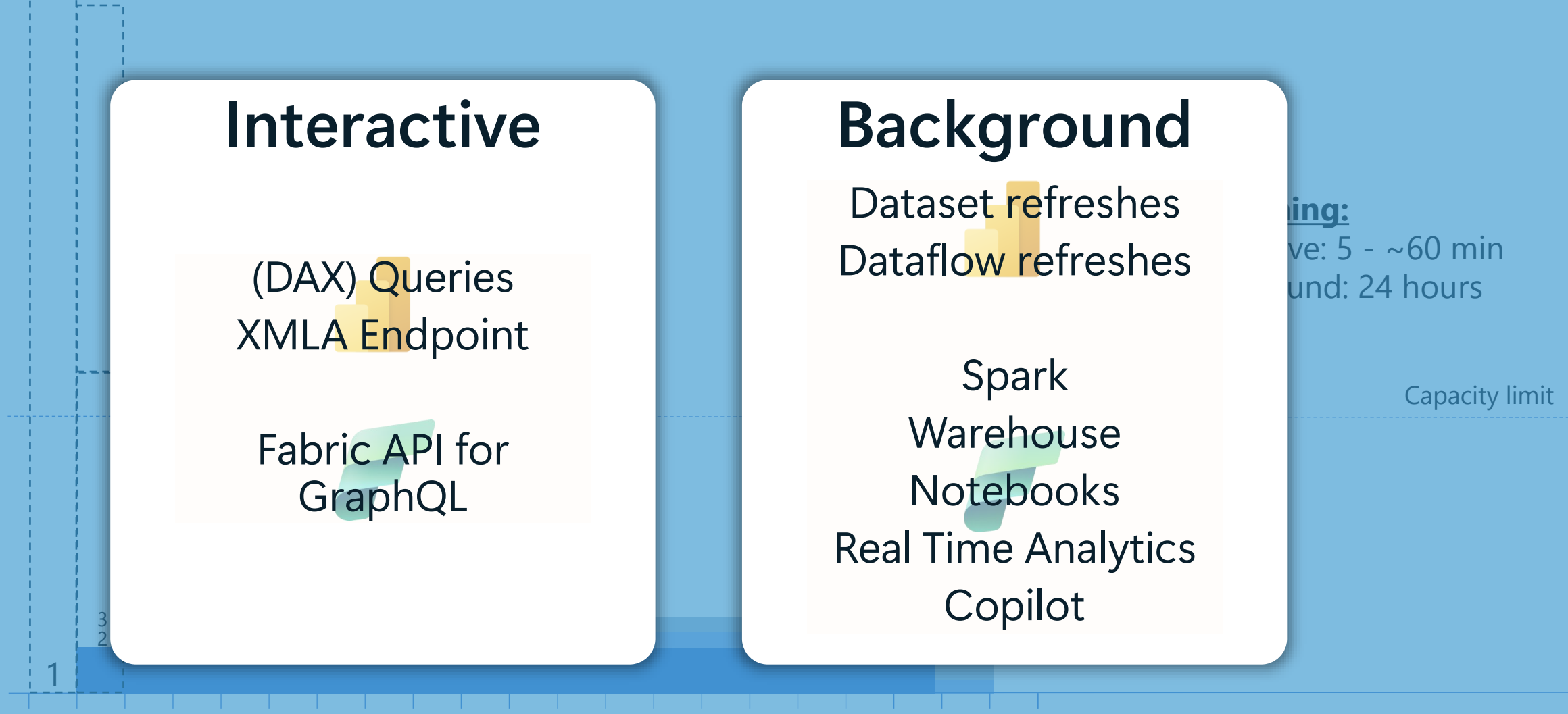
Capacities are Self-managing

Reported load on capacity with smoothing



Capacities are Self-managing

Reported load on capacity with smoothing



Interactive

(DAX) Queries
XMLA Endpoint

Fabric API for GraphQL

Background

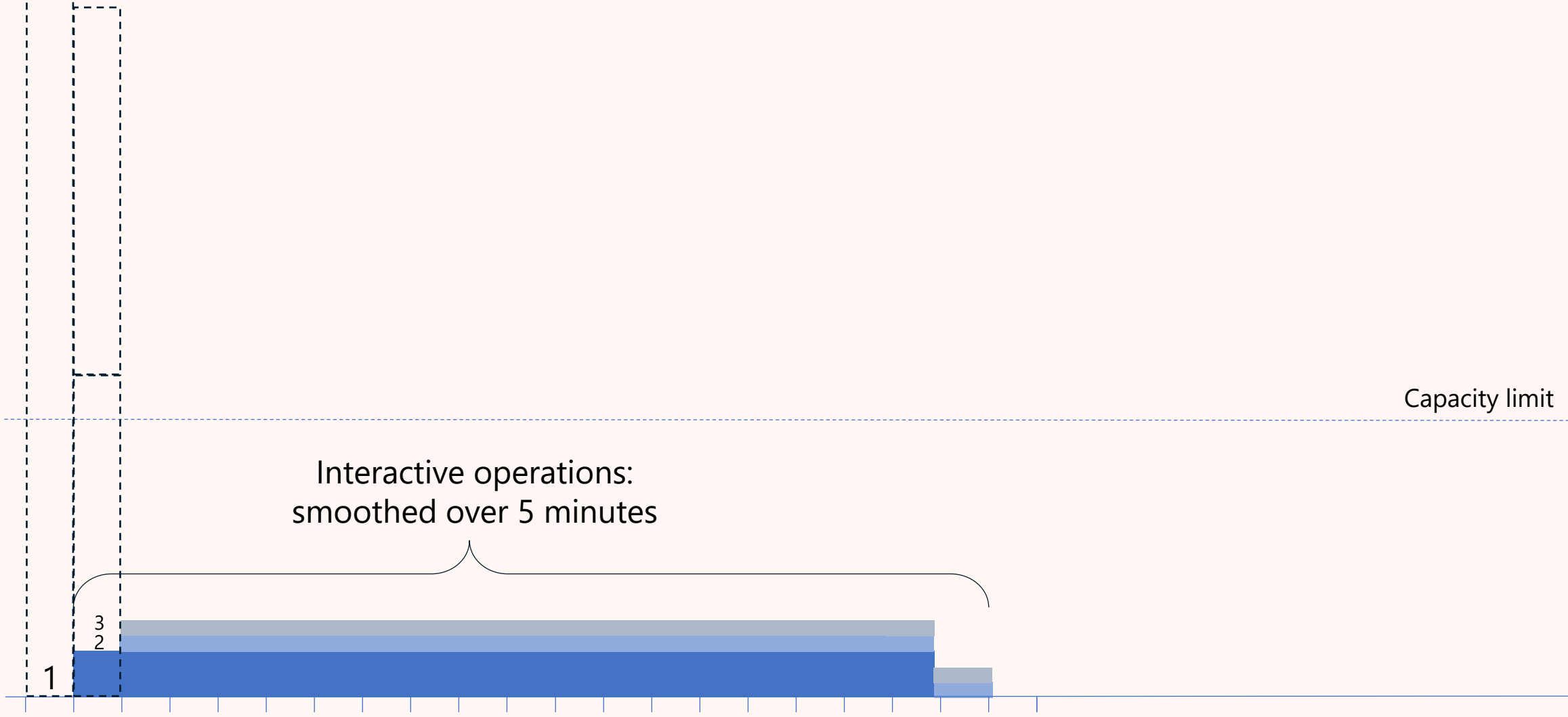
Dataset refreshes
Dataflow refreshes

Spark
Warehouse
Notebooks
Real Time Analytics
Copilot

https://learn.microsoft.com/en-us/fabric/enterprise/fabric-operations

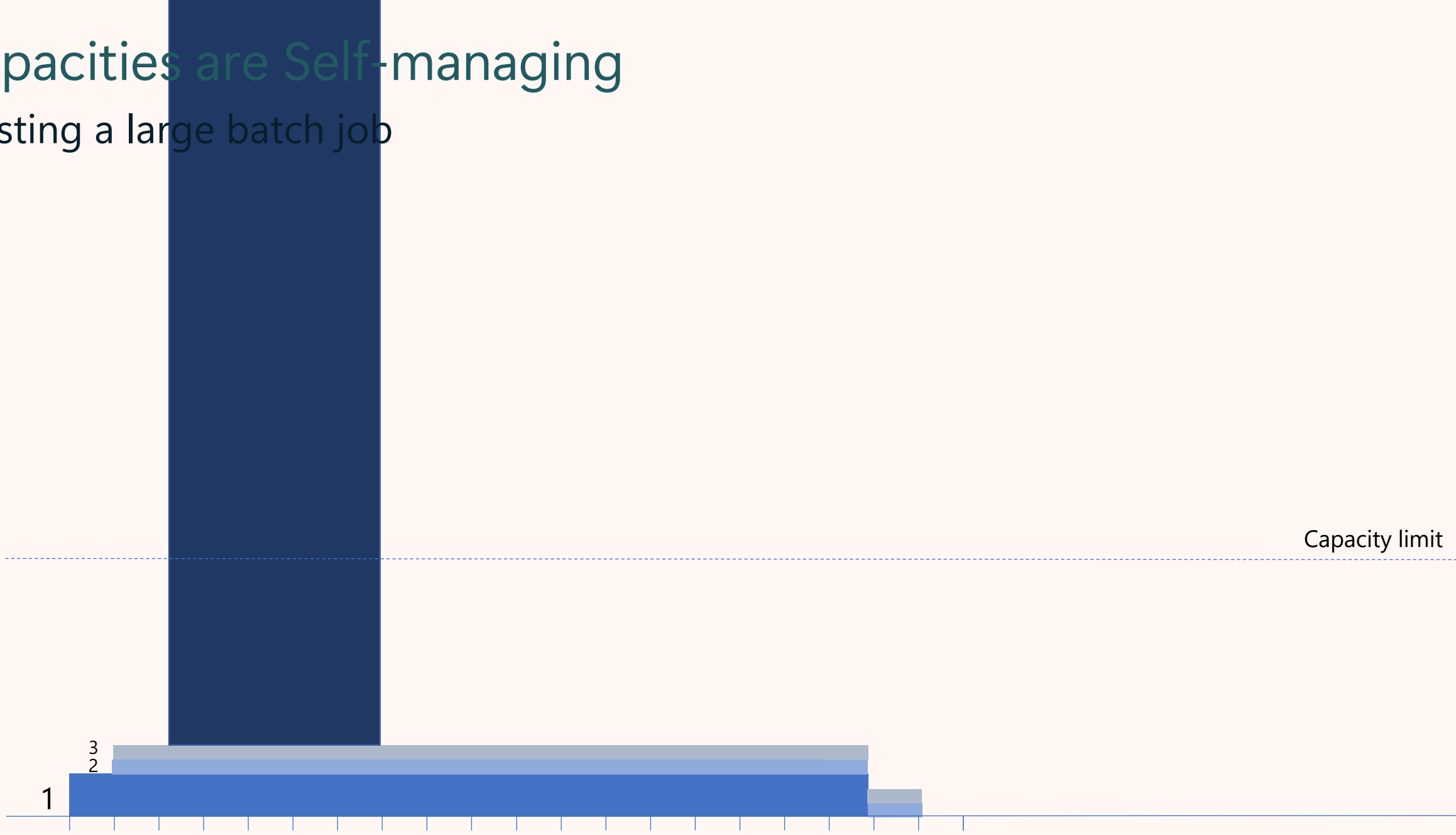
Capacities are Self-managing

Reported load on capacity with smoothing



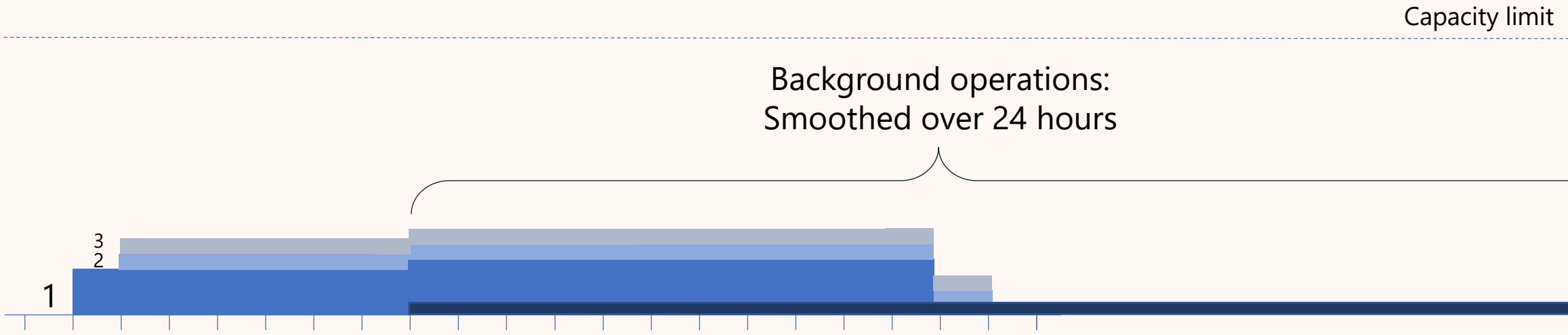
Capacities are Self-managing

Bursting a large batch job



Capacities are Self-managing

Smoothing a large batch job

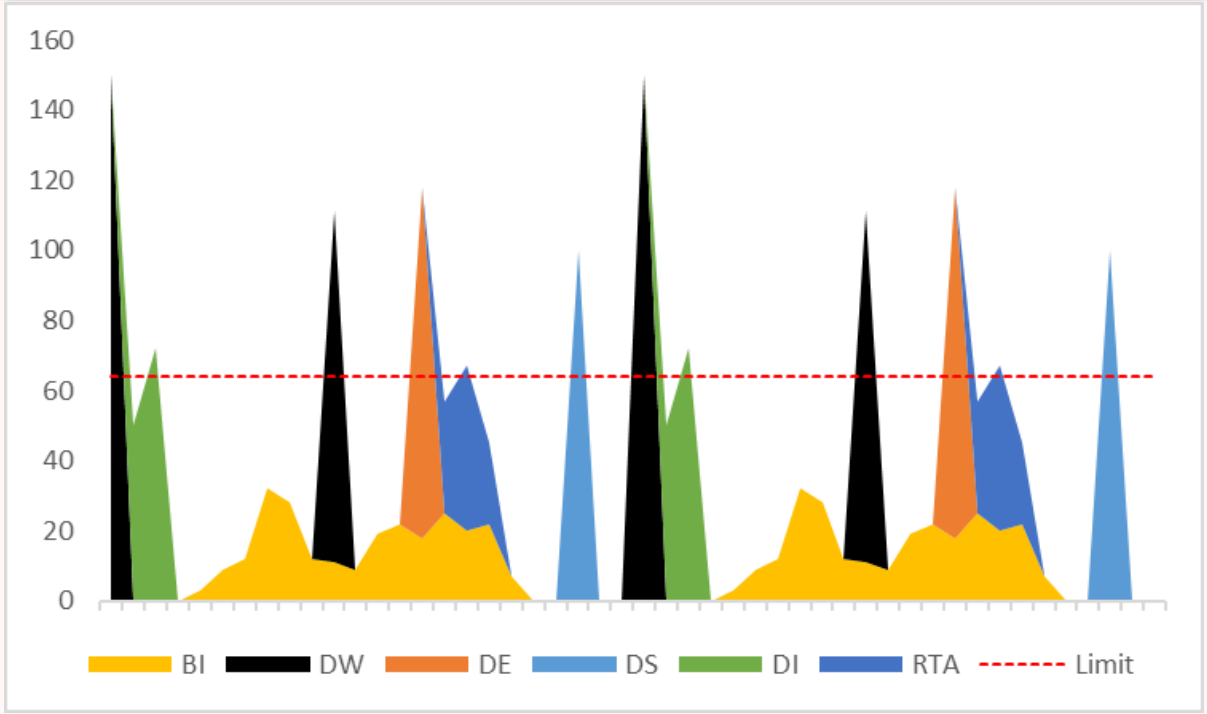


Bursting and smoothing | before and after

Looking at an example of a 64 CU capacity, running multiple workloads over a couple of days…

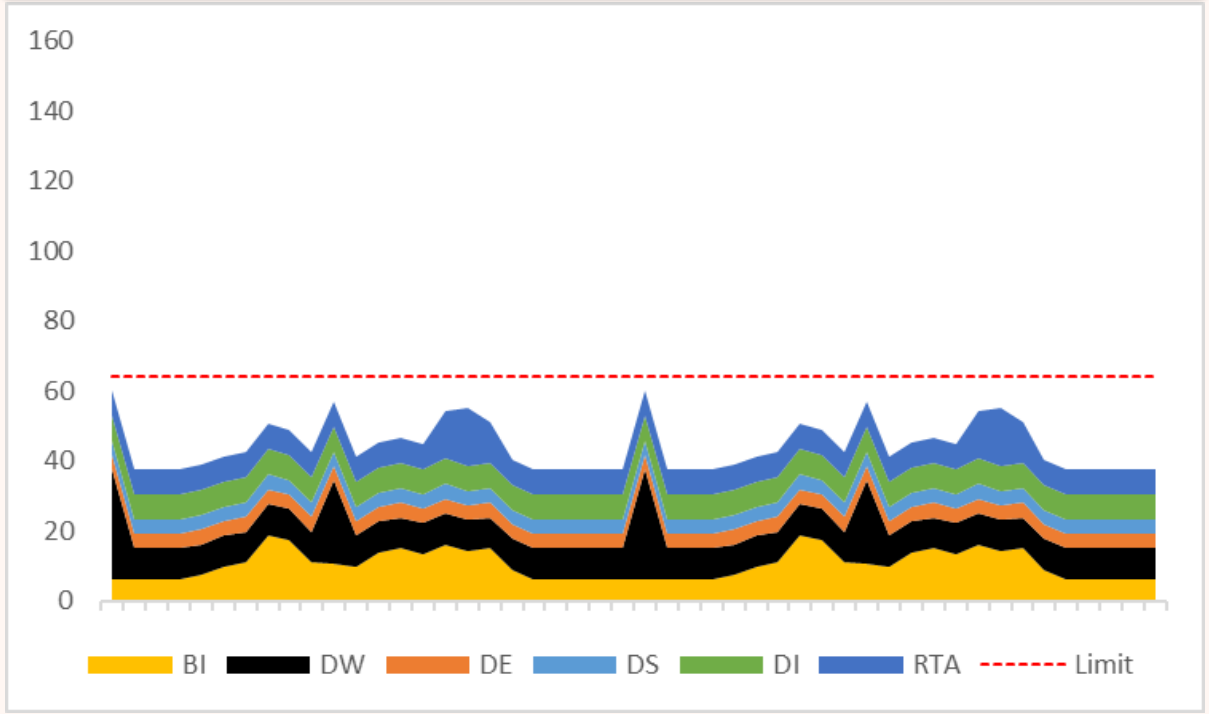
Before Smoothing

- Actual load as executed on the capacity before smoothing
- *Bursting* accelerates jobs execution by resource boosting
- The capacity could be overloaded 25% of the time
- Some of the overloads are more than 2x the limit
- There are periods of no/low usage



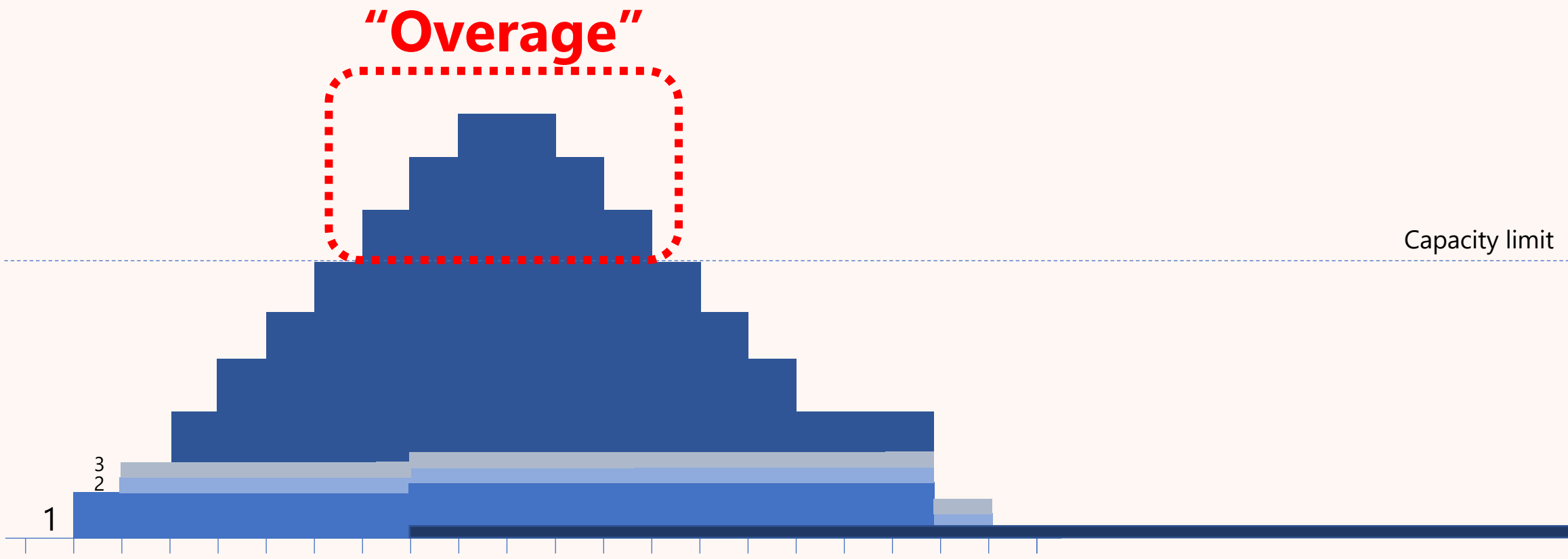
After Smoothing

- Shows the reported load (not runtime execution) against the capacity limits
- There is NO overload, and consumption is more stable
- The smoothing of usage fills in gaps



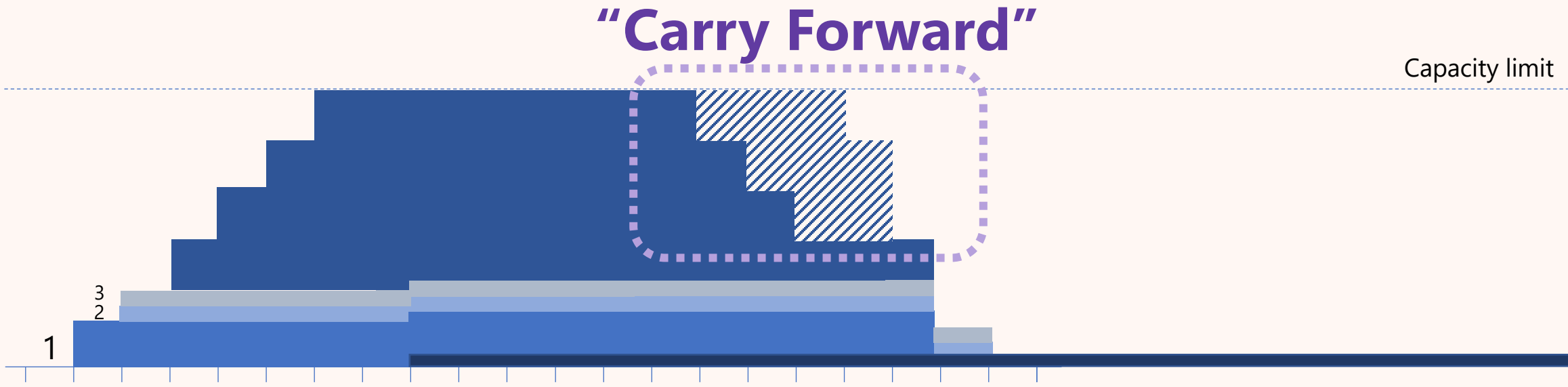
Capacities are Self-managing

What if you keep going over the limit?



Capacities are Self-managing

What if you keep going over the limit?



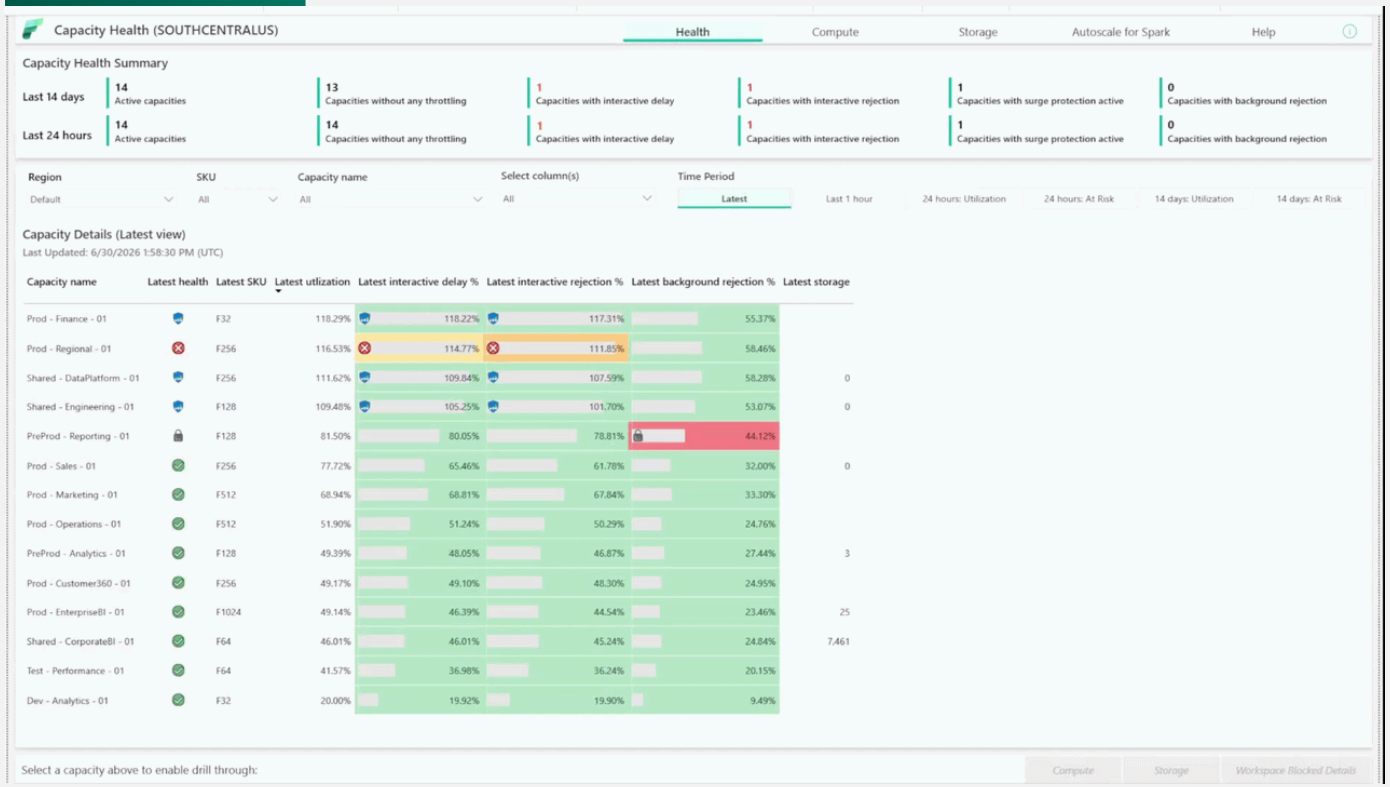
Microsoft Fabric
COMMUNITY CONFERENCE



Monitoring with Capacity Metrics, Monitoring Hub

Capacity Metrics App – New Look & Capabilities

Public preview



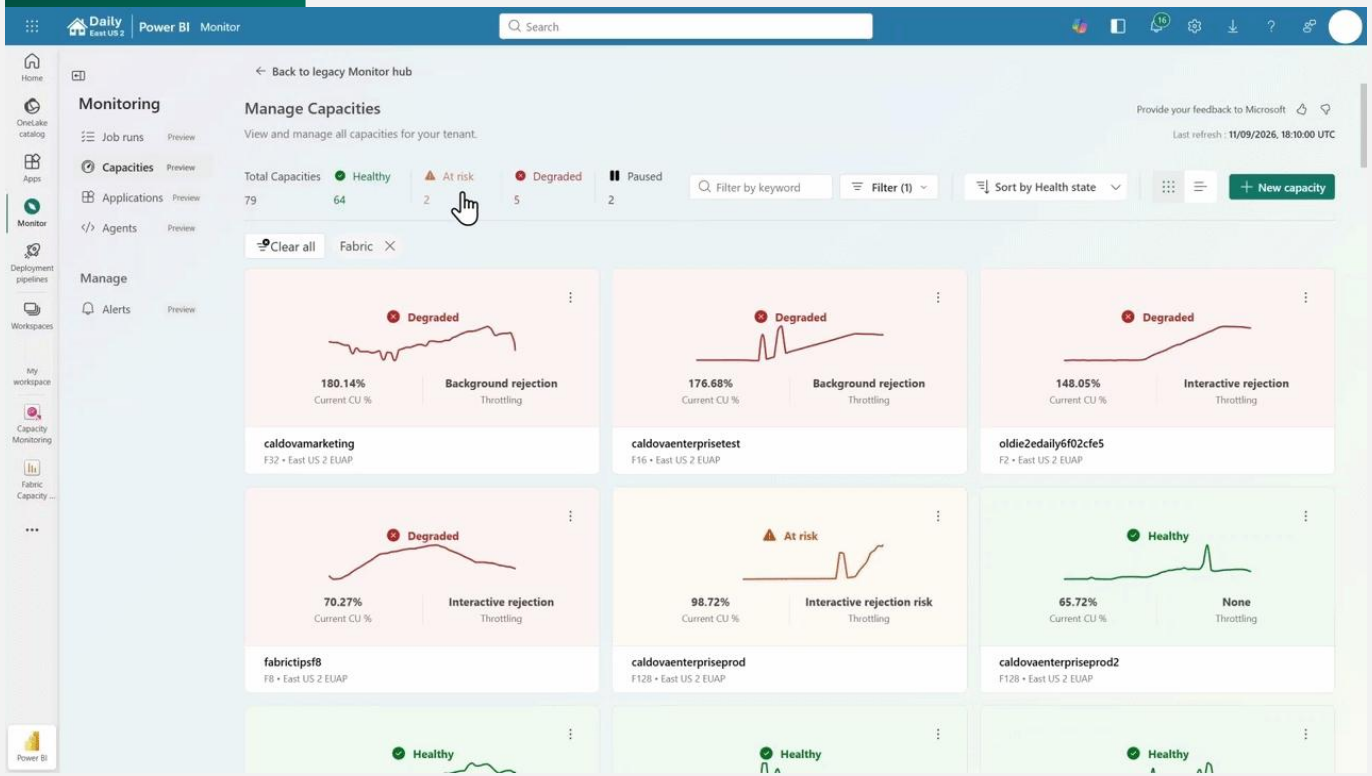
New Views: Including heatmap to identify daily & hourly trends; new CU table showing high item contributors

New UX/UI: Enhanced UX brings key metrics and insights to surface, allows for easy drill down; cleaner, modern UI

Simplified Metrics: Time spent in health state, CU / sec reporting for analysis, other changes per real-world feedback

Capacity Insights & Actions in Monitor Hub

Public preview



In-product observability for capacity admins with at-a-glance health view to focus on where action is needed

Take in-line actions right where issues are observed: adjust overage & surge settings, migrate workspaces, resize*

Top-to-bottom observability across altitudes and personas including applications, jobs, agents all in one UX

* Requires capacity admin to have Azure subscription permissions

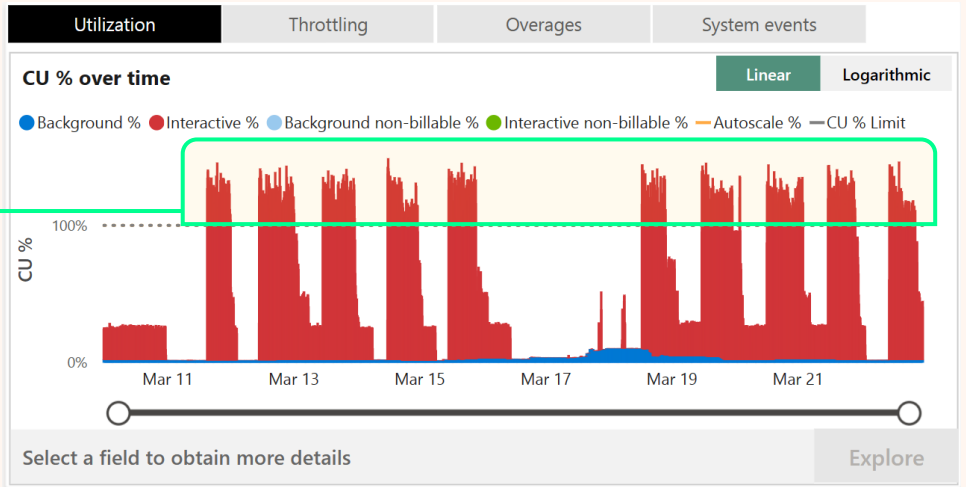
Microsoft Fabric
COMMUNITY CONFERENCE



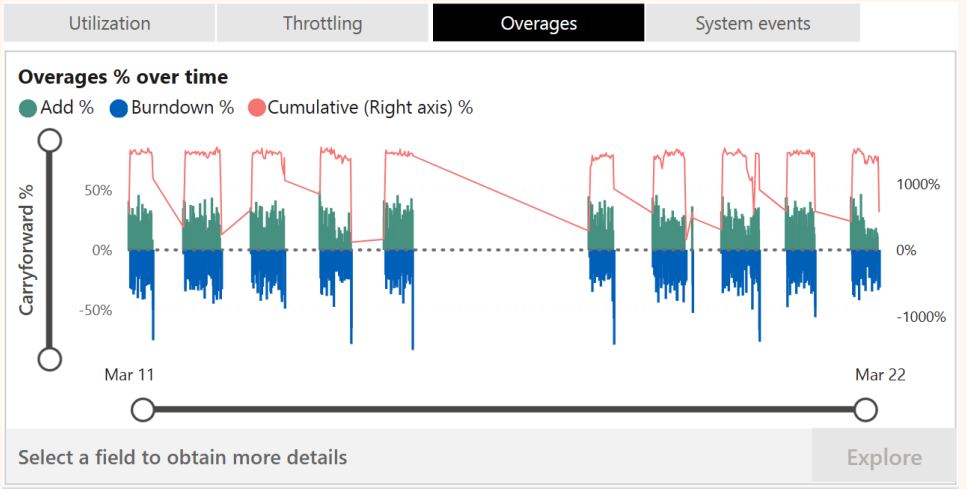
Capacity Throttling Policies

Throttling intro

- Throttling is the platform policy for managing consumption that exceeds throughput is provided by SKU choice
- When workloads exceed the throughput of a capacity a cumulative debt is tracked to be burned down
- Cumulative debt is used to determine throttling policies and is burned down when resources are free

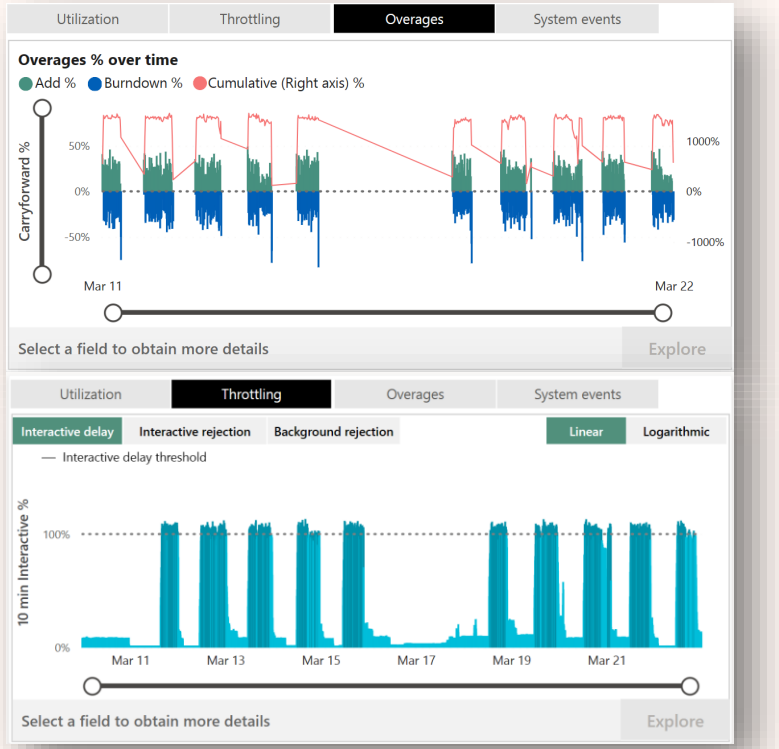


Overage Operation	Description
Overages - Added	• Timepoint when job requests exceed the throughput of a capacity, overages are was added to the cumulative buffer to burn down. • This graph simplifies identification of the optimal timepoint to load timepoint drill to analyze the user operations that contributed to an overage.
Overages - Burndown	• Overages being reconciled when future capacity is free to burn down
Overages - Cumulative	• The total amount of queued work on the capacity to be burned down in the future when the capacity is not fully utilized



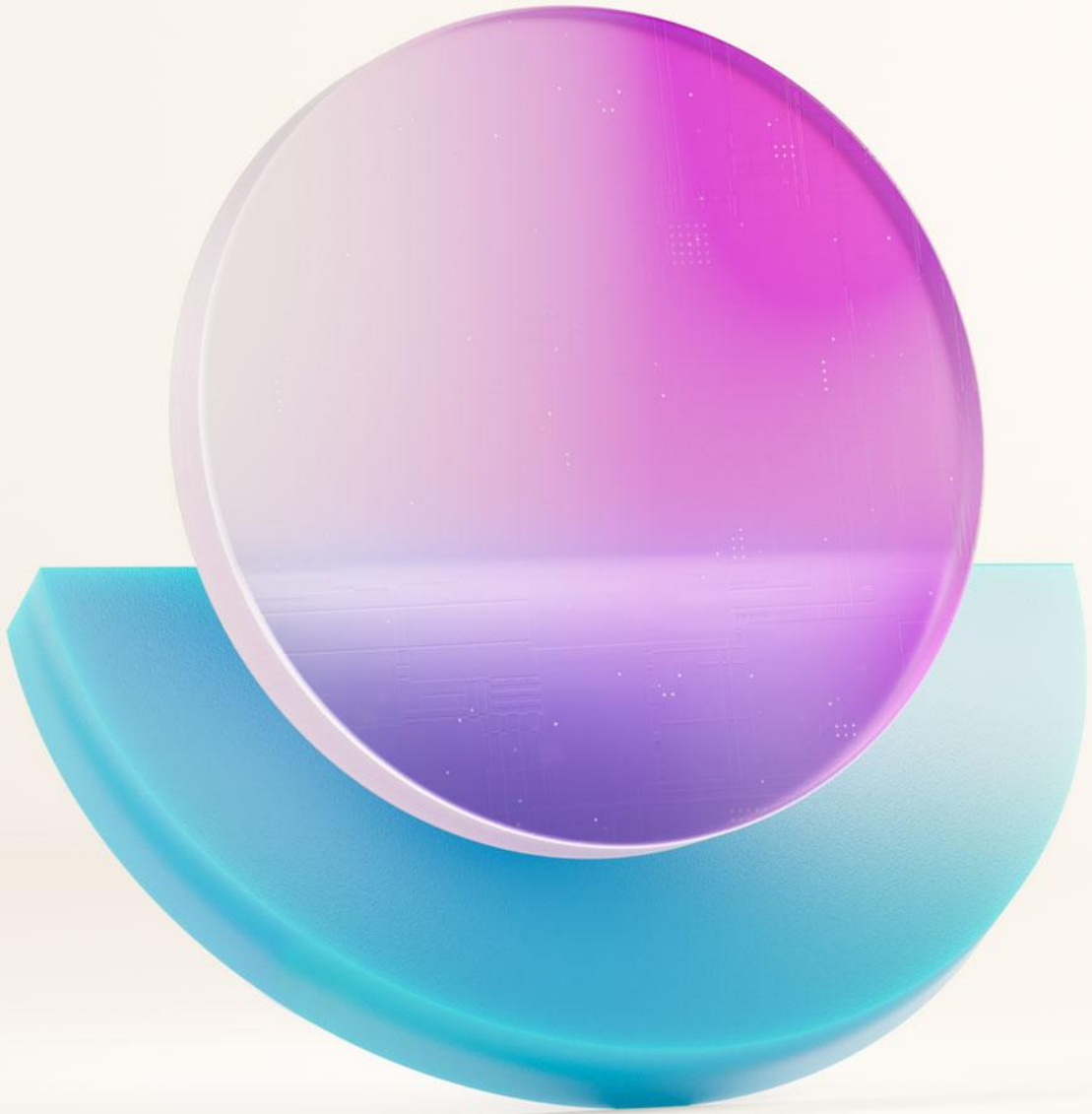
Capacity throttling evolution for Fabric

- For Fabric, throttling policies were refined to deliver multiple benefits
  - **Reduced throttling** for capacities that only experience occasional spikes
  - **Added overage protection** – rejection policies prevent overloaded capacities from irrecoverable overload
  - **Optimizations for long-running jobs:** We're optimizing the platform for long-running jobs, so if a job exceeds capacity limits, it will run to completion and the overage will be burned down against future capacity



Smoothed Capacity - Future Use	Platform Policy	Customer Impact
<= 10m	Overage Protection	Jobs can consume 10 minutes of future capacity use without throttling
> 10m → <= 60m	Interactive Delay	User requested interactive type jobs will be throttled
> 60m → <= 24h	Interactive Rejection	User requested interactive type jobs will be rejected
> 24h	Background Rejection	User Scheduled background jobs will be rejected from execution

Microsoft Fabric
COMMUNITY CONFERENCE



Capacity Planning

https://aka.ms/FabricCapacityPlanning

Fabric SKU Estimator

GA

Enabling customers to better estimate their SKU before purchase

Online calculator

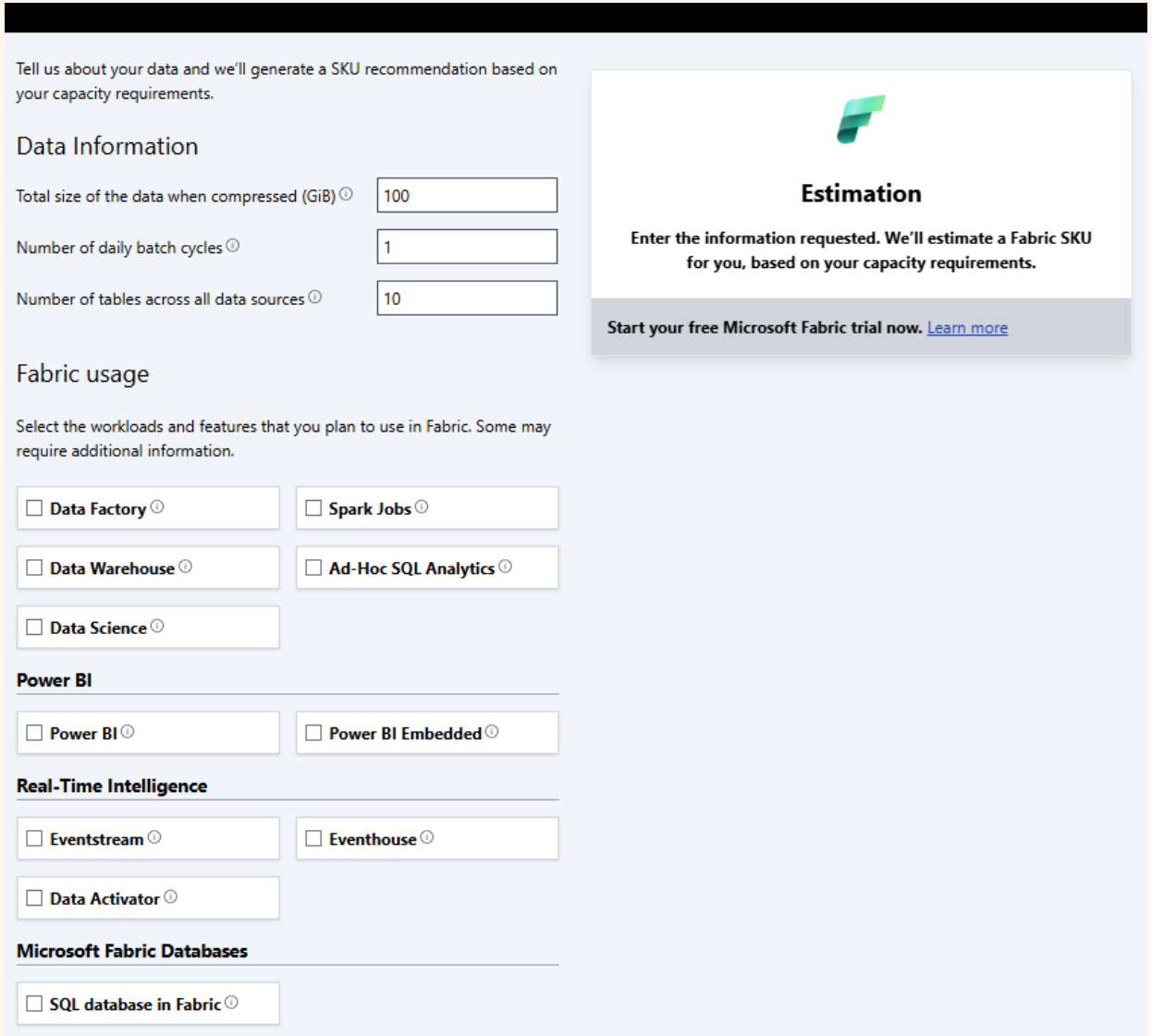
- Provides capacity estimations customized to your unique requirements
- Results vary depending on your input

Impact

- Recommended SKU when starting on Fabric
- Test solutions to ensure they're correctly sized.

Learn more?

- Fabric Tech Talks - Estimating Capacity in Microsoft Fabric
- Blog – Master SKU Estimations



https://www.microsoft.com/en-us/microsoft-fabric/capacity-estimator

ESTIMATING, PLANNING

OR .. WHAT IS *GOOD ENOUGH?*

Define what enough capacity means

Translate expectations into measures the business and operators can evaluate.

Processing	What must finish, by when, and with what freshness?
Consumption	What response is acceptable at expected concurrency?
Critical periods	When is disruption unacceptable - and who decides?

Agree the service target and the critical period before selecting a capacity size.

Inventory work, not just data and users

A useful demand profile describes what runs, how often, and what overlaps.

WORK	INPUT TO CAPTURE	COMMON BLIND SPOT
Ingest + transform	Data change, runs, duration	Overlapping processing windows
Refresh + train	Frequency, model/data size	Retries and repeated full work
Query + consume	Concurrency, query pattern	Averages hide busy periods

For every major operation, record volume, frequency, concurrency, dependencies and growth.

Choose the estimation path that fits your evidence

Use the strongest evidence you have - and expose the gaps.

EXISTING FABRIC

Measure what runs

Consumption + experience.
Include busy periods.
Inspect the operation mix.

MIGRATION

Profile, then test

Use source workload facts.
Test representative behavior.
Avoid one-to-one SKU mapping.

NEW SOLUTION

Estimate, then pilot

Use the SKU Estimator.
Document assumptions.
Validate before production.

Fabric Tech Talks - Estimating Capacity in Microsoft Fabric
Blog – Master SKU Estimations

Do not treat another platform's compute size as a direct Fabric SKU conversion.

Separate normal demand, peaks, and growth

One average and one growth multiplier are not a capacity forecast.

NORMAL DEMAND

A representative day

Regular processing.
Expected usage pattern.
Known concurrency.

BUSINESS PEAK

A critical period

Closing or seasonal work.
Heavier overlapping demand.
Tighter completion windows.

GROWTH

The next planning horizon

More data or frequency.
More concurrent activity.
New solutions entering the pool.

Change the relevant drivers: data processed, operation frequency, concurrency and new workloads.

Check what utilization cannot tell you

A lightly used capacity can still be unsuitable for a specific workload.

01

Workload limits

Model size, memory and SKU-specific thresholds.

02

Licensing + features

Required entitlements and supported workload features.

03

Region + quota

Availability, residency needs and capacity expansion readiness.

04

Dependencies

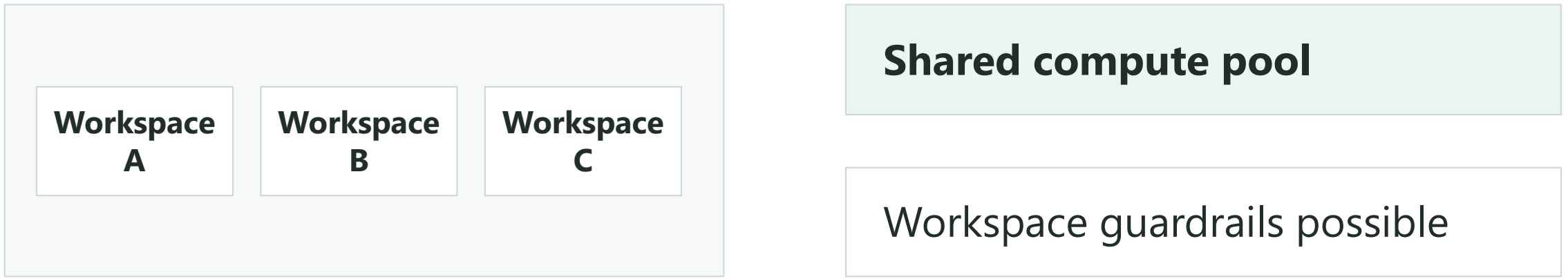
Shared models and upstream work that may constrain the experience.

Select the smallest suitable SKU - not simply the smallest SKU with an appealing average.

WORKSPACE PLANNING

Logical boundaries are not compute boundaries

Separate workspaces on the same capacity still share its compute pool.



Workspace controls can add guardrails; they do not create a separate dedicated compute pool.

Choose workspace granularity deliberately

Group content that belongs together operationally - not just organizationally.

TOO BROAD

Hard to change safely

Unclear accountability.
Different access needs collide.
A change affects too much.

DELIBERATE GROUPING

A coherent unit of work

Compatible ownership.
Shared release and support.
Explicit dependencies.

TOO FRAGMENTED

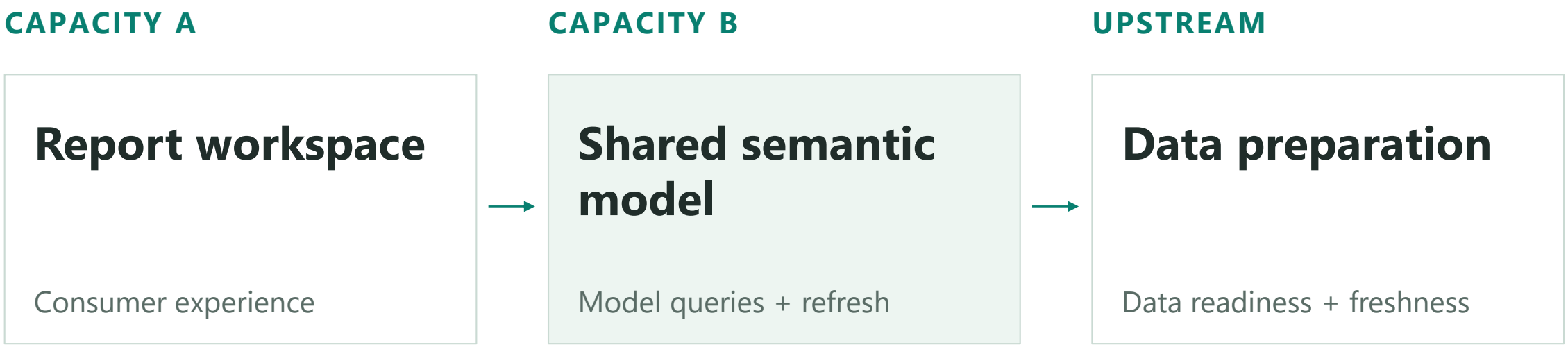
Hard to operate simply

More permission surfaces.
More cross-workspace links.
More moving pieces.

Use ownership, access, release cadence and support needs to justify a workspace boundary.

Trace where the work actually executes

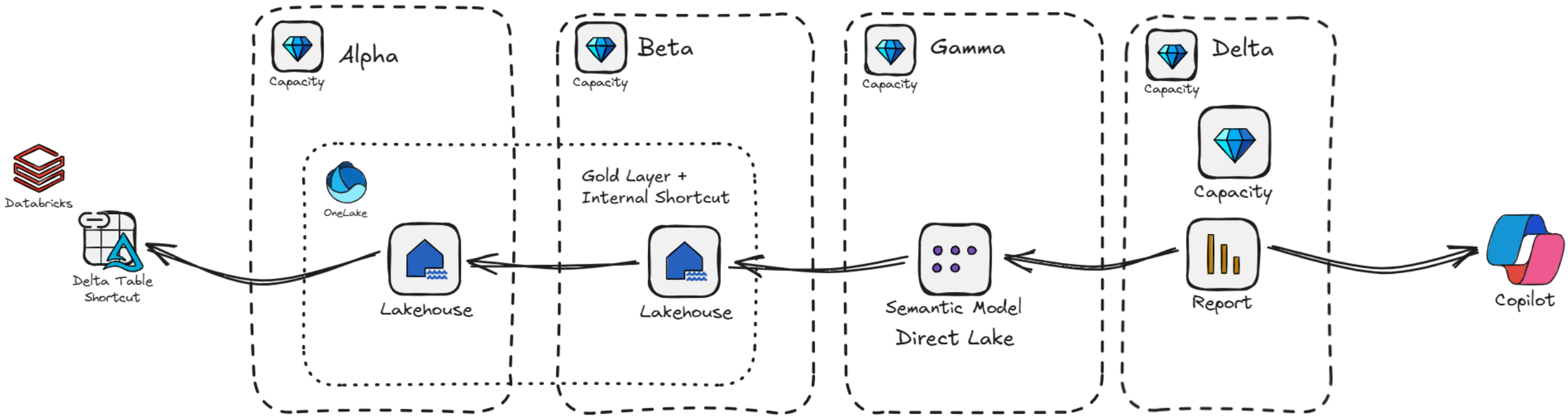
Moving the visible report is not the same as moving its important compute demand.



Read left to right: identify what each layer depends on.

Inspect both the report and model capacities - then follow the upstream processing dependencies.

EXERCISE: WHERE DO CU'S FLOW?



User asks Copilot (Sidepanel) to Summarize the Report,
based on a Direct Lake semantic model,
bringing together data from two different Gold Lakehouses

- Copilot usage (Copilot in Fabric, background) gets attributed to the Report (Delta)
- Semantic Model usage (Query, interactive) gets attributed to the Semantic Model (Gamma)
- OneLake usage (OneLake Read, background) gets attributed to Beta for both Lakehouses
- External Shortcut usage (OneLake, background) gets attributed to Alpha

Decide which workspaces can safely share

Share because workloads are compatible - not simply because a pool already exists.

Service compatibility	Similar criticality, deadlines and tolerance for disruption.
Demand compatibility	Measured combined usage, growth and sufficient headroom.
Deployment compatibility	Region, limits, dependencies and accountable support.

Different schedules can help; smoothing means timing alone does not prove safe sharing.

CAPACITY PLANNING

Design the capacity estate around service needs

Topology changes utilization, isolation, limits and operating effort.

SHARED

Consolidate compatible demand

Higher pooled utilization.
More contention exposure.
Needs admission guardrails.

DEDICATED

Protect distinct service needs

Separate compute demand.
Potential spare resources.
More operating responsibility.

HYBRID

Use both intentionally

Protect the critical path.
Share compatible workloads.
Reassess as demand evolves.

One large pool and several smaller pools with the same total CUs are not automatically equivalent.

Bring a symptom, a decision, or a constraint

Use the framework to find the next useful action - not to guess an SKU.

SYMPTOM

What is happening?

Who is affected?
When does it occur?
What evidence is available?

DECISION

What must change?

Size, placement or policy?
Which trade-off matters?
What would justify the choice?

CONSTRAINT

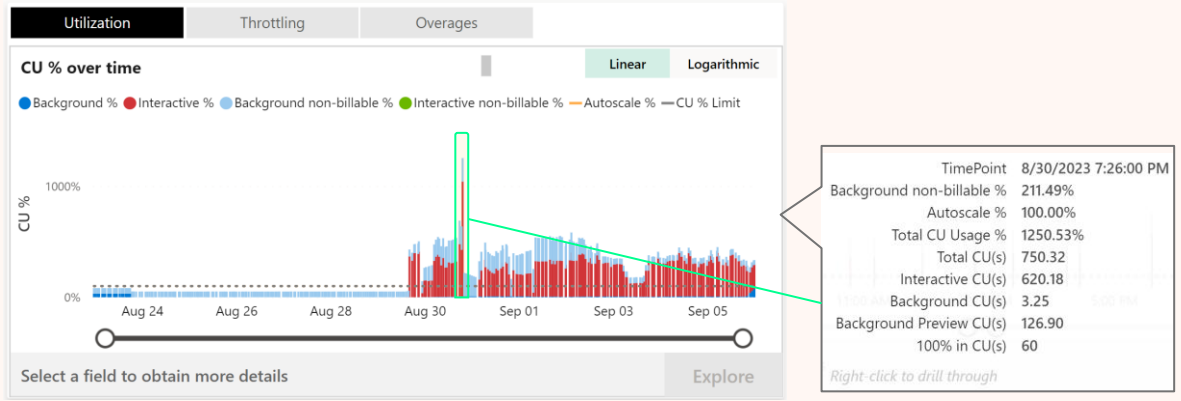
What is limiting you?

Workload limit or dependency?
Quota, license or region?
Authority or budget?

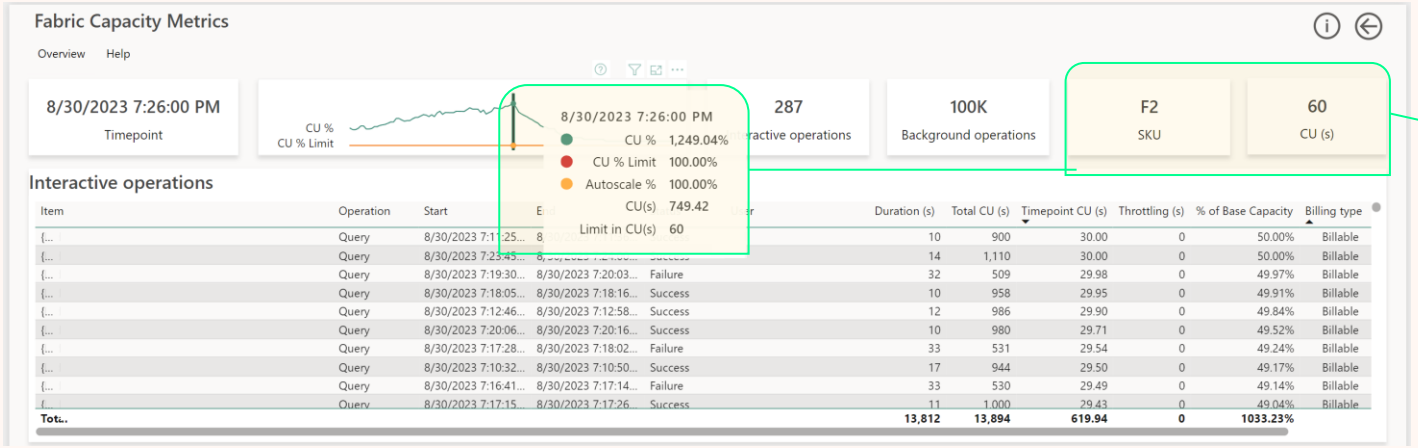
What must the service achieve? What evidence exists? Where does the work run? Who owns the decision?

Capacity planning case study - measurement

Start with a test or trial capacity to evaluate the load of specific Fabric Experiences i.e., Power BI Datasets, Spark Notebooks or a Datawarehouse



If usage is above the current capacity limits , choose the desired utilization rate to accommodate via capacity scale up



Load Capacity Metrics timepoint drill to analyze :

- Total CU's consumed : 749 CU(s)
- Capacity Size : (F2)
- CU(s) available on your capacity : 60 CU(s)

Capacity planning case study – SKU selection

To accommodate a 749 CU(s) load the admin can purchase an F32 capacity providing 960 CU(s) of throughput

Universal Compute Capacities SKU Sizing

SKU	Capacity Units (CU)	CU's (per 30s)	Power BI SKU	Power BI V-cores
F2	2	60	-	0.25
F4	4	120	-	0.5
F8	8	240	A1	1
F16	16	480	A2	2
F32	32	960	A3	4
F64	64	1920	P1	8
F128	128	3840	P2	16
F256	256	7680	P3	32
F512	512	15360	P4	64
F1024	1024	30720	P5	128
F2048	2048	61440	-	256

Fabric Capacity Reservations

Commit to spending for 1-year periods, to get a 41% discount

Existing Azure Concept

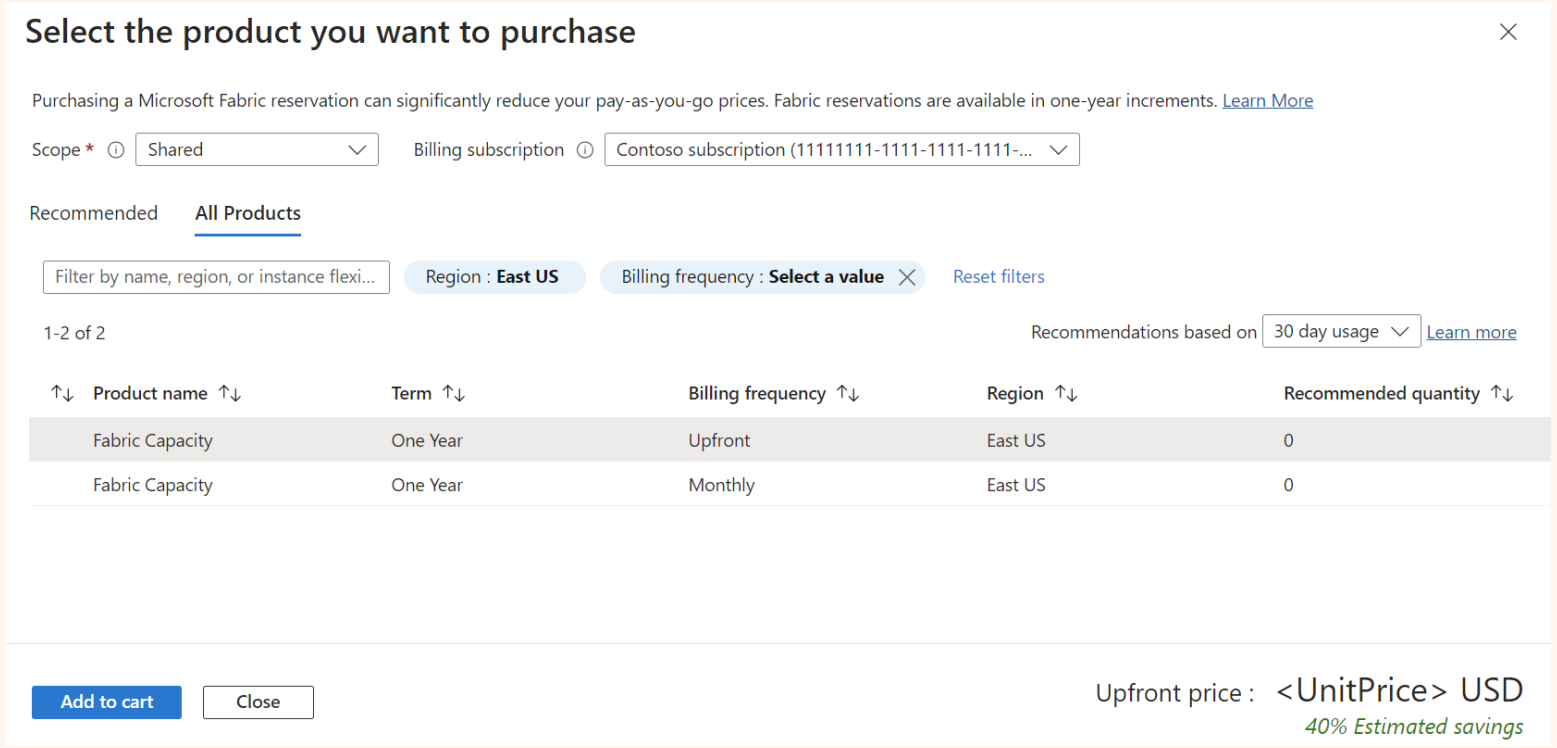
- Cancel, Refund, Exchange by policies

Reservations can be configured by

- Scope:
  - Billing Account
  - Subscription(s)
  - Resource Group(s)
  - Region
- Management Group

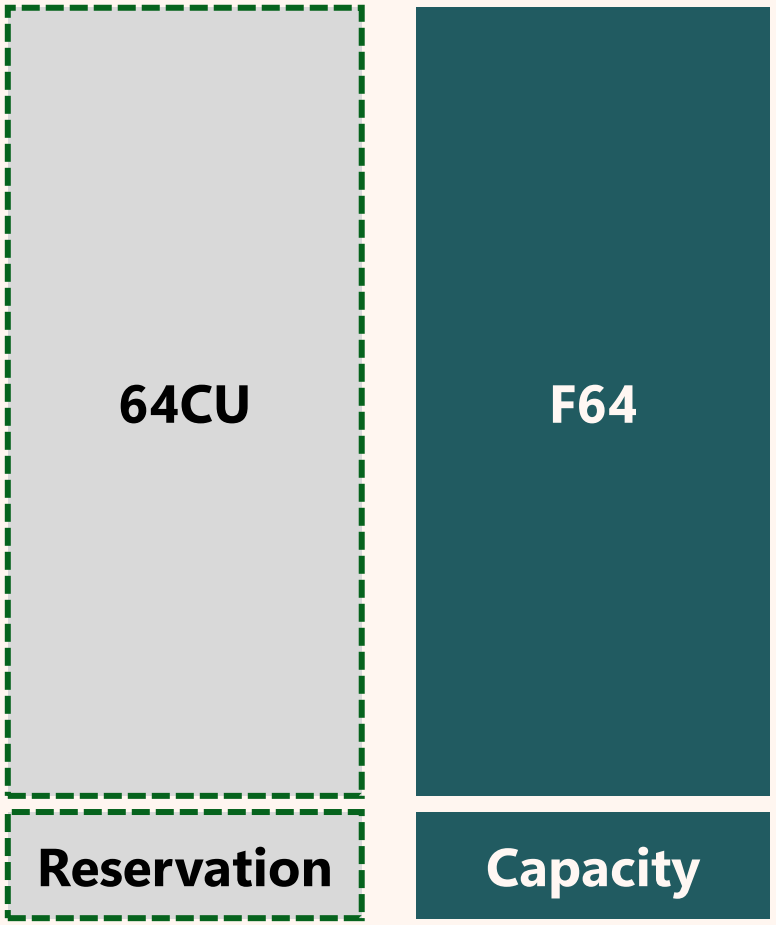
Reservation that is enforced

- Even when no capacity matches the scope, billing happens
- Not automatically renewed, unless configured
- Upon expiry, capacities impacted automatically switch to PayGo



Fabric Capacity Reservations

Example 1 – Single Reservation matching a single capacity



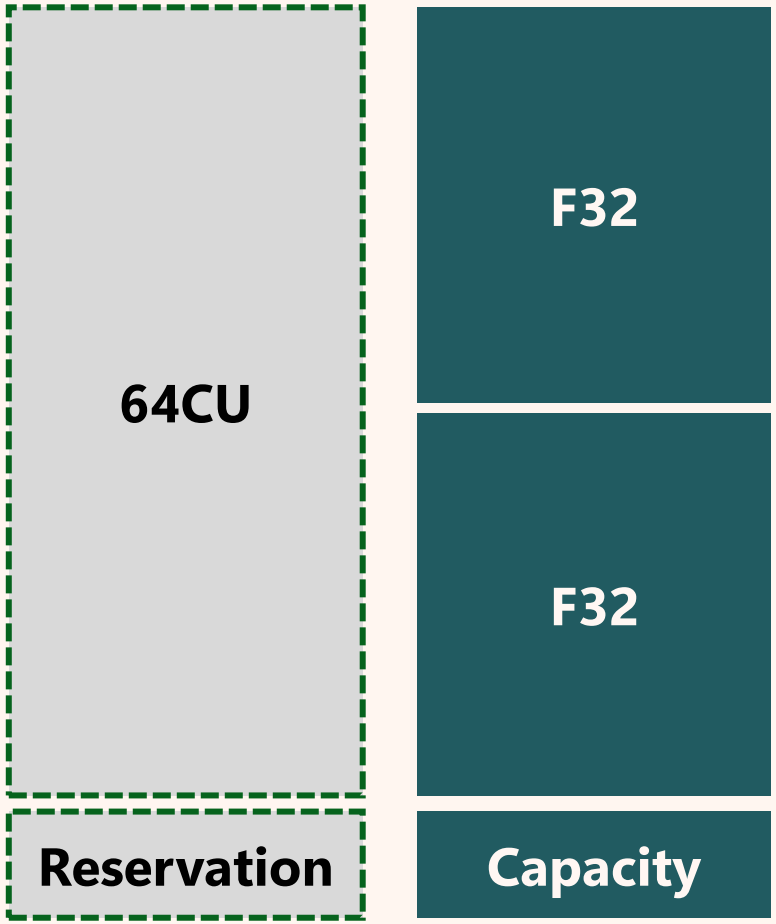
Without a reservation, you pay-as-you-go

Active capacity is covered by Reservation*, discount (~41%) applies

*) Billing Account / Subscription / Resource Group / Region Need to align

Fabric Capacity Reservations

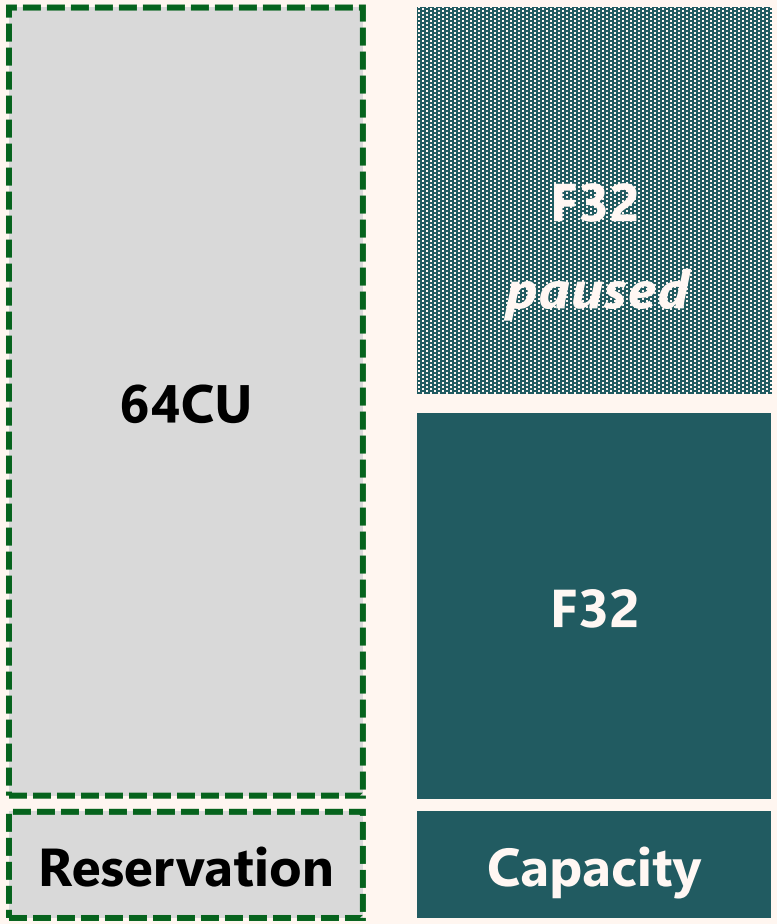
Example 2 – Single Reservation matches active capacities



All active capacities are covered by Reservation, discount applies

Fabric Capacity Reservations

Example 3 – Single Reservation exceeds Active Capacities



Reservation only applies to active capacity, 32CU are "wasted"

Reservation sets the floor

Fabric Capacity Reservations

Example 4 – Single SKU exceeding the reservation amount

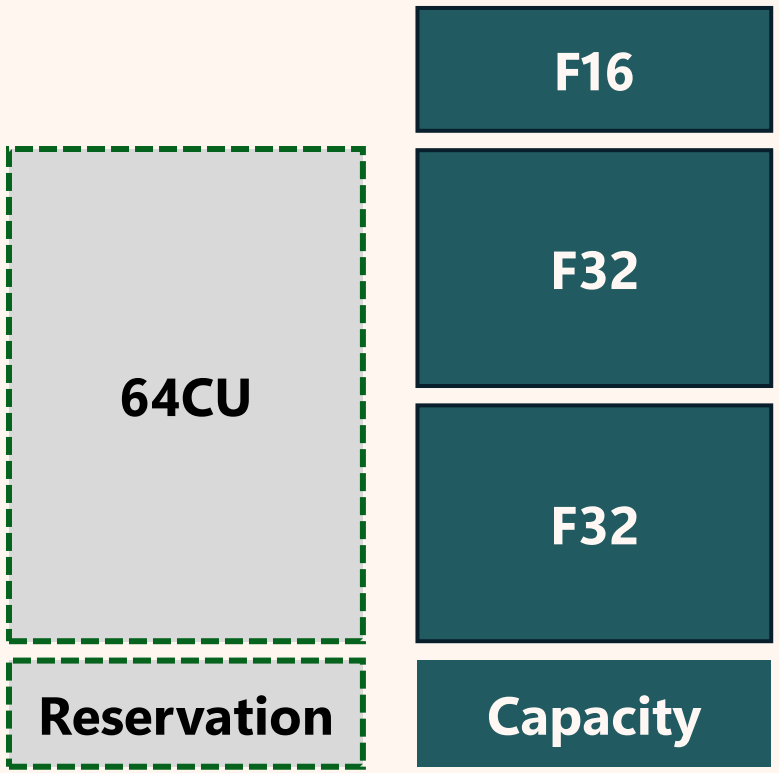
Time scaling up	PAYGO	F64 Reservation	PAYGO + F64 RI	F128 Reservation
10%	€ 1.000	€ 6.000	€ 7.000	€ 12.000
20%	€ 2.000	€ 6.000	€ 8.000	€ 12.000
30%	€ 3.000	€ 6.000	€ 9.000	€ 12.000
40%	€ 4.000	€ 6.000	€ 10.000	€ 12.000
50%	€ 5.000	€ 6.000	€ 11.000	€ 12.000
60%	**€ 6.000**	**€ 6.000**	**€ 12.000**	**€ 12.000**
70%	€ 7.000	€ 6.000	€ 13.000	€ 12.000
80%	€ 8.000	€ 6.000	€ 14.000	€ 12.000
90%	€ 9.000	€ 6.000	€ 15.000	€ 12.000
100%	€ 10.000	€ 6.000	€ 16.000	€ 12.000



ed billing for F128
city, 64CU reserved,
64CU PaYGo

Fabric Capacity Reservations

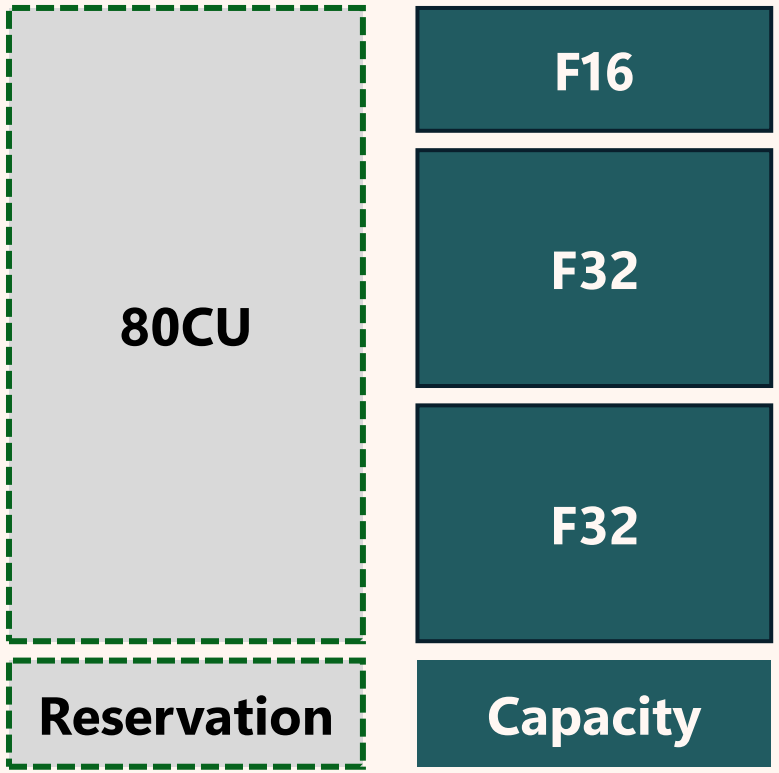
Example 5 – Multiple SKUs exceeding reservation amount



Billing can apply to all the capacities, is only shown in Billing

Fabric Capacity Reservations

Example 5 – Multiple SKUs exceeding reservation amount

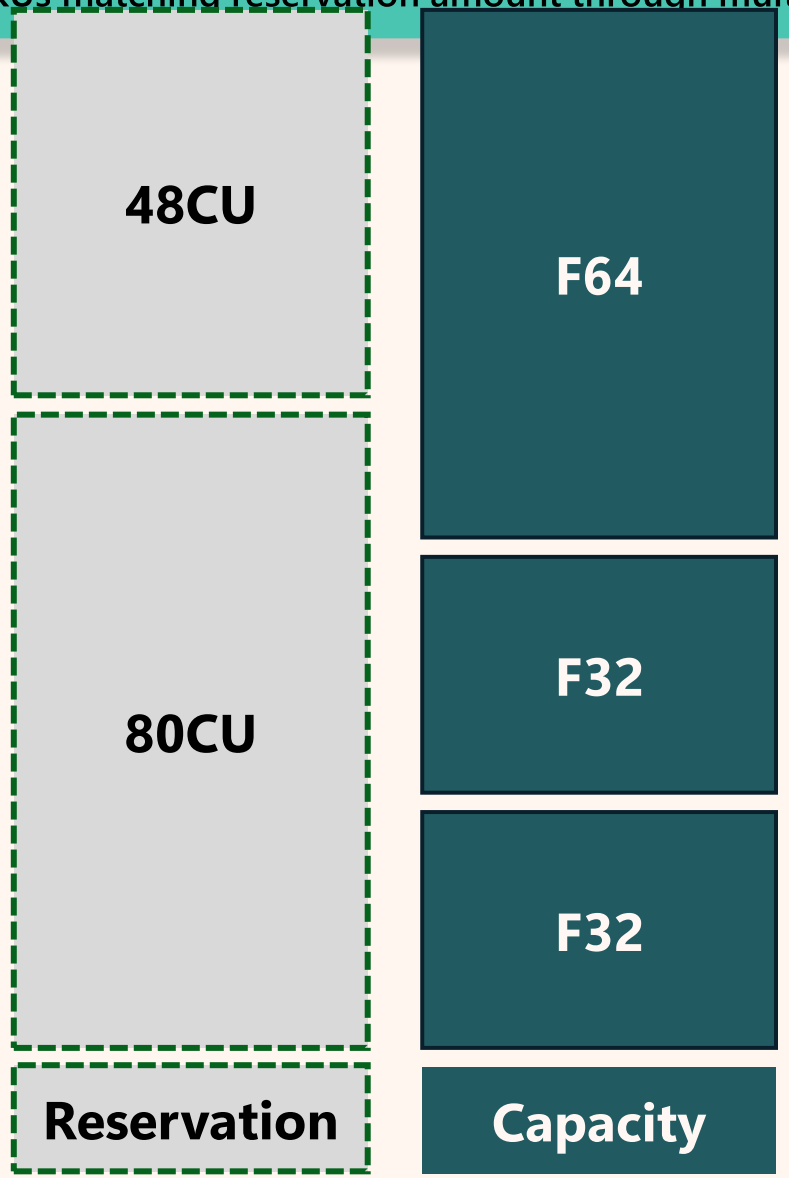


Reservations are based on CUs, not SKUs

Can be any (?) even number

Fabric Capacity Reservations

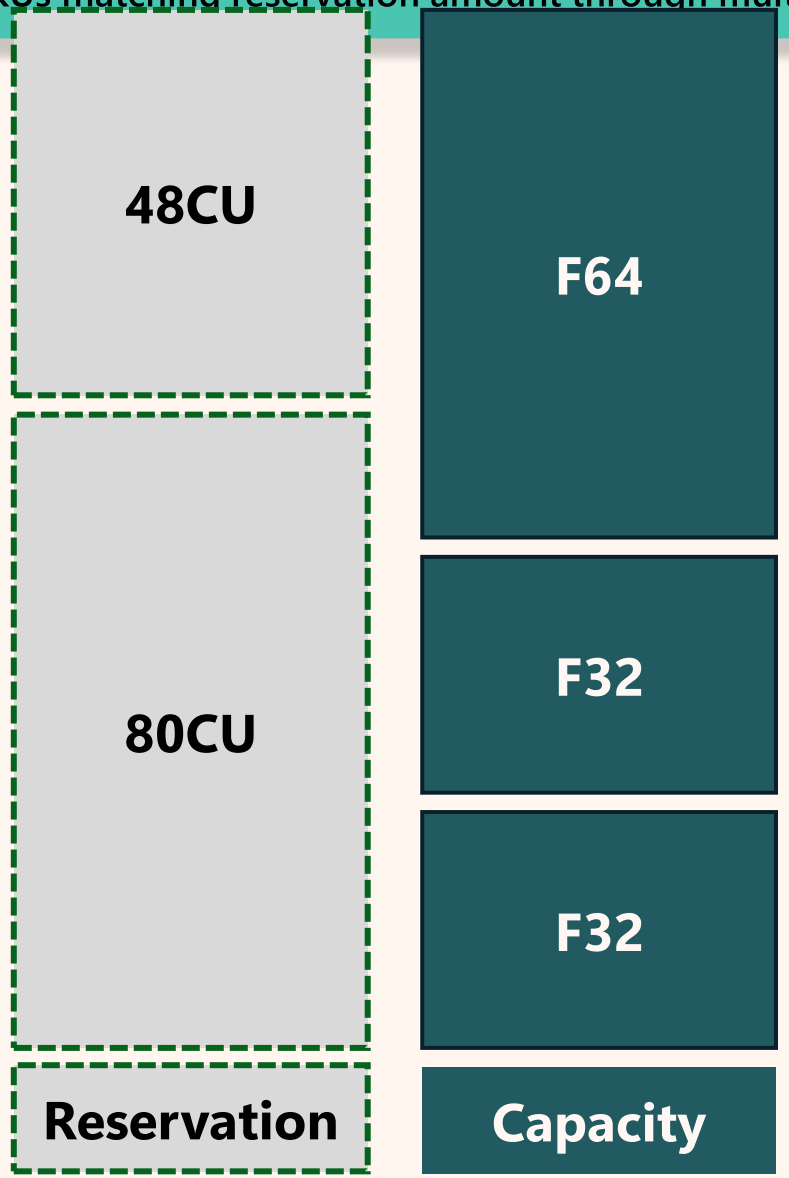
Example 6 – Multiple SKUs matching reservation amount through multiple reservations



Only when second Reservation is added, all Capacities are covered

Fabric Capacity Reservations

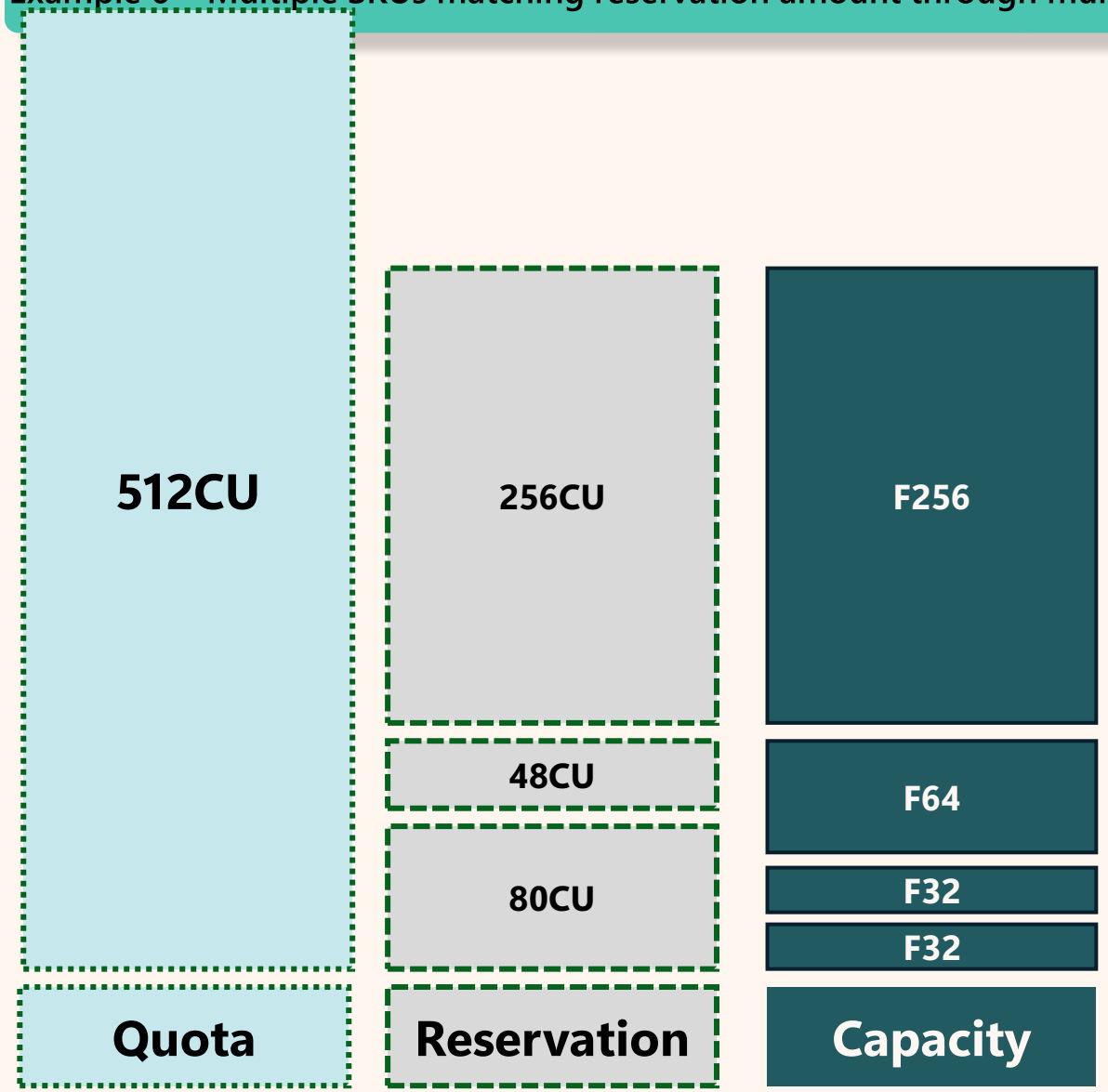
Example 6 – Multiple SKUs matching reservation amount through multiple reservations



Reservation sets the floor

Fabric Capacity Reservations

Example 6 – Multiple SKUs matching reservation amount through multiple reservations



Reservation sets the floor

Quota determines the ceiling*

*) per Subscription/Region

Azure Quota Management Service Integration

GA (March 2025)

Better resource allocation to meet Microsoft's customer capacity needs (blog || docs)

Fabric Quota limits

- Limits the number of Capacity Units (CUs) you can provision across multiple capacities in a subscription, based on subscription type and region

Customers can request a quota adjustment

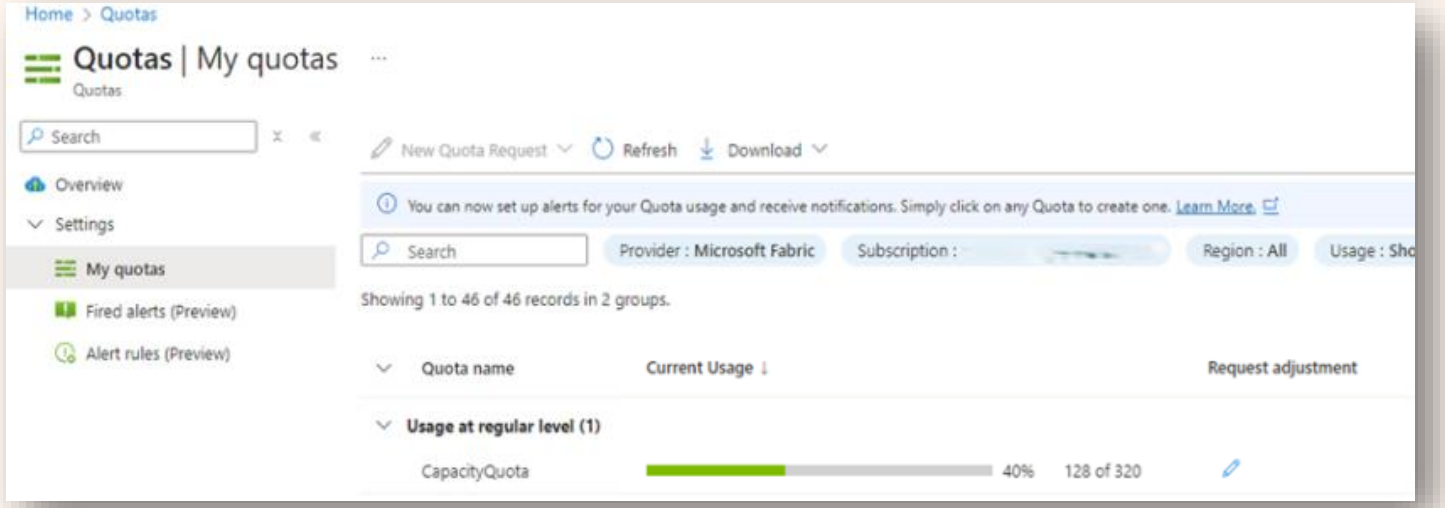
- Auto approved up to specific limits
- Customers can request additional quota through Microsoft customer support

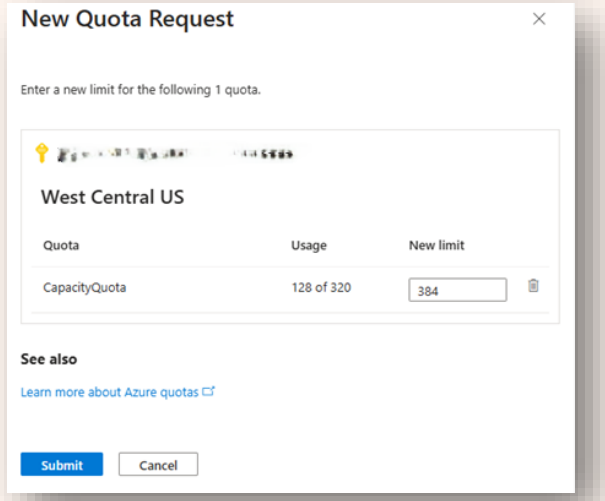
Update automation scripts

- Customers who provision capacity dynamically should check quota first

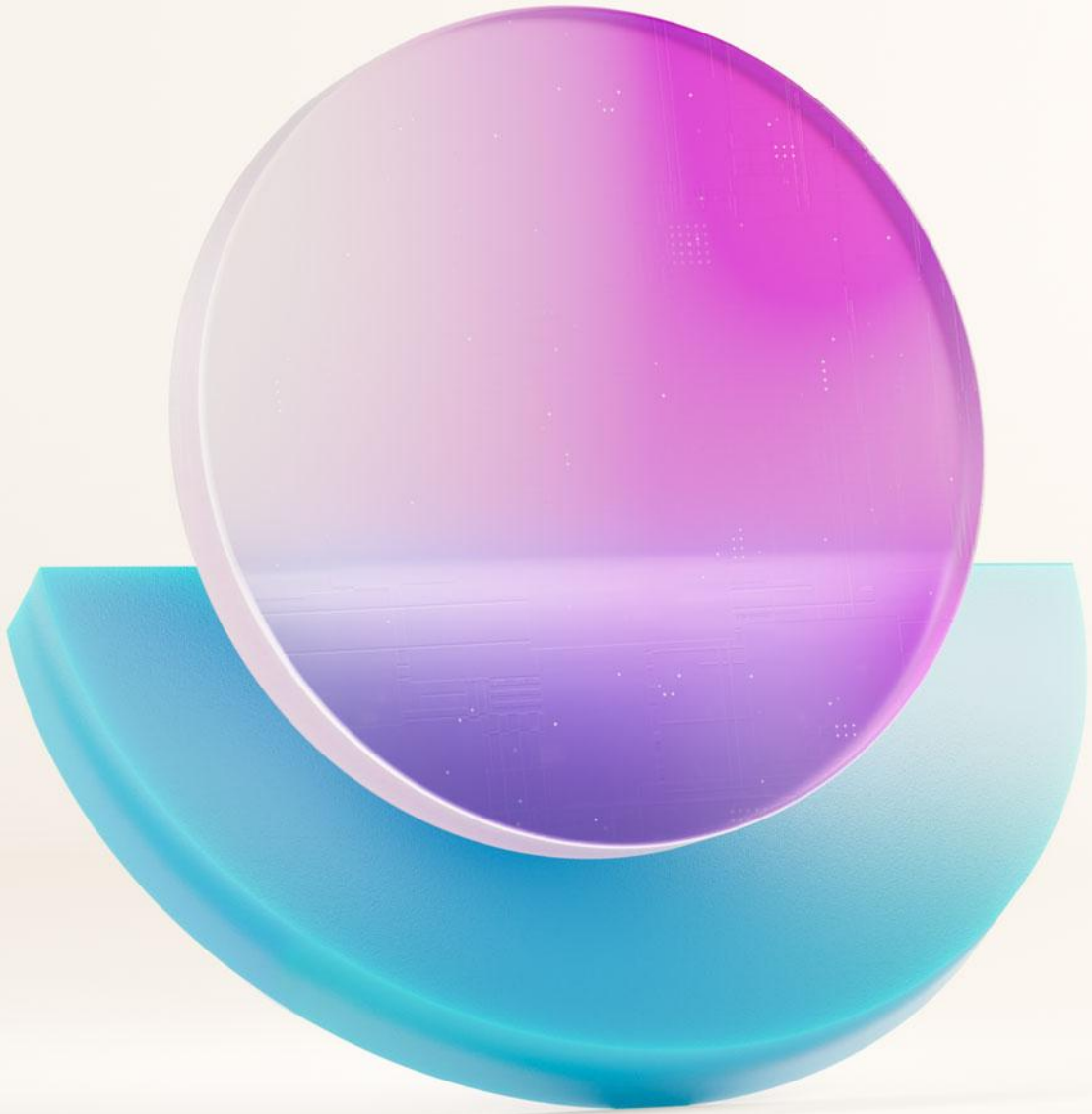
Impact

- Better resource allocation
- Security and compliance – reduce risk of unauthorized excessive usage



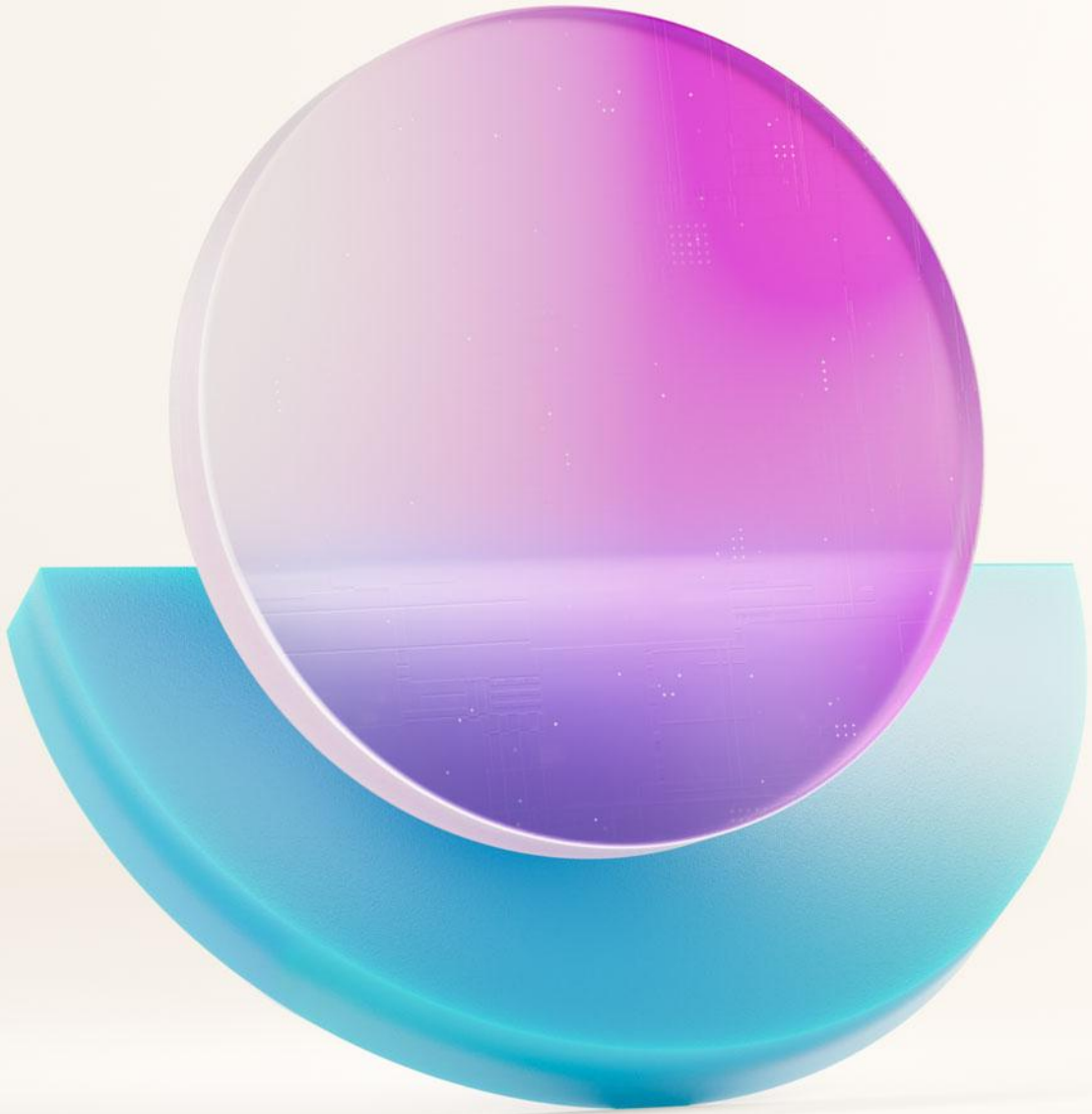


Microsoft Fabric
Community Conference



Dealing with Throttling

Microsoft Fabric
Community Conference



Protecting Capacities

Surge Protection v1 – Background usage limits

GA (Jul 2025)

Enabling Capacity Admins to get ahead of throttling. (blog || docs)

Simple experience that limits overuse by background jobs

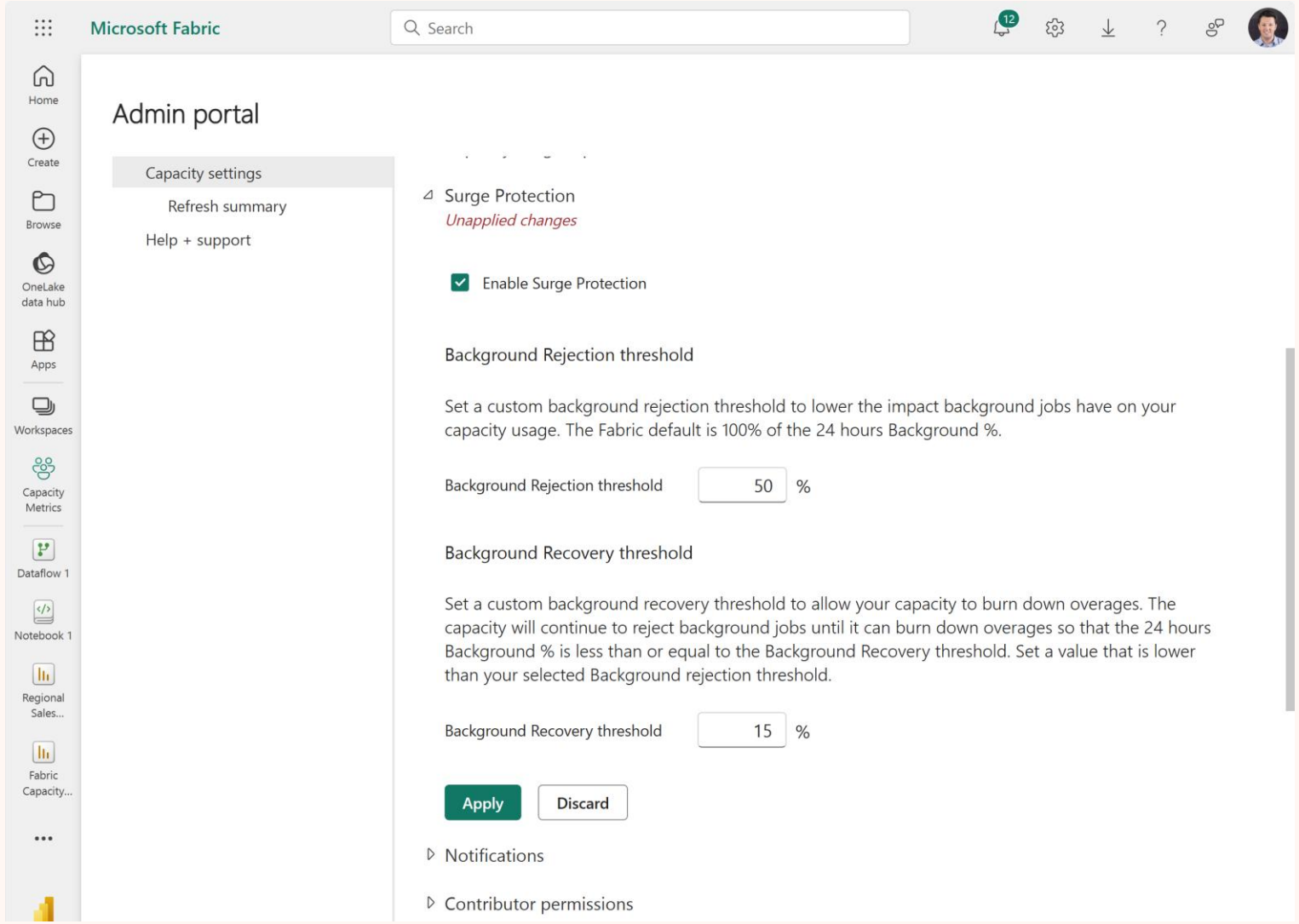
- Throttles background jobs before a full 24-hours of CUs is consumed.
- Helps to **protect interactive** usage like Power BI

Recovery limit

- Keep throttling until the capacity is 'healthy' as defined by the customer
- Helps prevent a capacity from immediately being throttled again

Impact

- Throttling background jobs will help 40-60% of capacities experiencing Interactive Rejections.



Surge Protection V2 – Workspace limits

GA (Sep 2026)

Enabling Capacity Admins to get ahead of throttling.

Simple experience that limits overuse by workspaces

- **Blocks a workspaces** for 24-hours if it exceeds a limit set by the Capacity Admin
- **Mission critical workspaces** are excluded from surge protection limits

Self-healing

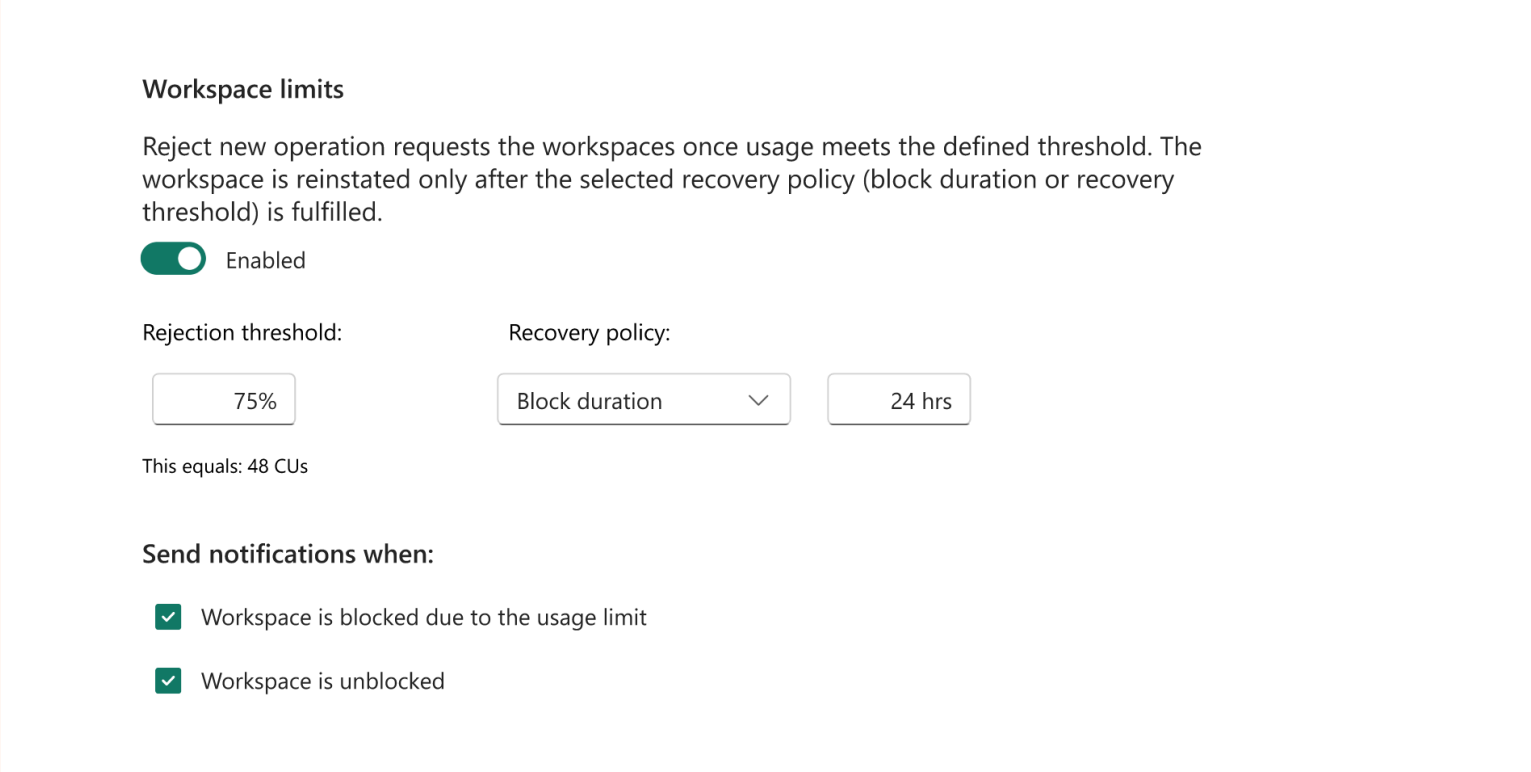
- Automatically restores workspace after 24 hours.

Capacity Admins are in control

- Can add / remove blocked workspaces
- APIs are provided

Impact

- Limit blocking to mis-behaving workspaces allowing the rest of the org to continue working without interruption.



CHOOSE THE INTERVENTION

Fabric capacity under pressure

Protect, scale, or pause? Three choices with different consequences.

01 / CAPACITY OVERAGE

Protect continuity

Pay for excess usage.
Keep the current SKU.

02 / SCALE UP OR DOWN

Change the resource budget

Resize the capacity.
Match ongoing demand.

03 / PAUSE AND RESUME

Accept an interruption

Settle outstanding usage.
Restart deliberately.

The pressure is understood. Now choose the operational trade-off.

CHOOSE THE INTERVENTION

Three questions before you act

Decide by demand pattern, availability requirements, and acceptable cost.

01	**How long will demand last?**	Occasional spike or sustained demand?
02	**Must workloads stay available?**	No interruption or an approved outage?
03	**What extra cost is acceptable?**	Premium excess usage, a larger SKU, or interruption?

Keep three cases in mind: reporting spike / month-end peak / throttled development capacity.

CAPACITY OVERAGE

Preserve continuity. Keep the same SKU.

Paid capacity overage is an opt-in feature in preview, not a capacity resize.

WHAT IT CHANGES

Carryforward is billed instead of CU throttling

Reconciles carryforward when throttling would otherwise begin.

WHAT IT DOES NOT CHANGE

No larger SKU. No extra memory.

Other resource limits and future smoothed consumption still apply.

Built-in overage protection and opt-in paid capacity overage are different mechanisms.

Use overage for bounded excess demand

Best when continuity matters and the overload is limited.

GOOD FIT

An unexpected reporting spike

Short or small overloads.
Users need uninterrupted access.
Additional spend is approved.

WRONG EXPECTATION

Persistent undersizing

Sustained heavy overloads.
Memory or concurrency limits.
An alternative to workload tuning.

Repeated activation is a prompt to review sizing and workload efficiency.

A financial guardrail, not an unlimited safety net

Set a limit in CU-hours and understand when the meter runs.

3x

PAYG CU-hour rate

Only billed excess usage, not the entire capacity.

24h

Rolling usage window

Not a budget that resets at midnight.

5 min

Limit evaluation interval

The configured limit can be exceeded.

Enabling overage during heavy throttling can immediately bill accumulated carryforward.

01 / CAPACITY OVERAGE

Configure the safety net before the incident

Fabric Admin portal > Capacity settings > Capacity overage

1 / CONFIGURE

Approve the allowance

Capacity admin.
Quota and CU-hour limit.
Named cost owner.

2 / ACTIVATE

Allow up to five minutes

No pause or restart.
Treat deep existing overload with caution.

3 / RESPOND

Follow billed usage

Capacity Metrics.
Azure Cost Management.
Review repeat activation.

Before enabling: know the limit, the owner, and the response when the allowance runs out.

CAPACITY SCALING

Scaling changes the budget, not the history

A larger SKU creates more headroom; it does not erase previous consumption.

SAME CAPACITY / LARGER BASE ALLOCATION

F64 | 64 CUs

F128 | 128 CUs

More room for demand and recovery

Carryforward burns down faster only if spare capacity remains.

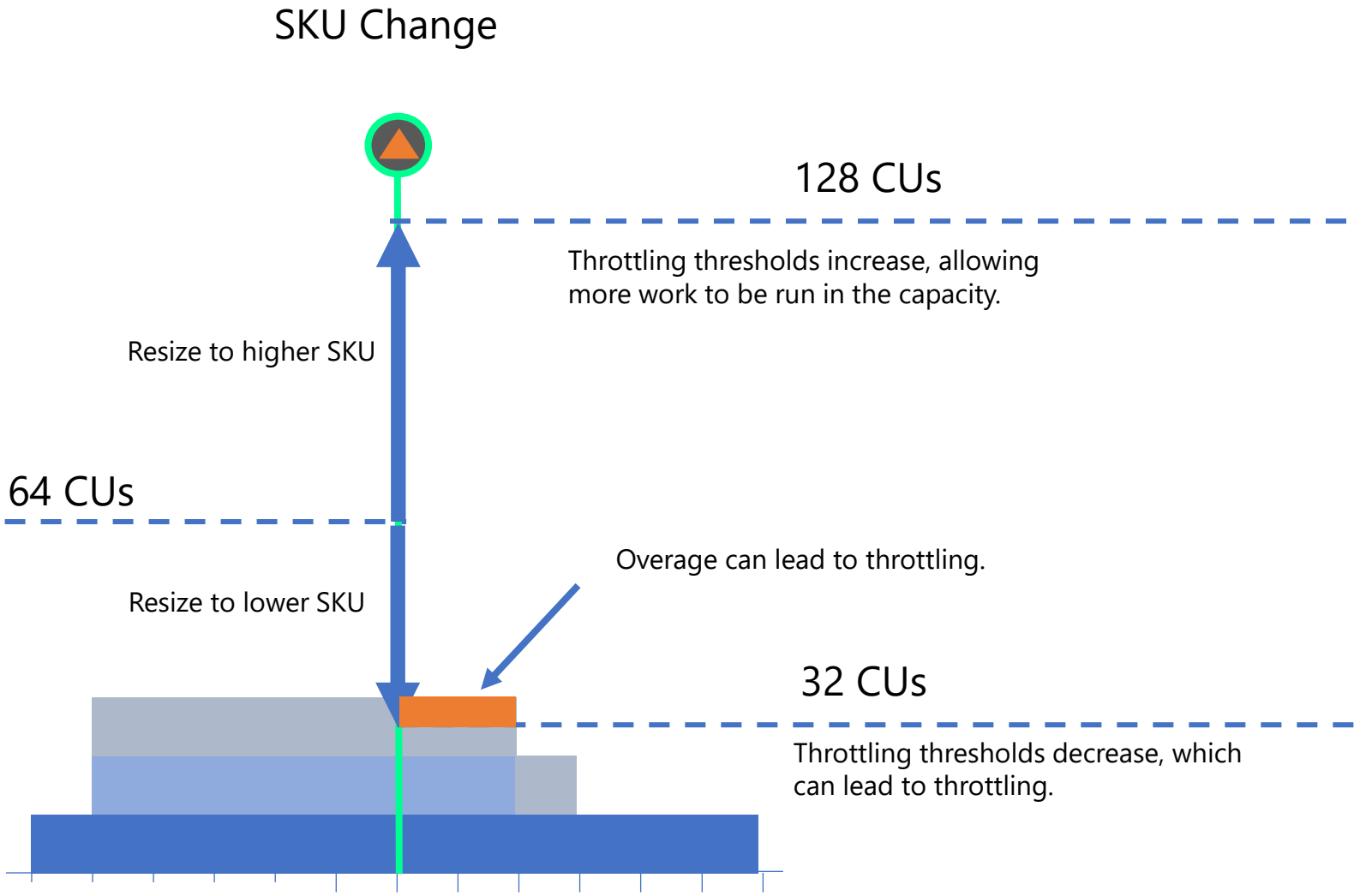
Twice the base CUs does not mean every job will run twice as fast.

How Capacity Resize works

When a capacity is **resized**…

The allowed CUs per timepoint increase or decrease.

This changes the throttling allowed limits based on the new SKU's CUs and the throttling windows.



Key Insights

- Sizing up will incur the cost of the new SKU
- Sizing down could lead to more throttling
- Review your Throttling Thresholds before sizing down your SKU.

Scale ahead of predictable pressure

Month-end pattern: increase early, retain headroom, then reassess.

BEFORE

Scale up

Quota and change window are ready.

DURING

Serve the peak

Keep room for the planned workload.

AFTER

Allow recovery

The job can finish before pressure ends.

RETURN

Reassess size

Choose a safe steady-state SKU.

Azure portal > Fabric capacity > Scale > Change size > Resize

Scale down only when the smaller SKU is ready

A completed job is not, by itself, a safe downscaling signal.

Consumption	Remaining smoothed usage and carryforward
Demand	Upcoming work and enough recovery headroom
Constraints	Memory needs and the F64 viewing-license boundary
Cost	A smaller SKU can trigger new billed overages

Reducing the SKU can exchange apparent capacity savings for overage charges.

Build a policy, not a resizing loop

Automate deliberate transitions, not every utilization fluctuation.

BEFORE AUTOMATION

Bound the change

Minimum and maximum SKU
Quota and Azure permissions
Cooldown, ownership, rollback

PLAN AN INTERRUPTION WINDOW

F256 and below
< >
F512 and above

Crossing can cancel running jobs.

Document safe return conditions and the response if a resize disrupts work.

CAPACITY PAUSING

Settle consumption. Then restart.

Pause/resume changes availability and creates a reconciliation billing event.

1 / PAUSE

Availability is interrupted

Assigned content can become unavailable. Plan the outage.

2 / RECONCILE

Outstanding usage is billed

Remaining smoothed consumption plus cumulative overages.

3 / RESUME

Begin without future usage debt

Capacity billing restarts. New workloads can run. Prior cost is not erased.

Consumption is settled, not forgiven. This is not a free reset.

How Capacity Pause & Resume works

When a capacity is **paused**…

Pause event on Capacity

64 CUs

Total smoothed usage is shown as compute utilization on the timepoint directly after the Pause event.

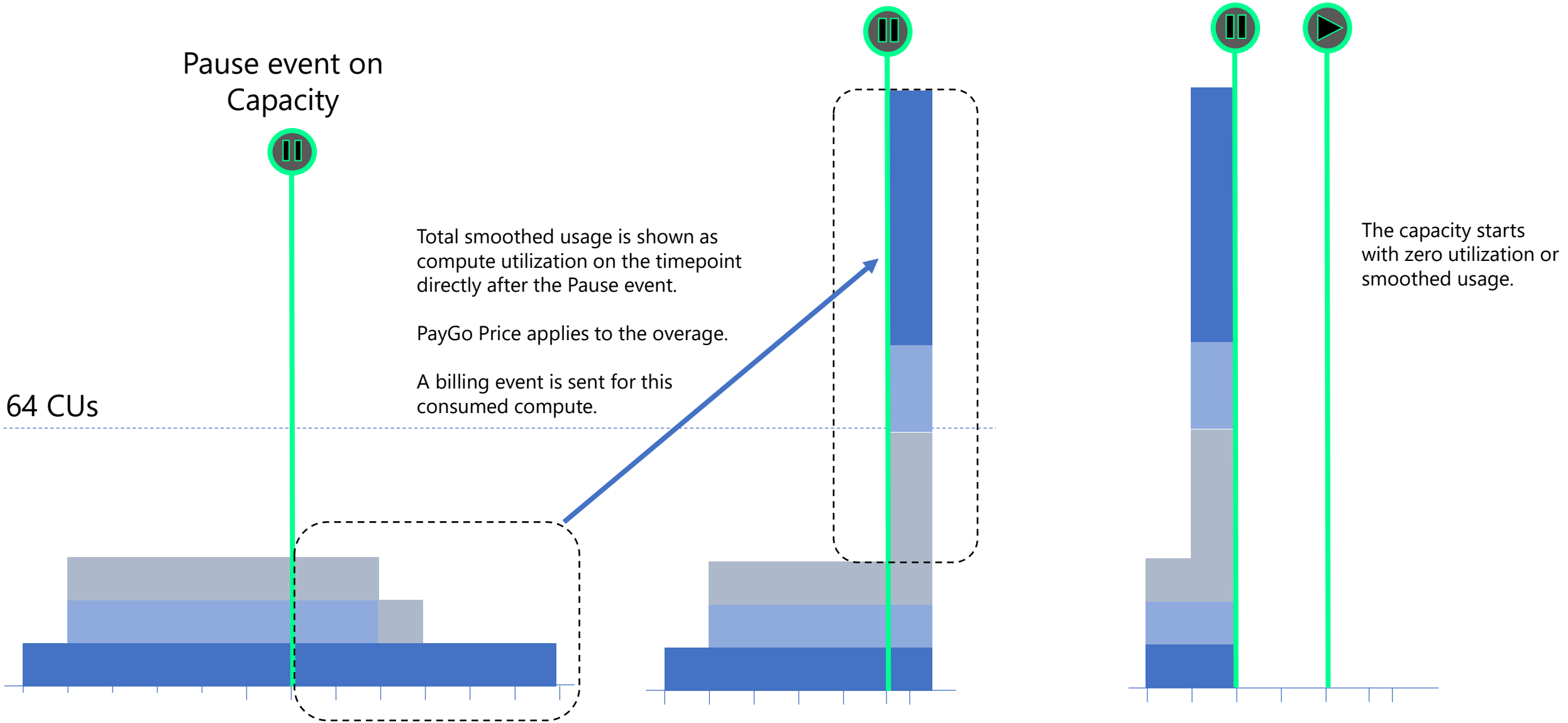
PayGo Price applies to the overage.

A billing event is sent for this consumed compute.

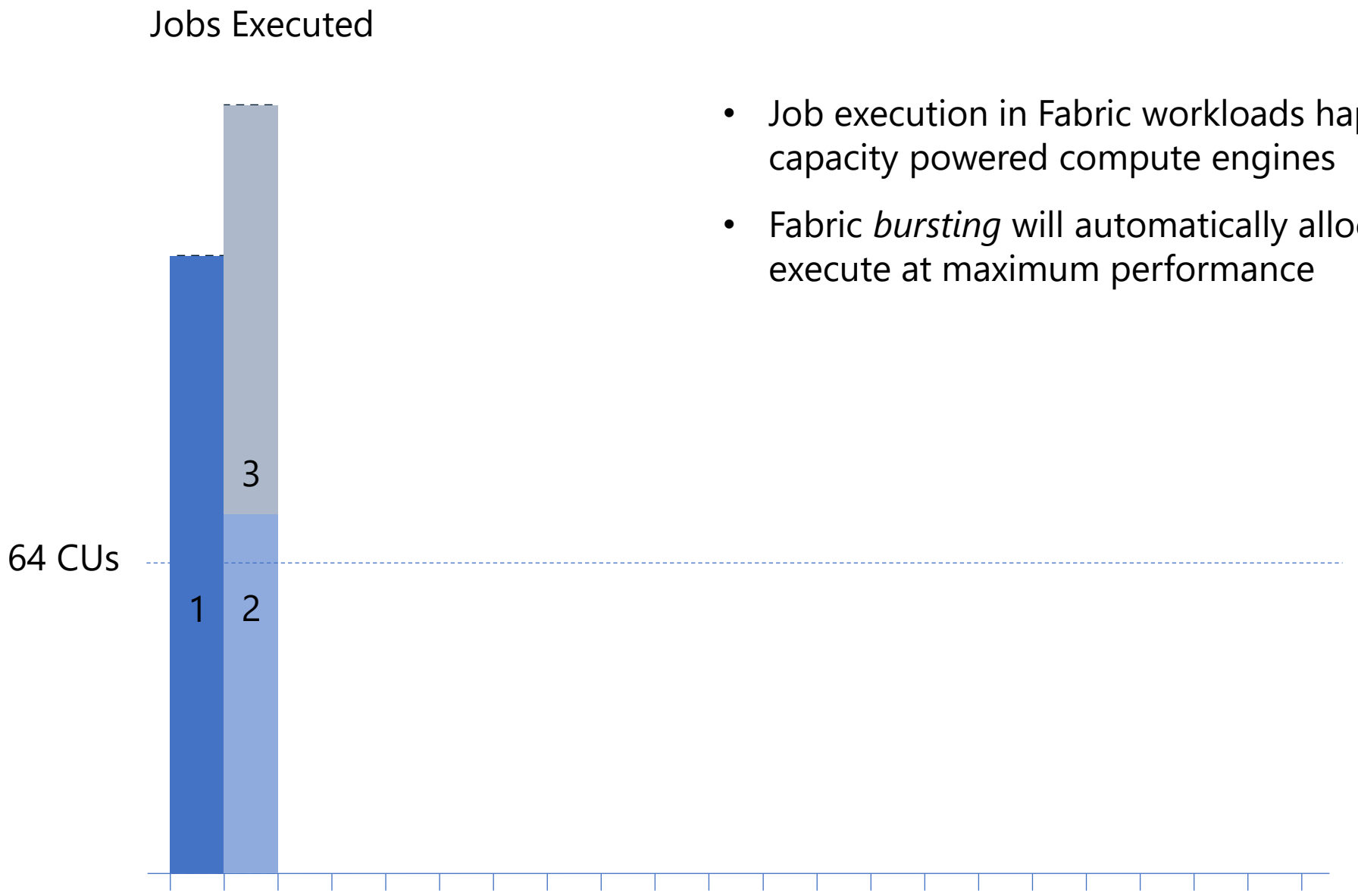
Smoothed usage is **reconciled**.

Later, it can be **resumed**.

The capacity starts with zero utilization or smoothed usage.



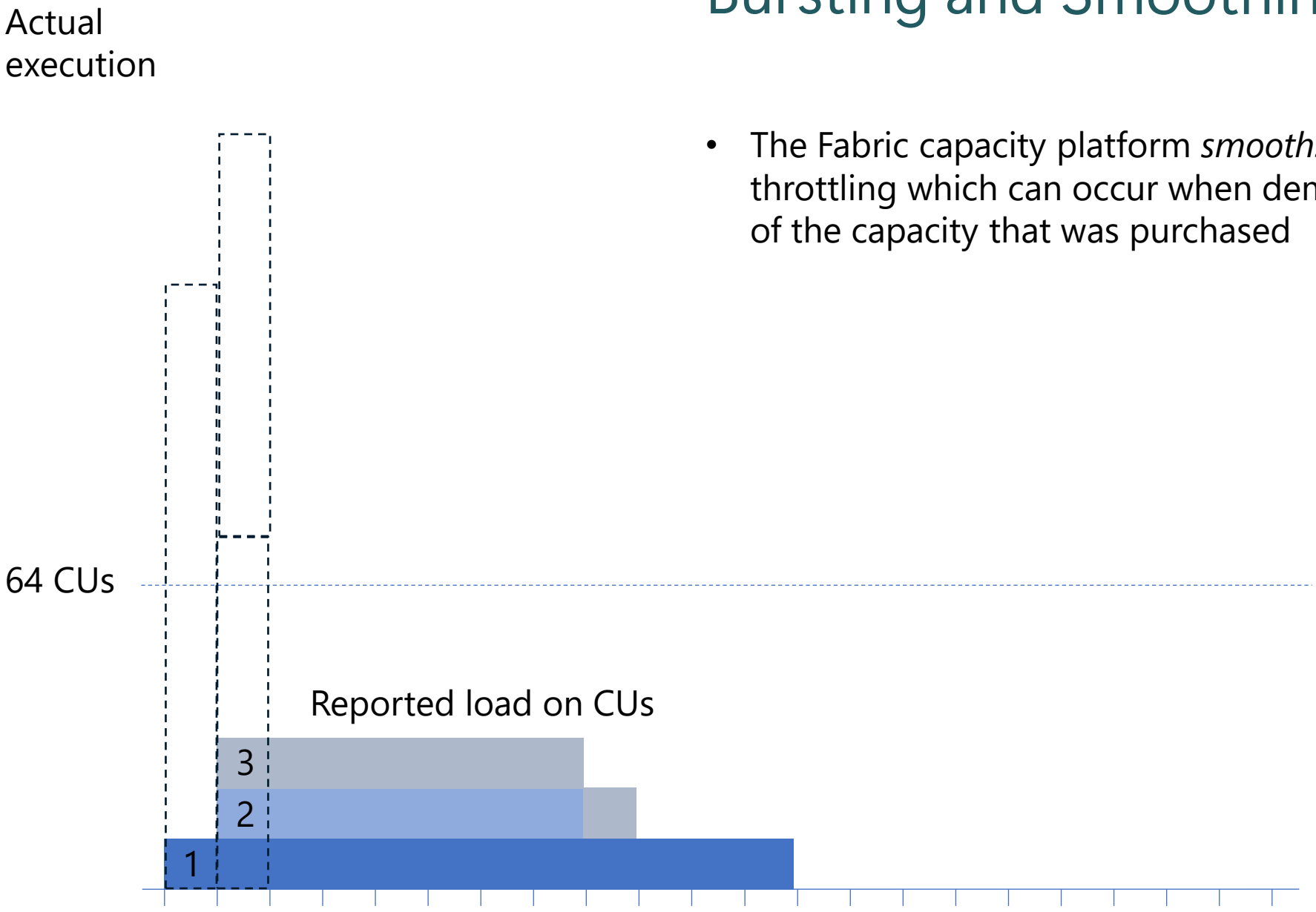
Bursting and Smoothing



- Job execution in Fabric workloads happens on-demand via capacity powered compute engines
- Fabric *bursting* will automatically allocate resources as needed to execute at maximum performance

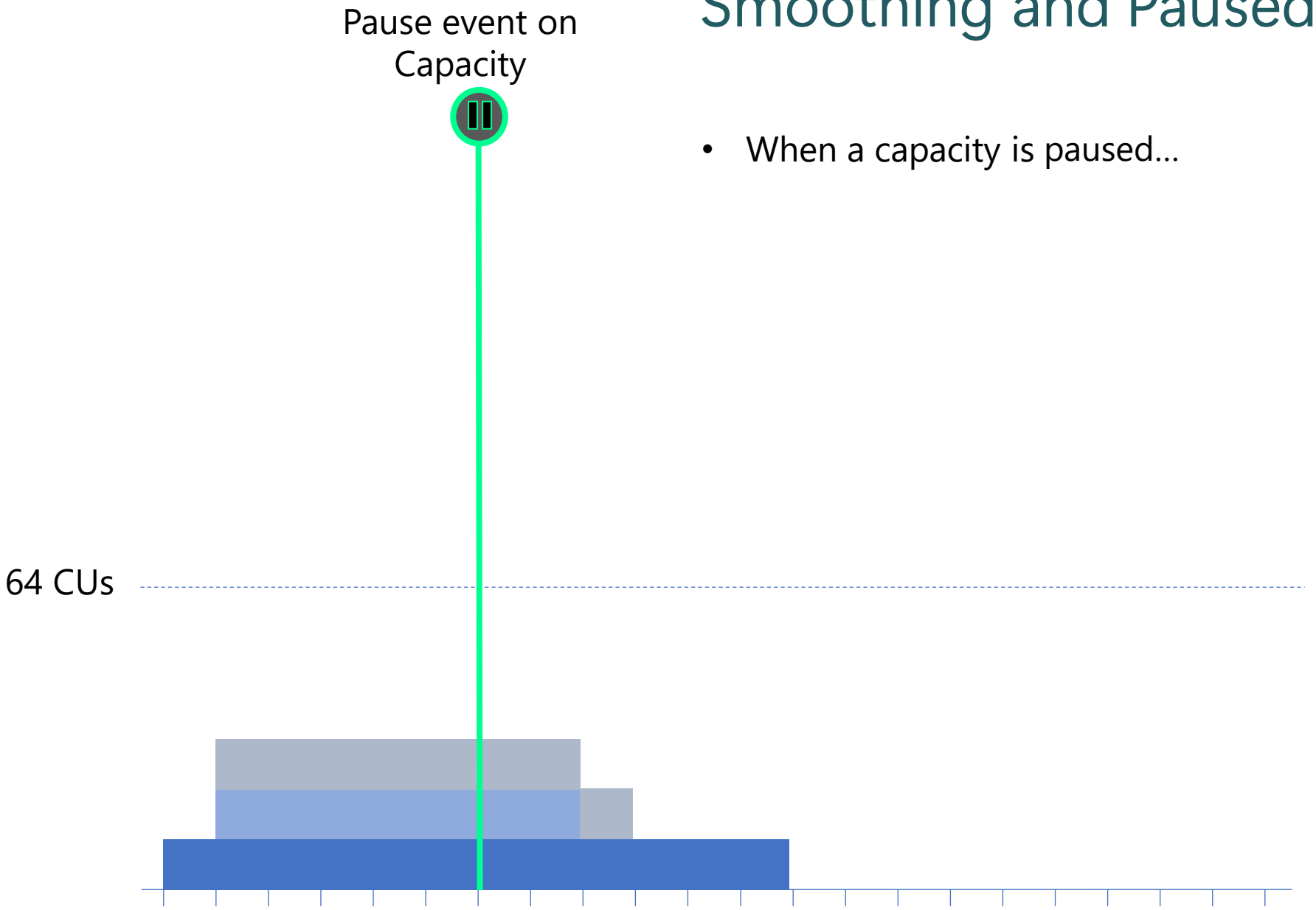
Bursting and Smoothing

- The Fabric capacity platform *smooths* usage out to reduce throttling which can occur when demand exceeds the throughput of the capacity that was purchased

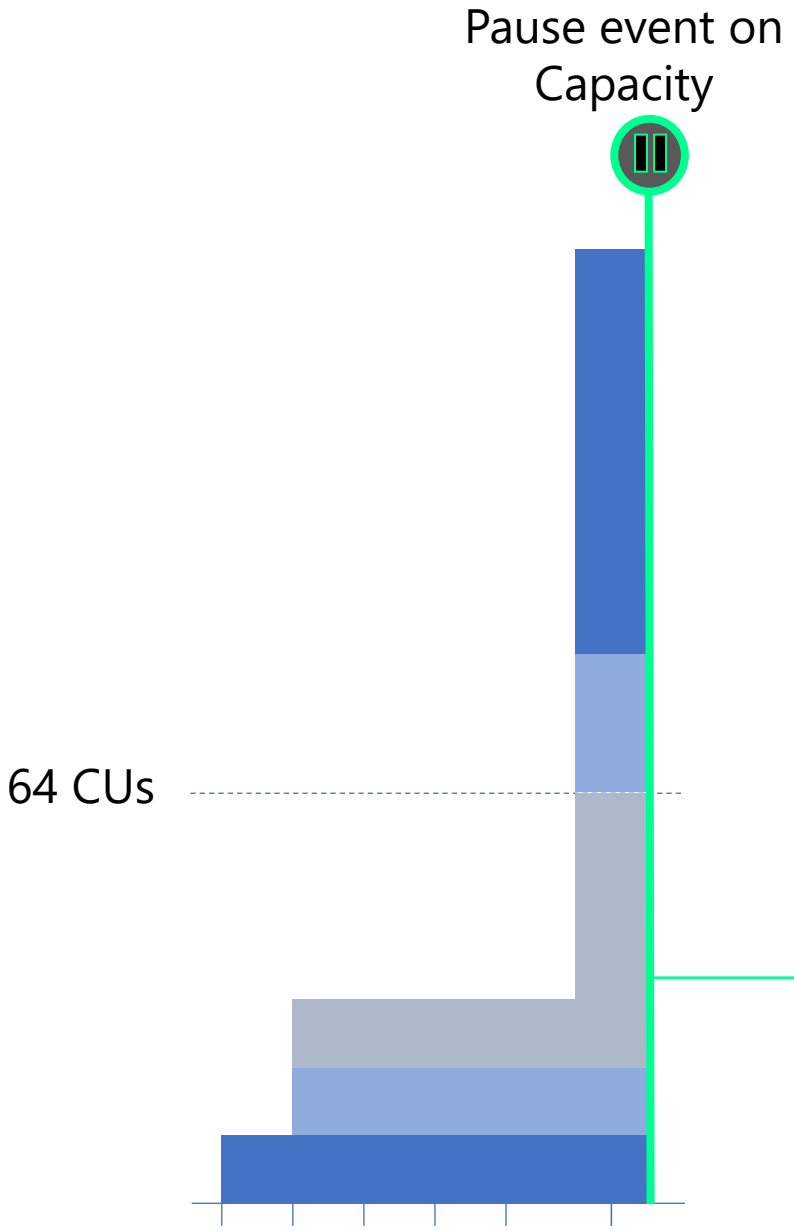


Smoothing and Paused Capacities

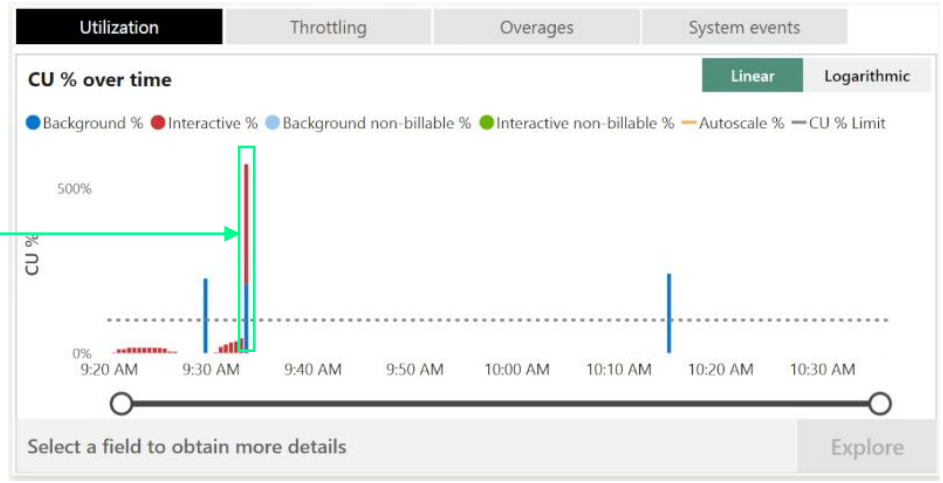
- When a capacity is paused…



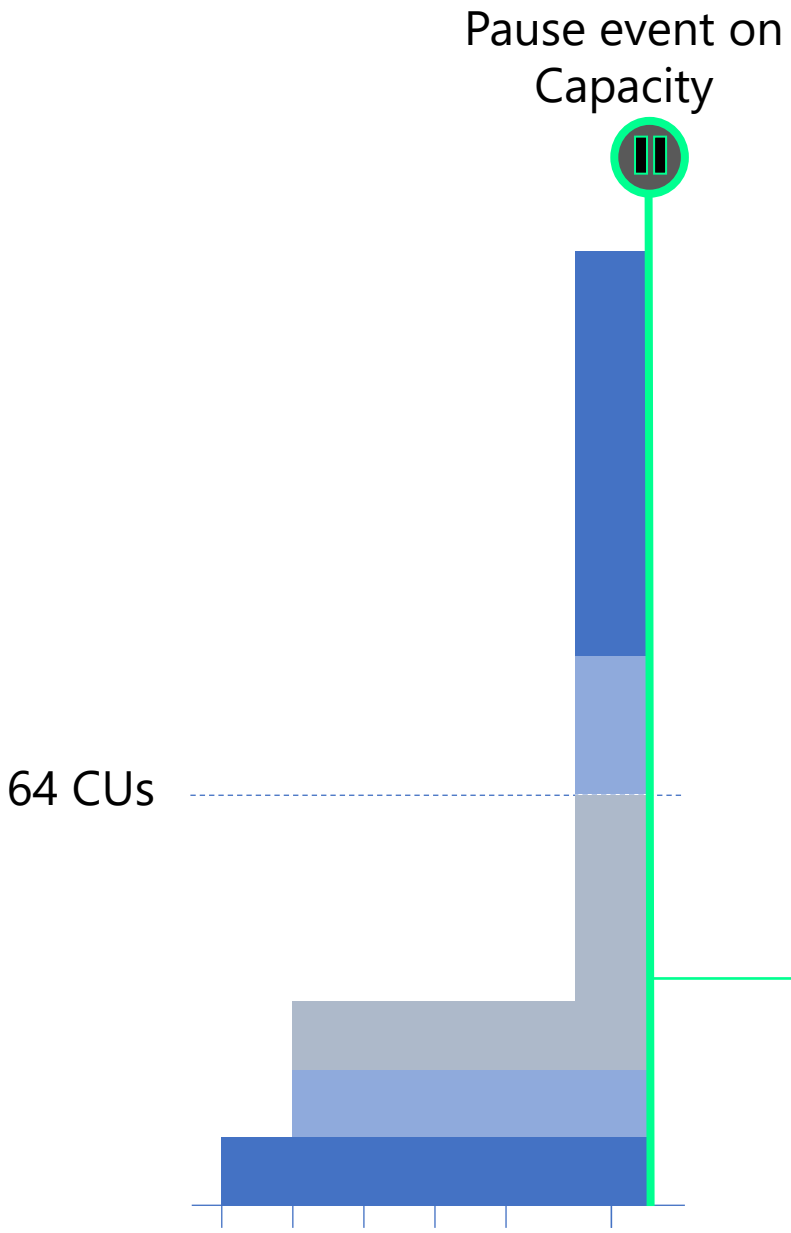
Smoothing and Paused Capacities



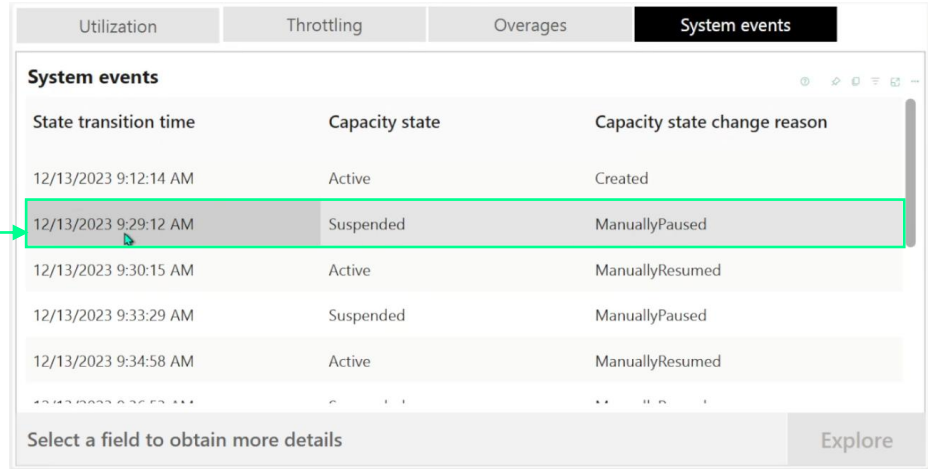
- When a capacity is paused…
- Usage that was smoothed into the future will be “reconciled” and charged against the capacity at the timestamp the capacity was paused
- Reconciled usage will show up as a spike in capacity metrics



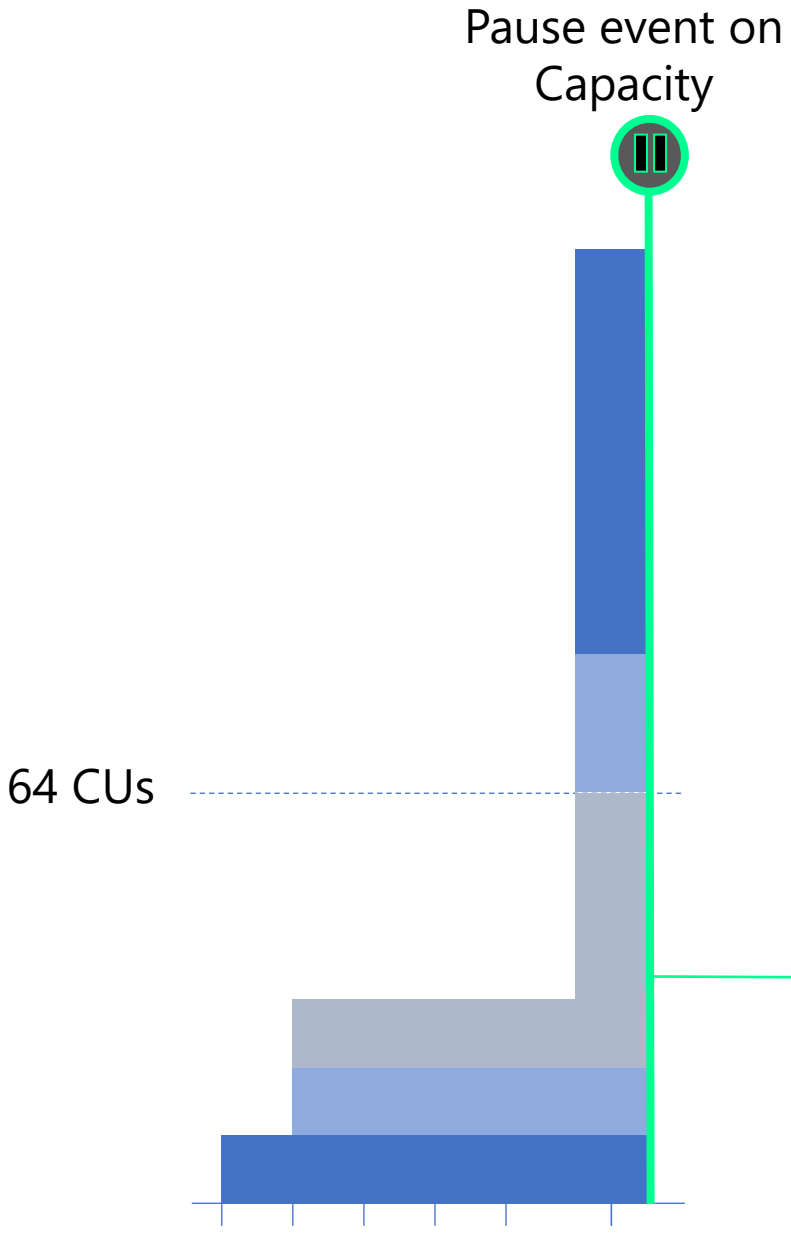
Smoothing and Paused Capacities



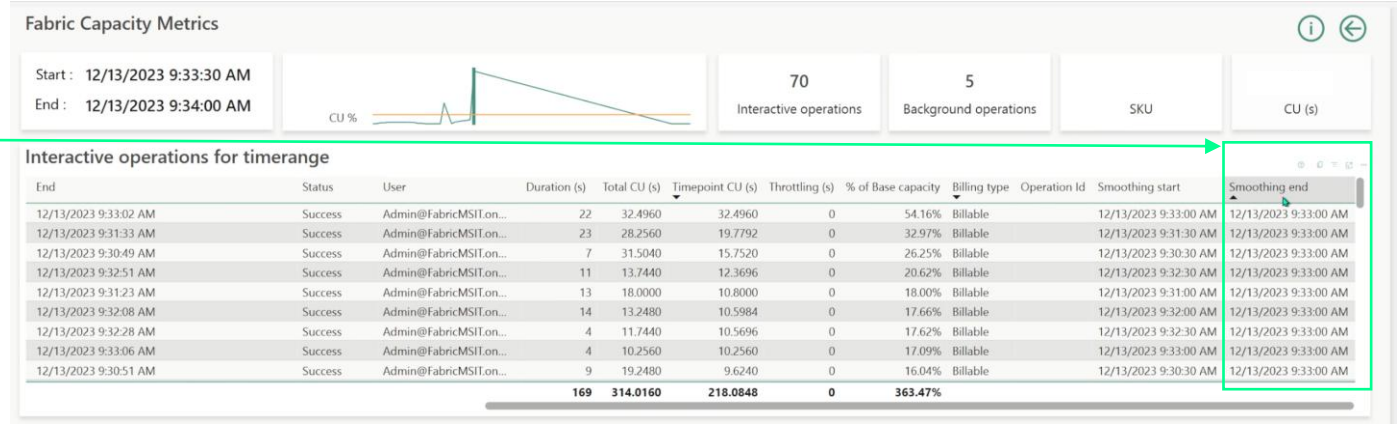
- When a capacity is paused…
- Usage that was smoothed into the future will be “reconciled” and charged against the capacity at the timestamp the capacity was paused
- Pause events can be viewed in the new System events tab



Smoothing and Paused Capacities



- When a capacity is paused…
- Usage that was smoothed into the future will be “reconciled” and charged against the capacity at the timestamp the capacity was paused
- Pause events timestamp is shown in the smoothing end field in timepoint drill views



End	Status	User	Duration (s)	Total CU (s)	Timepoint CU (s)	Throttling (s)	% of Base capacity	Billing type	Operation Id	Smoothing start	Smoothing end
12/13/2023 9:33:02 AM	Success	Admin@FabricMSIT.on...	22	32.4960	32.4960	0	54.16%	Billable		12/13/2023 9:33:00 AM	12/13/2023 9:33:00 AM
12/13/2023 9:31:33 AM	Success	Admin@FabricMSIT.on...	23	28.2560	19.7792	0	32.97%	Billable		12/13/2023 9:31:30 AM	12/13/2023 9:33:00 AM
12/13/2023 9:30:49 AM	Success	Admin@FabricMSIT.on...	7	31.5040	15.7520	0	26.25%	Billable		12/13/2023 9:30:30 AM	12/13/2023 9:33:00 AM
12/13/2023 9:32:51 AM	Success	Admin@FabricMSIT.on...	11	13.7440	12.3696	0	20.62%	Billable		12/13/2023 9:32:30 AM	12/13/2023 9:33:00 AM
12/13/2023 9:31:23 AM	Success	Admin@FabricMSIT.on...	13	18.0000	10.8000	0	18.00%	Billable		12/13/2023 9:31:00 AM	12/13/2023 9:33:00 AM
12/13/2023 9:32:08 AM	Success	Admin@FabricMSIT.on...	14	13.2480	10.5984	0	17.66%	Billable		12/13/2023 9:32:00 AM	12/13/2023 9:33:00 AM
12/13/2023 9:32:28 AM	Success	Admin@FabricMSIT.on...	4	11.7440	10.5696	0	17.62%	Billable		12/13/2023 9:32:30 AM	12/13/2023 9:33:00 AM
12/13/2023 9:33:06 AM	Success	Admin@FabricMSIT.on...	4	10.2560	10.2560	0	17.09%	Billable		12/13/2023 9:33:00 AM	12/13/2023 9:33:00 AM
12/13/2023 9:30:51 AM	Success	Admin@FabricMSIT.on...	9	19.2480	9.6240	0	16.04%	Billable		12/13/2023 9:30:30 AM	12/13/2023 9:33:00 AM
			169	**314.0160**	**218.0848**	**0**	**363.47%**				

Pausing a Fabric Capacity

It might actually cost you more ..

Don’t blindly pause

- Especially if you’re hoping it will reduce costs
- Be mindful of ‘open balance’
- How long would you need to pause for it to be beneficial?

Throttling

- In a throttled state, this can add up
- What is the price for business continuity?

	Average Percentage Utilisation Next 24 Hours	Min Pause Hours for Saving	Cost of Pausing European F64
10,000%	3.47%	0.83	$10.13
20,000%	6.94%	1.67	$20.27
30,000%	10.42%	2.50	$30.40
40,000%	13.89%	3.33	$40.53
50,000%	17.36%	4.17	$50.67
60,000%	20.83%	5.00	$60.80
70,000%	24.31%	5.83	$70.93
80,000%	27.78%	6.67	$81.07
90,000%	31.25%	7.50	$91.20
100,000%	34.72%	8.33	$101.33
110,000%	38.19%	9.17	$111.47
120,000%	41.67%	10.00	$121.60
130,000%	45.14%	10.83	$131.73
140,000%	48.61%	11.67	$141.87
150,000%	52.08%	12.50	$152.00
160,000%	55.56%	13.33	$162.13
170,000%	59.03%	14.17	$172.27
180,000%	62.50%	15.00	$182.40
190,000%	65.97%	15.83	$192.53
200,000%	69.44%	16.67	$202.67
210,000%	72.92%	17.50	$212.80
220,000%	76.39%	18.33	$222.93
230,000%	79.86%	19.17	$233.07
240,000%	83.33%	20.00	$243.20
250,000%	86.81%	20.83	$253.33
260,000%	90.28%	21.67	$263.47
270,000%	93.75%	22.50	$273.60
280,000%	97.22%	23.33	$283.73
288,000%	100.00%	24.00	$291.84

https://www.linkedin.com/pulse/fabric-billing-part-4-implications-pause-restart-matthew-farrow-fznse/

Use pause when interruption is acceptable

Two legitimate patterns. Neither is an always-on continuity strategy.

PATTERN A

Planned inactivity

Unused development/test capacity.
Pause for a planned inactive window.
Resume before work returns.

PATTERN B

Controlled recovery

A severely throttled capacity.
An approved interruption window.
Controlled demand on restart.

Not the right response when live reporting must remain continuously available.

Pause deliberately. Resume deliberately.

Azure portal > Fabric capacity > Pause / Resume

01

Prepare

Agree the window. Stop new submissions.

02

Review

Ensure no active use. Review outstanding usage.

03

Pause

Accept the interruption and reconciliation.

04

Resume

Confirm availability. Restart work in stages.

A utilization spike at pause can be reconciliation, not a newly running workload.

MAKE THE CALL

Choose the lever that matches the constraint

Return to the three scenarios. Which assumption changes your answer?

SITUATION	PREFERRED RESPONSE	MAIN CAUTION
Brief reporting overload; continuity required	**Paid overage**	Premium charges; no larger resource pool
Predictable month-end peak or sustained demand	**Scale up, then down safely**	Recovery time and target-SKU limits
Idle capacity or recovery with an approved outage	**Pause / resume**	Interruption and usage reconciliation

Combine levers when useful: planned scaling plus bounded paid overage for unexpected spikes.

PUT IT INTO PRACTICE

Turn the choice into an operating policy

Document the permission to act before pressure becomes an incident.

PROTECT

Approved overage allowance

Limit, cost owner, and escalation response.

SCALE

Safe resource changes

Triggers, SKU bounds, and return conditions.

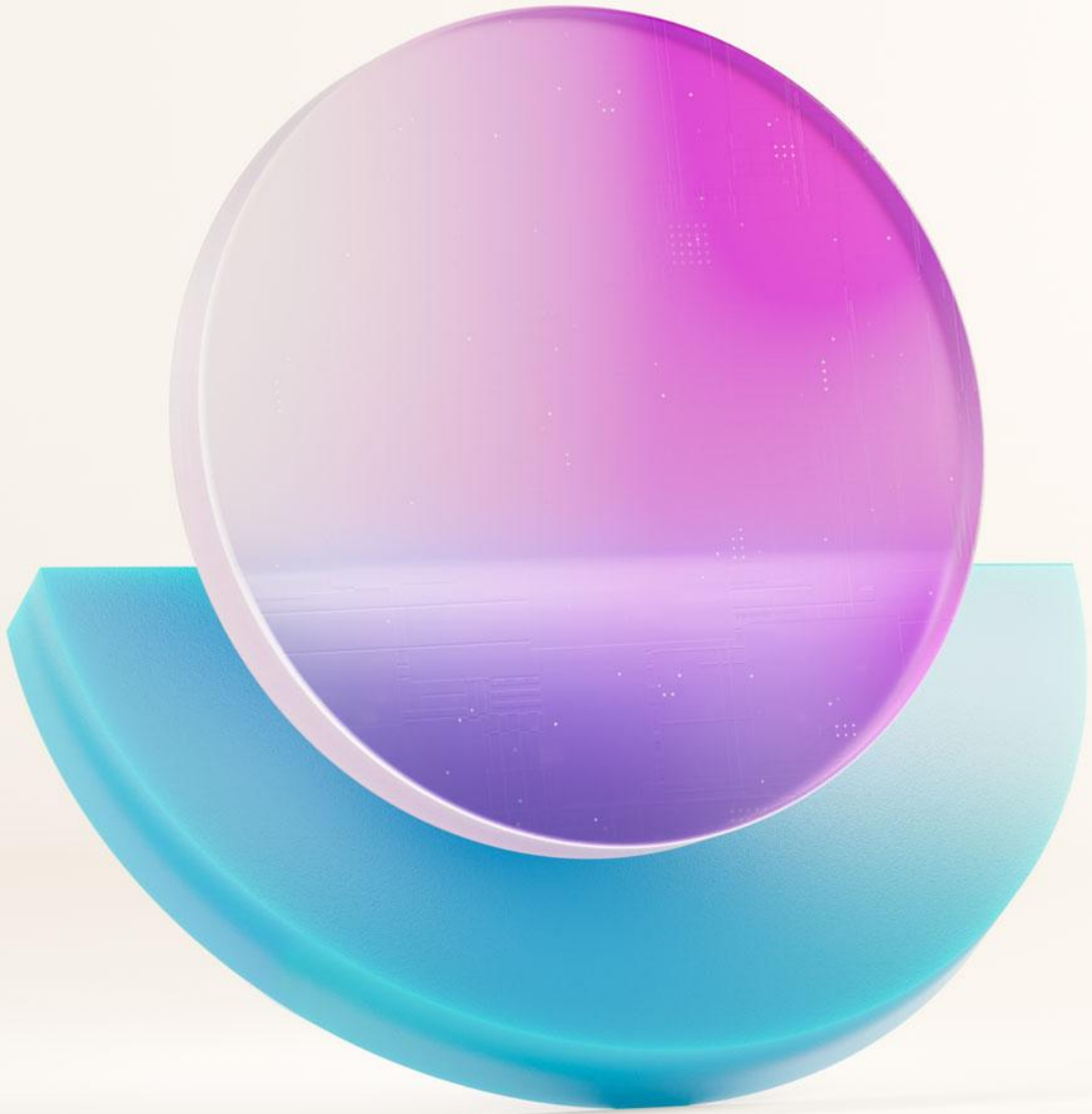
PAUSE

Approved interruption

Eligibility, settlement awareness, and restart procedure.

Protect occasional excess. Scale sustained demand. Pause only when interruption is acceptable.

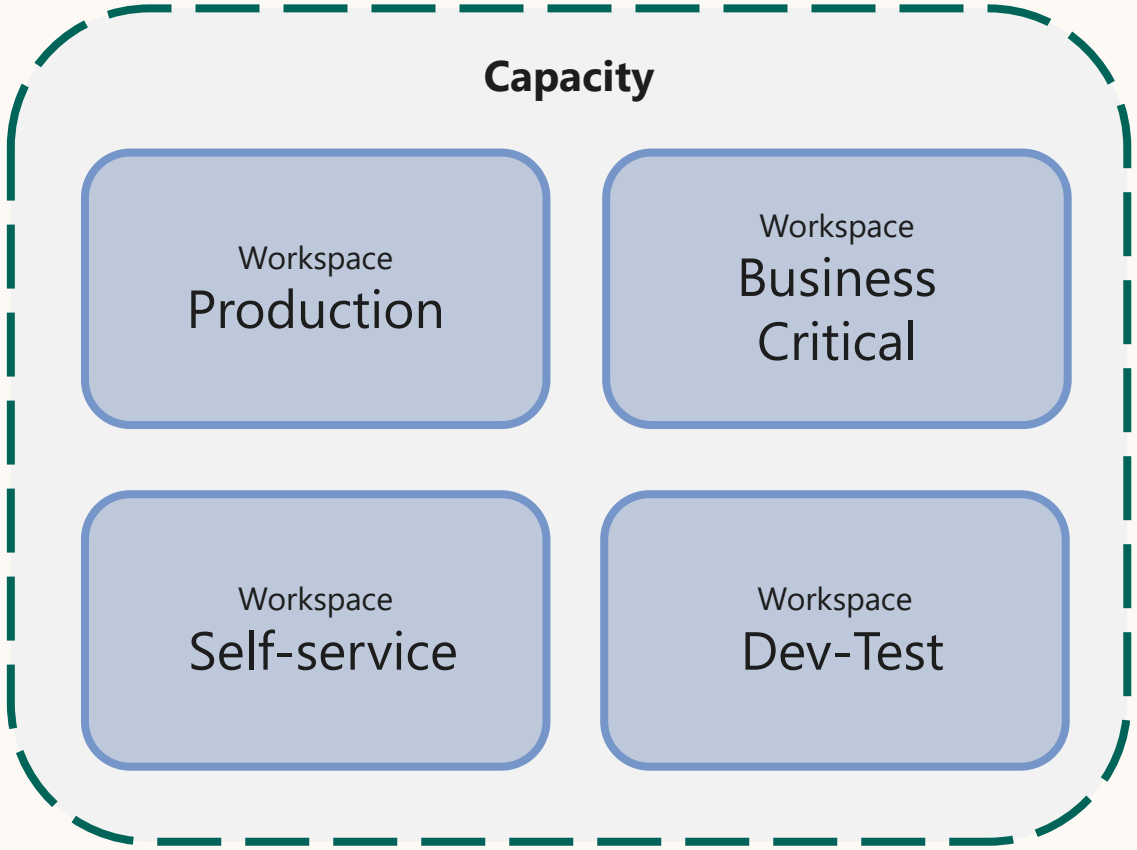
Microsoft Fabric
COMMUNITY CONFERENCE



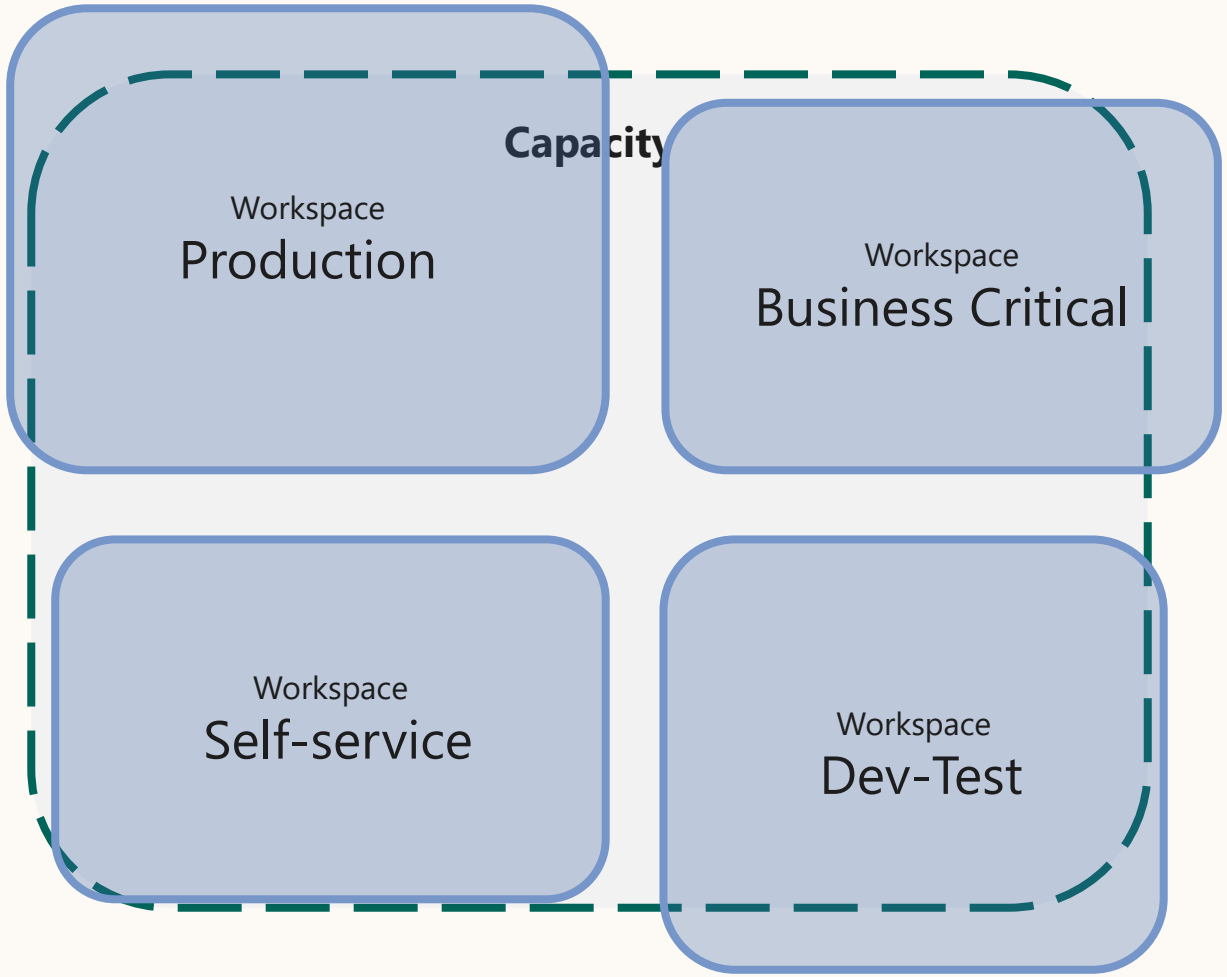
Workspace & Capacity Strategies

When 💩 hits the fan ..

What if the capacity runs out?



What if the capacity runs out?



Strategy 1: Optimize

Approach

- ❑ Work with content creators to optimize CU utilization
- ❑ Establish and follow best practices

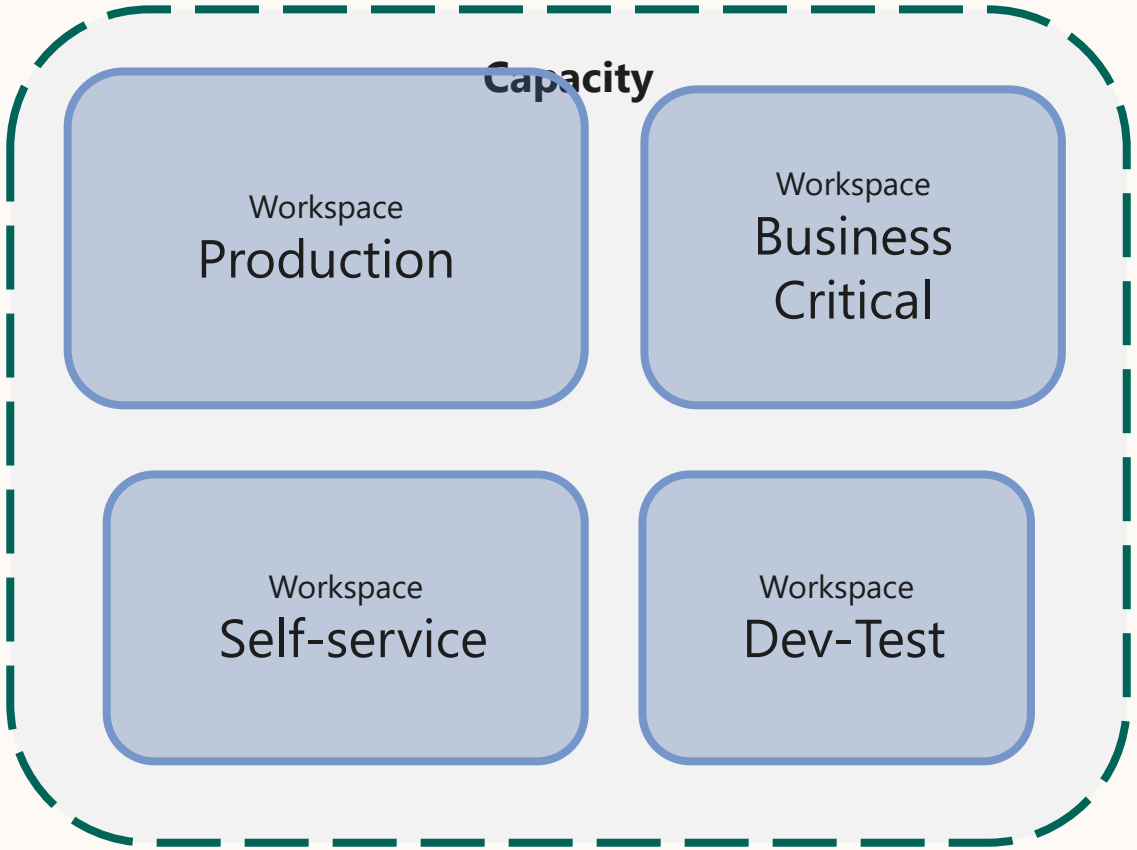
Pros

- ✓ Avoids increased cost
- ✓ Learning carries over

Consa

- x Can be difficult and time-consuming

What if the capacity runs out?



Strategy 1: Optimize

Approach

- ❑ Work with content creators to optimize CU utilization
- ❑ Establish and follow best practices

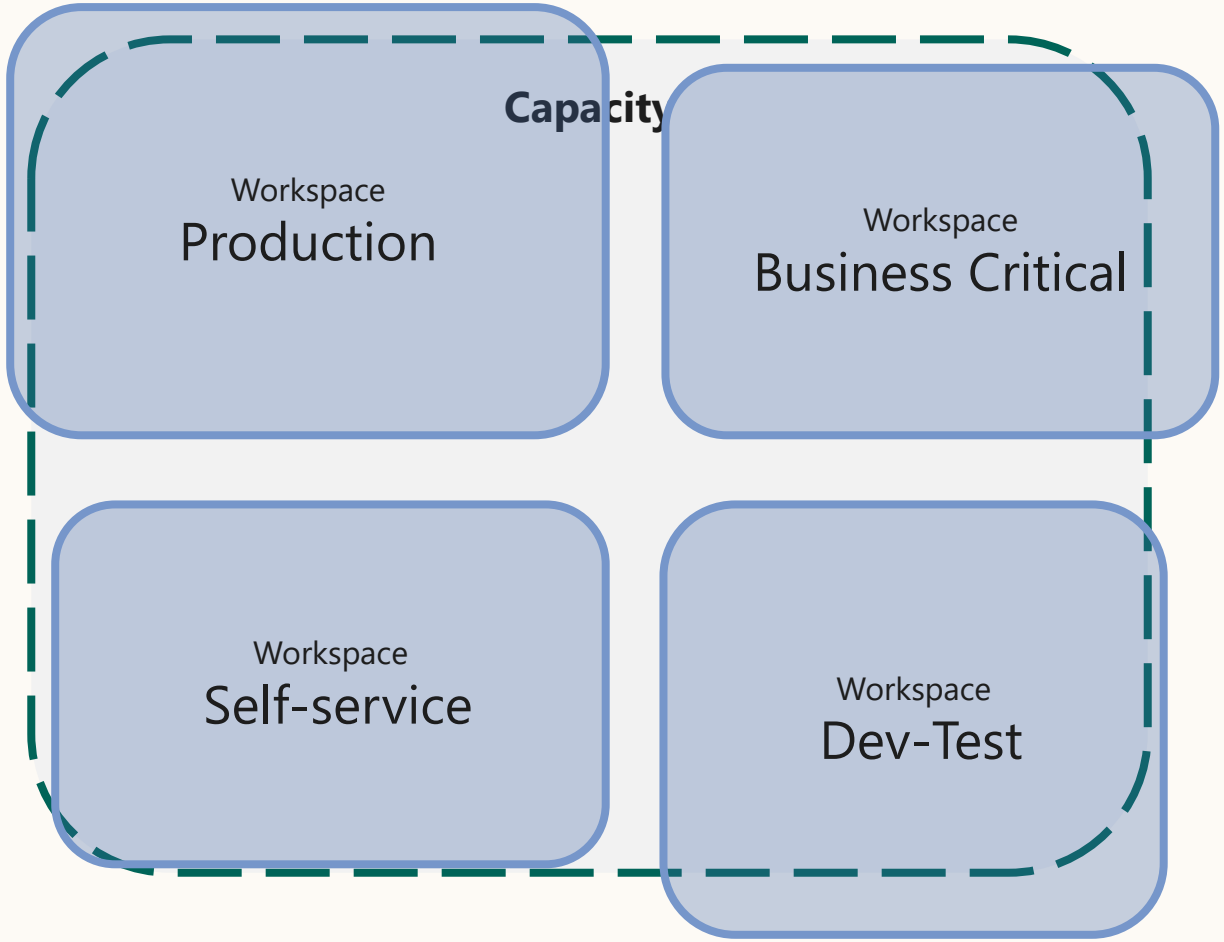
Pros

- ✓ Avoids increased cost
- ✓ Learning carries over

Cons

- x Can be difficult and time-consuming

What if the capacity runs out?



Strategy 2: Scale up

Approach

- ❑ Move to a bigger SKU size
- ❑ Turn on autoscale on a P SKU or setup size change on a F SKU

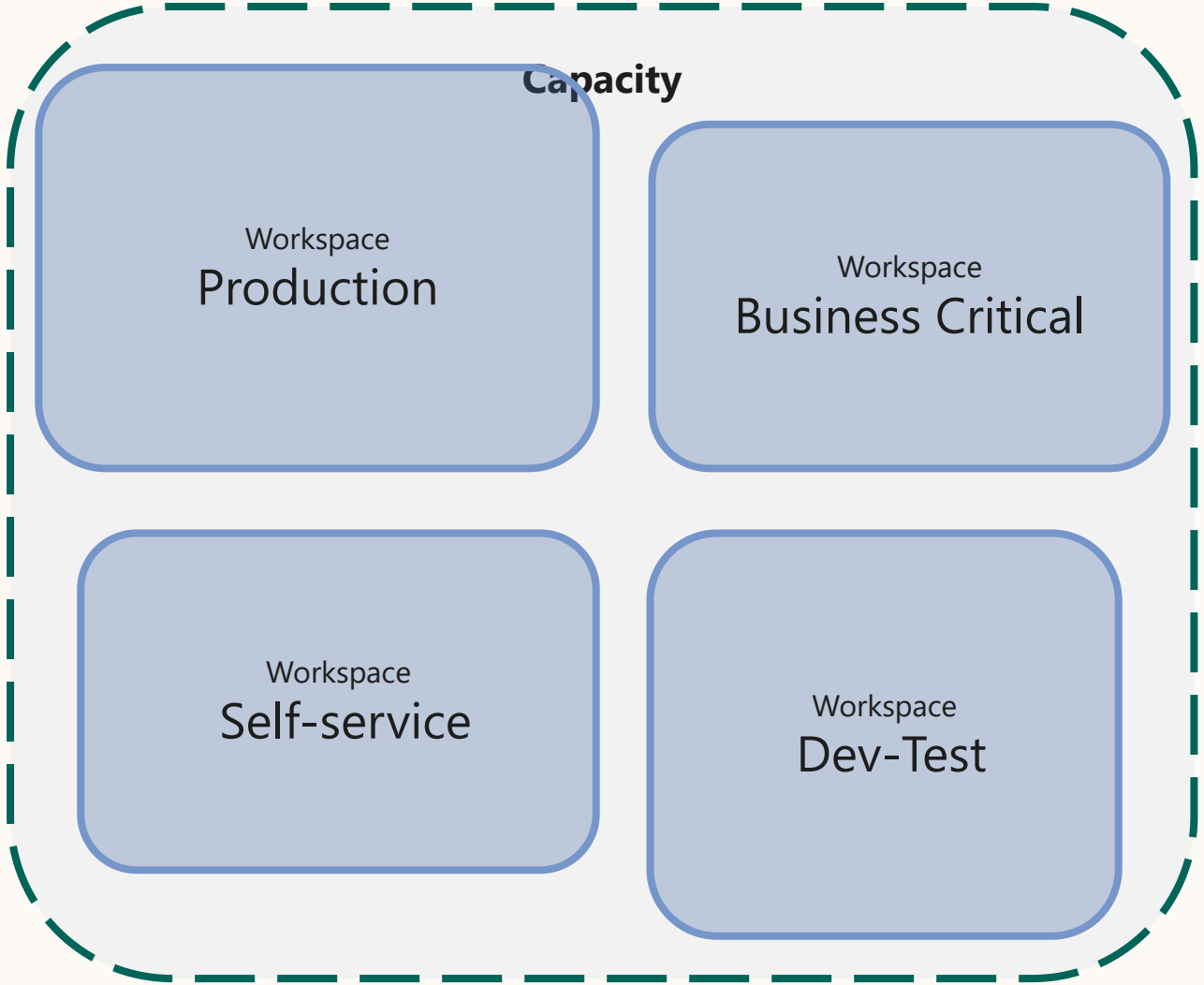
Pros

- ✓ Adds available CUs for every item
- ✓ Easy

Cons

- x Cost
- x Bad actors can still be a problem

What if the capacity runs out?



Strategy 2: Scale up

Approach

- ❑ Move to a bigger SKU size
- ❑ Turn on autoscale on a P SKU or setup size change on a F SKU

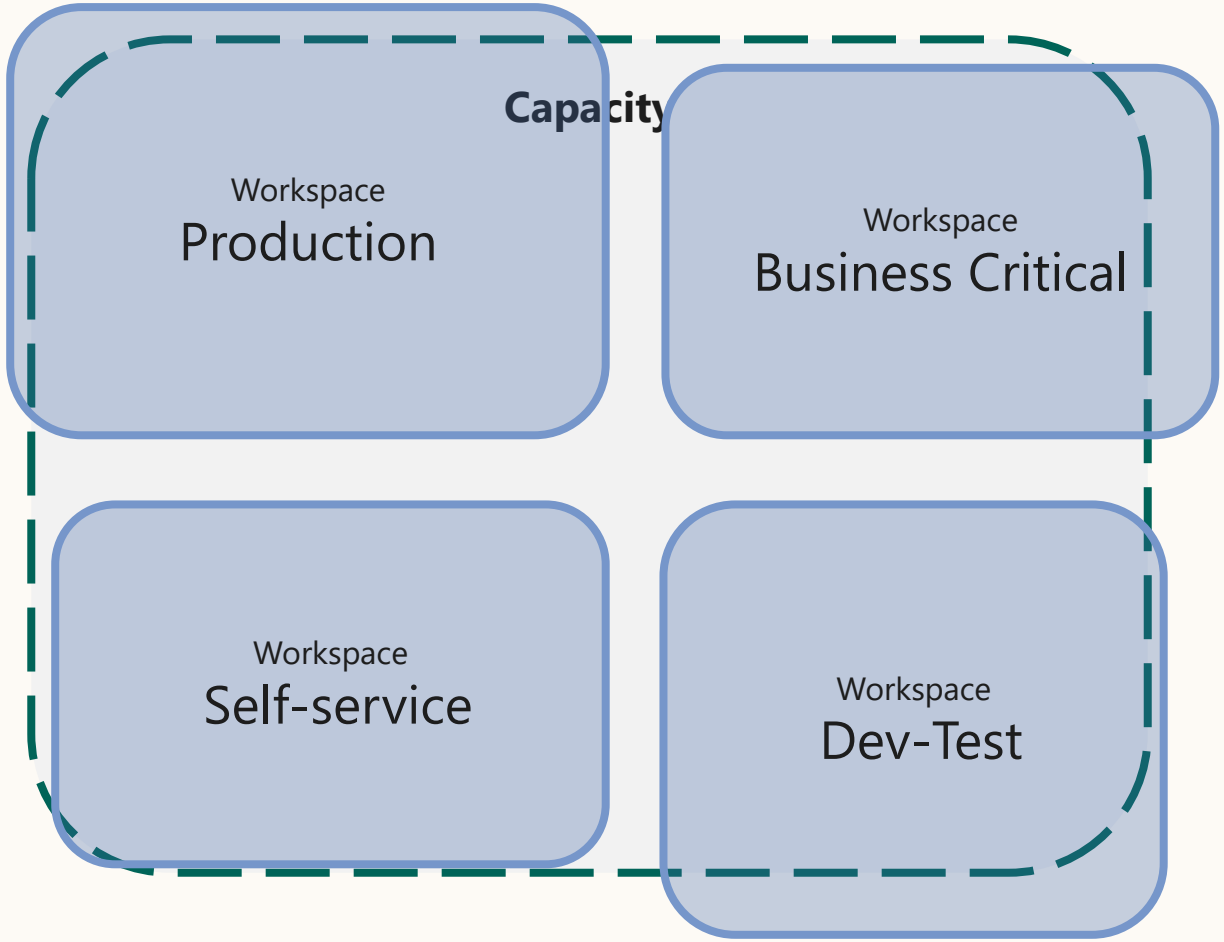
Pros

- ✓ Adds available CUs for every item
- ✓ Easy

Cons

- x Cost
- x Bad actors can still be a problem

What if the capacity runs out?



Strategy 3: Scale out

Approach

- ❑ Create multiple smaller SKUs based on workload, organization, type of work, etc..

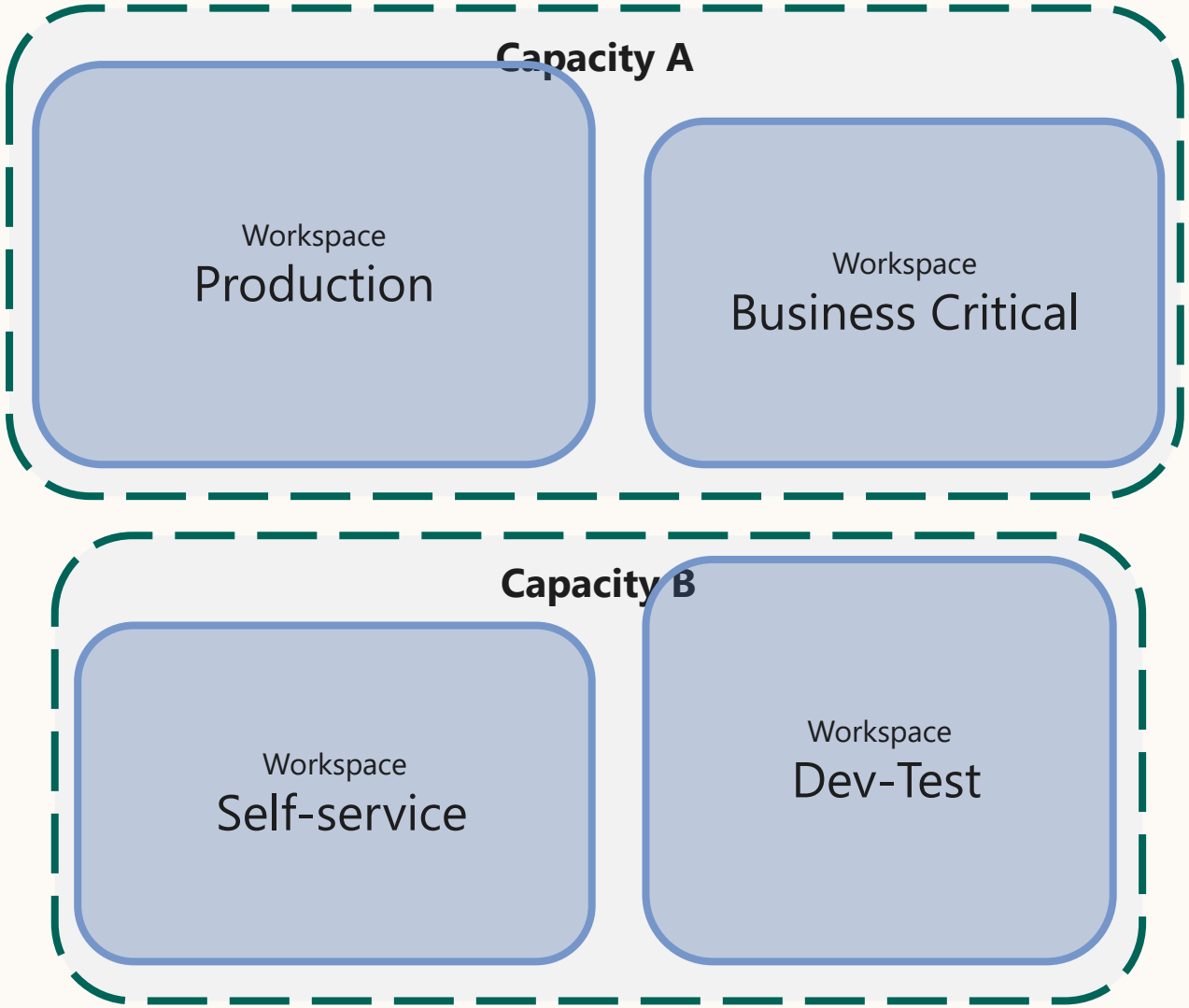
Pros

- ✓ Easy
- ✓ Provides some isolation from bad actors
- ✓ Flexibility

Cons

- x Cost
- x High CU items have increased change of throttling

What if the capacity runs out?



Strategy 3: Scale out

Approach

- ❑ Create multiple smaller SKUs based on workload, organization, type of work, etc..

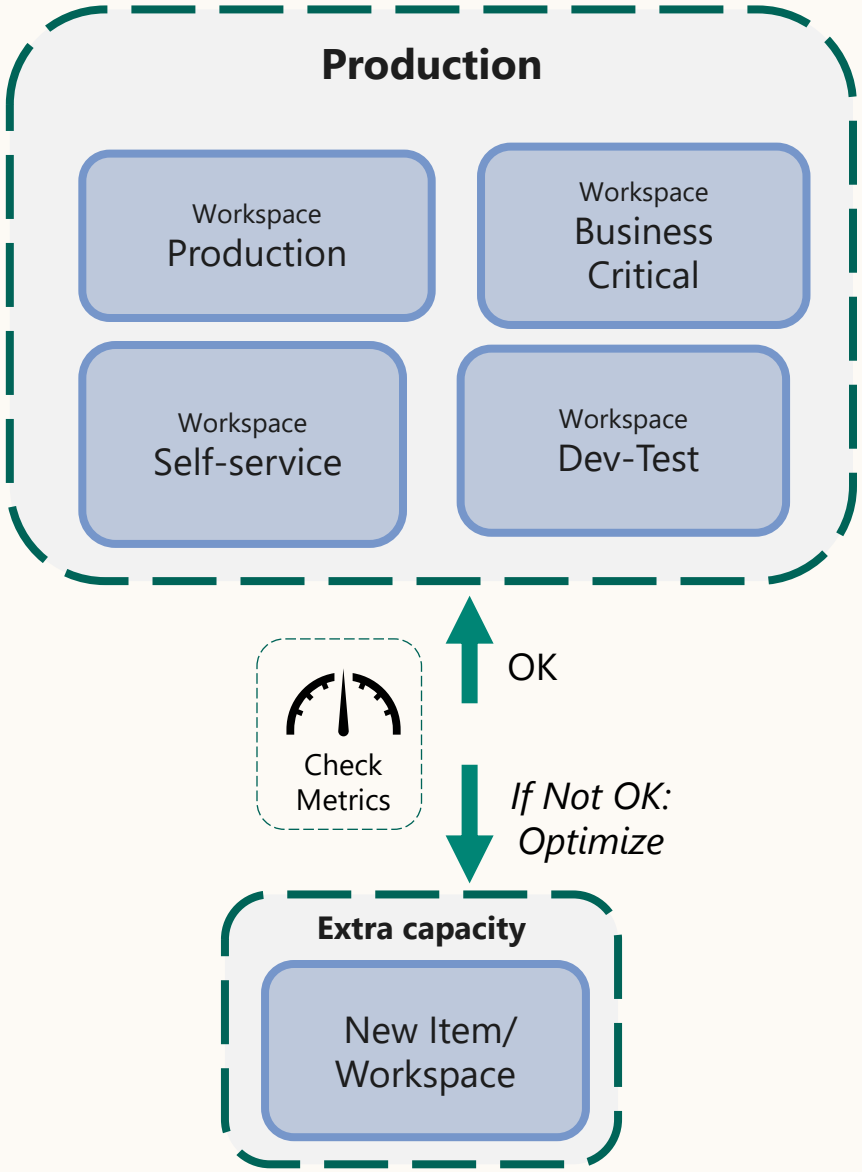
Pros

- ✓ Easy
- ✓ Provides some isolation from bad actors
- ✓ Flexibility

Cons

- x Cost
- x High CU items have increased change of throttling

What if the capacity runs out?



Strategy 4: Isolate

Approach

- ❑ Set up active monitoring and separate specific cases from other workloads in designated capacities

Pros

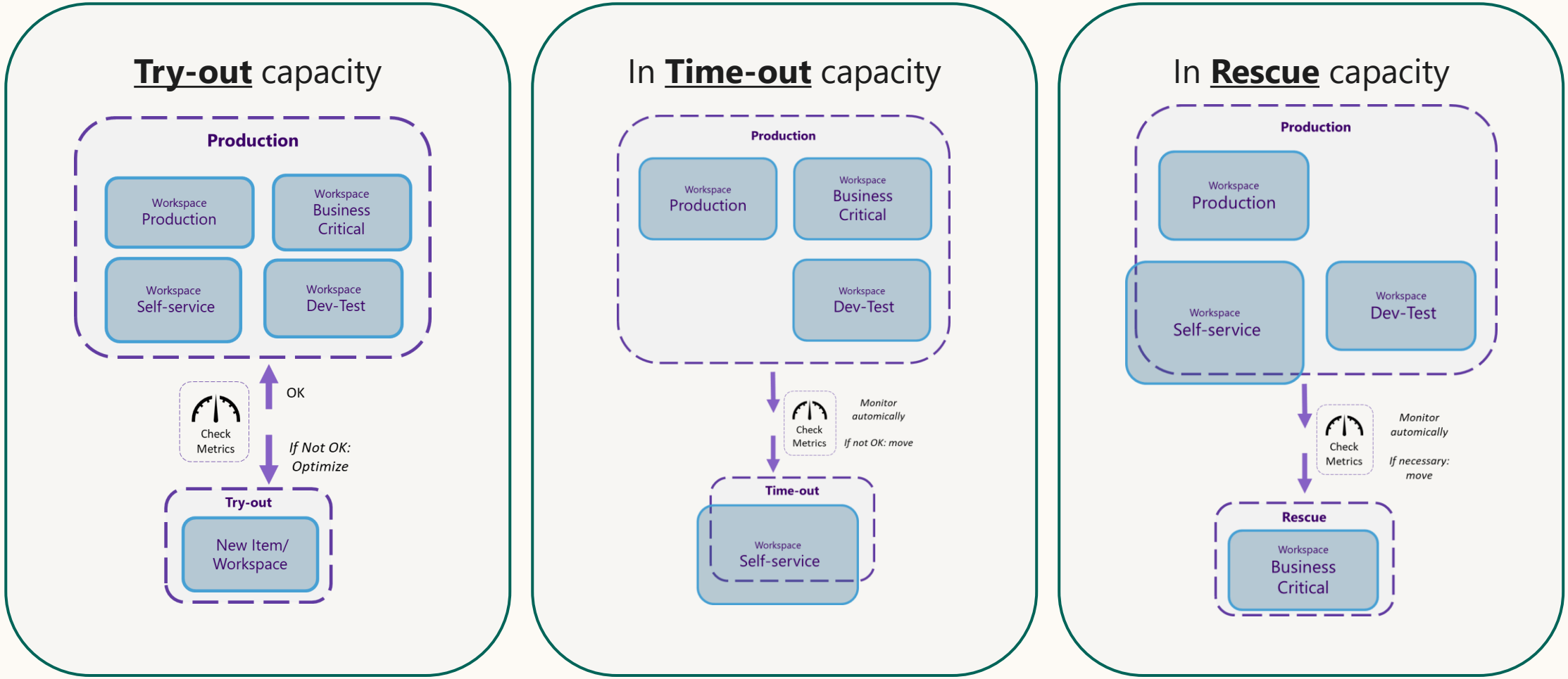
- ✓ Provides some isolation from bad actors
- ✓ Flexibility

Cons

- x Requires planning and maintenance

What if the capacity runs out?

Strategy 4: Isolate



Autoscale billing for Apache Spark

GA (July 2025)

Enable everyone in your org to use Spark and manage its cost (blog || docs)

Serverless style billing for Spark jobs

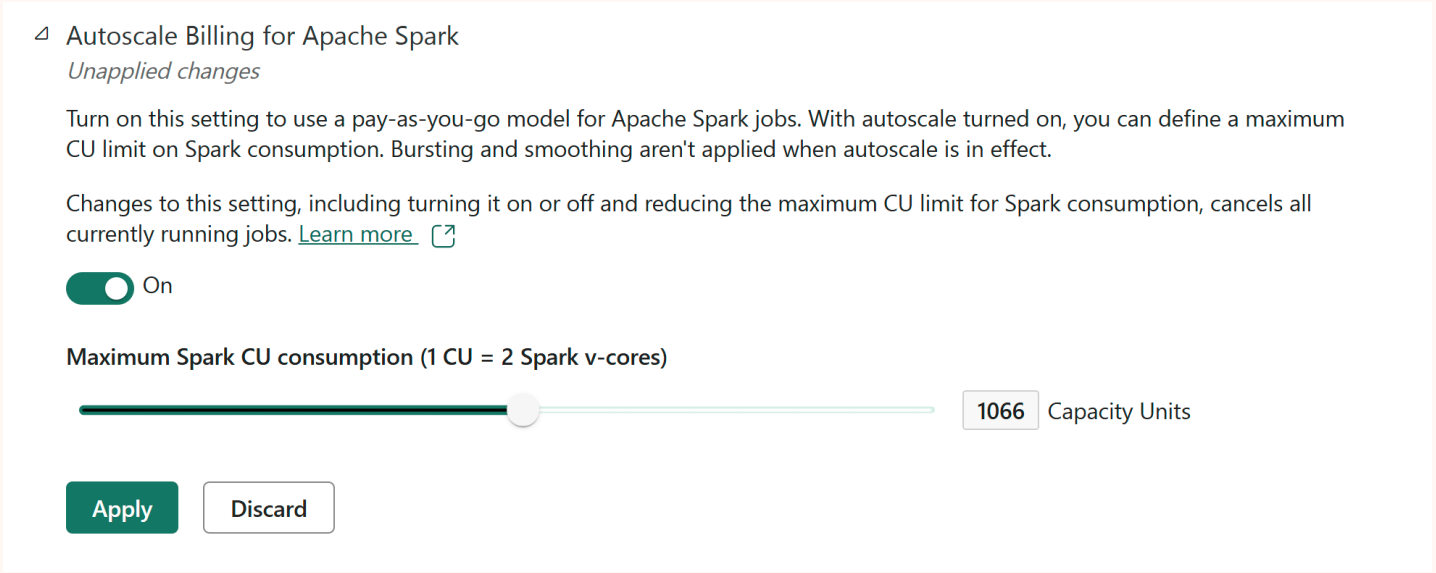
- Capacity admins can opt-in
- Set a max limit on CU used by Spark
- Only pay for what you use
- Spark manages the limit ensuring pools don't over consume
- Observability in a new metrics app page

Spark jobs are billed separately

- Jobs are billed when they execute
- Cost is at **Pay-as-you-go rate**
- Must also have an active capacity
- If Spark calls other workloads, like OneLake, those costs are billed to the capacity.

Impact

- Isolate your spiky Spark jobs from the rest of capacity compute
- Helps save on costs and reduce throttling



But .. Is it right for you?

- Part 1
- Part 2
- Part 3

Autoscale billing for Apache Spark

GA (July 2025)

Monitor consumption in Metrics app

New Autoscale Compute for Spark page

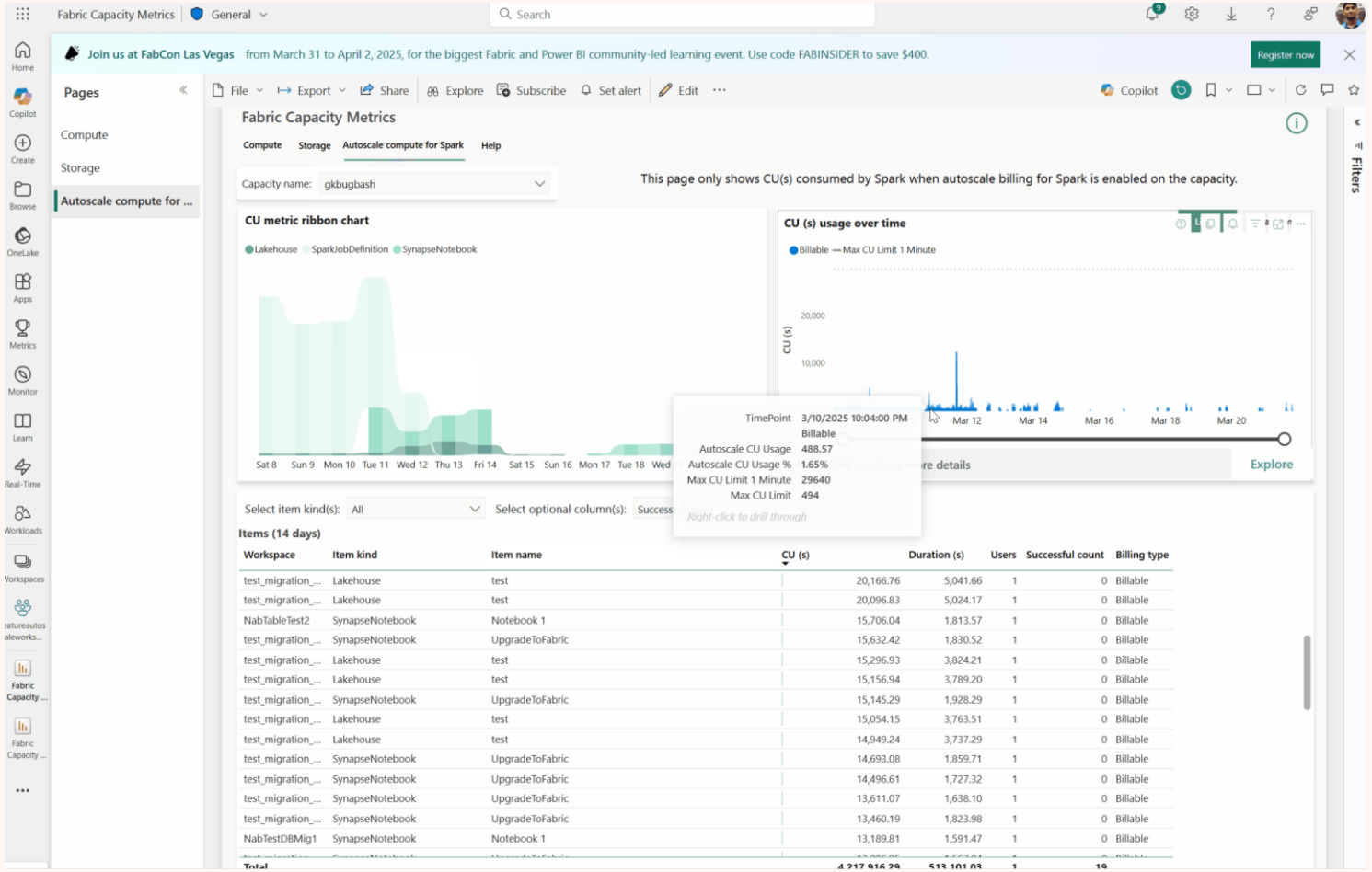
- Shows Spark CU consumed through Autoscale
- Easy to track to the configured autoscale limit.

Familiar experience

- Same experiences as for the capacity compute page.
- Provides drill down experience to see operation details

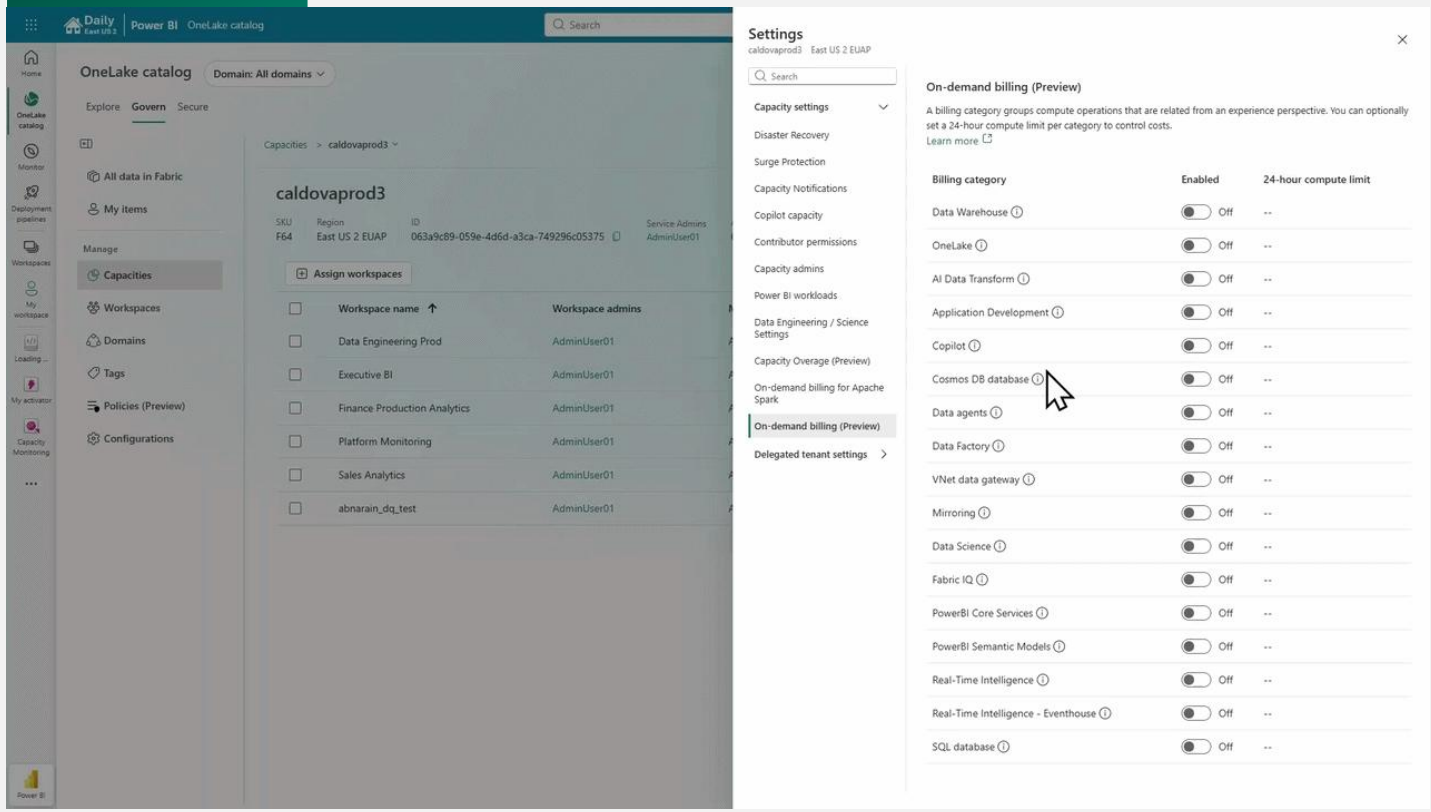
Impact

- Clearly understand the compute specific to Spark that will be reflected on your bill



Fabric on-demand billing and zero-provisioned

Public preview



Pay only for what you use with automatic compute scaling and no idle infrastructure costs.

Get started instantly with no upfront capacity planning

Optimize costs by combining on-demand for bursty workloads with capacity for steady workloads

Stay in control with spend limits, usage monitoring, and built-in governance

Fabric Copilot Capacity

GA (Nov 2024)

Enable everyone in your org to use Copilot and manage its cost (blog || docs)

Enable everyone to use Copilot

- All users can use Copilot experiences
- Consumption of Copilot goes to only the selected capacity

Select who can use a Copilot capacity

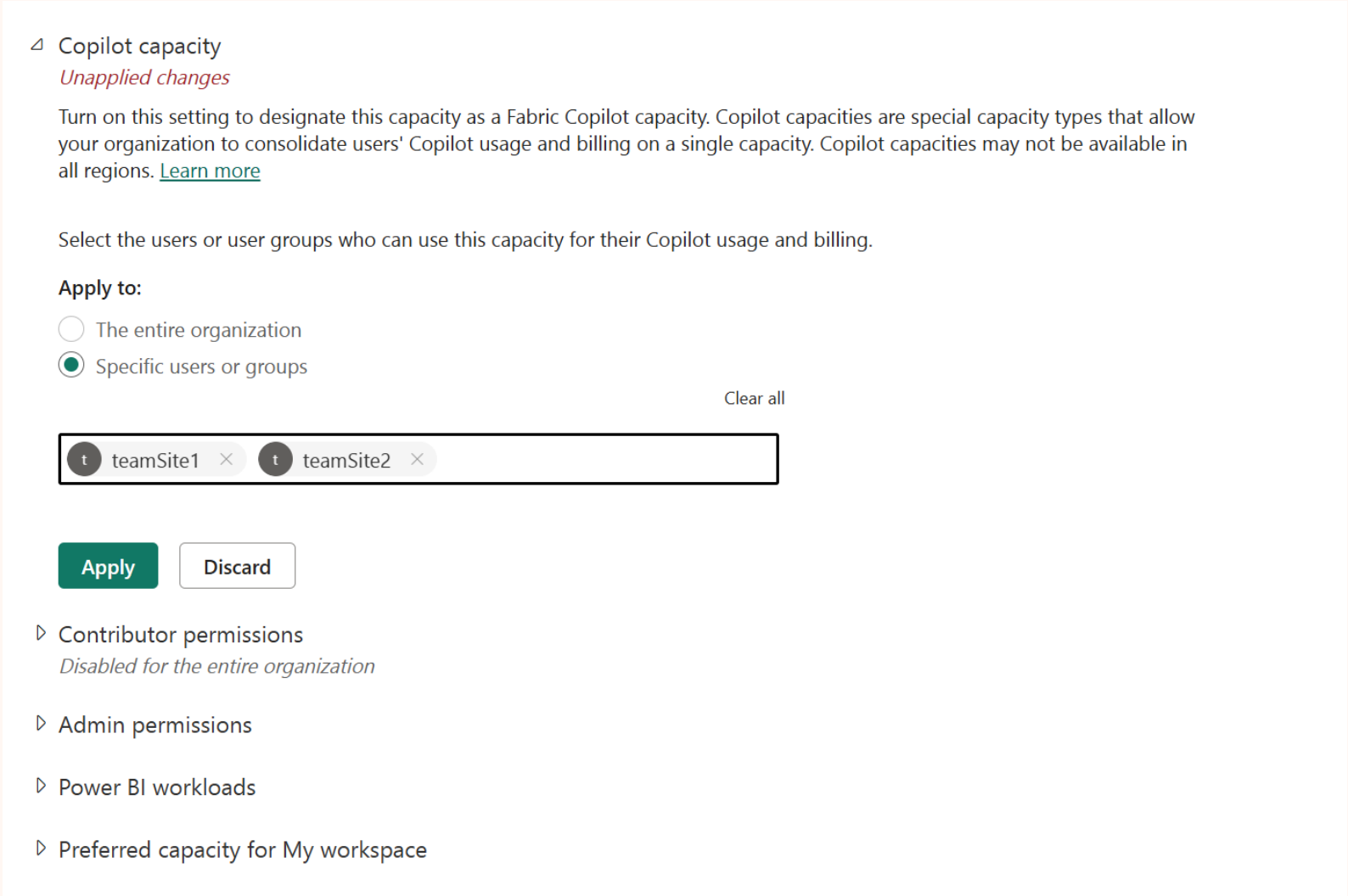
- Select the Users or groups who use a specific Copilot capacity
- A user can have only one Copilot capacity
  - Newest one matters..

No longer just P & F SKUs

- Pro, Premium Per User, and Trial

Tenant Setting

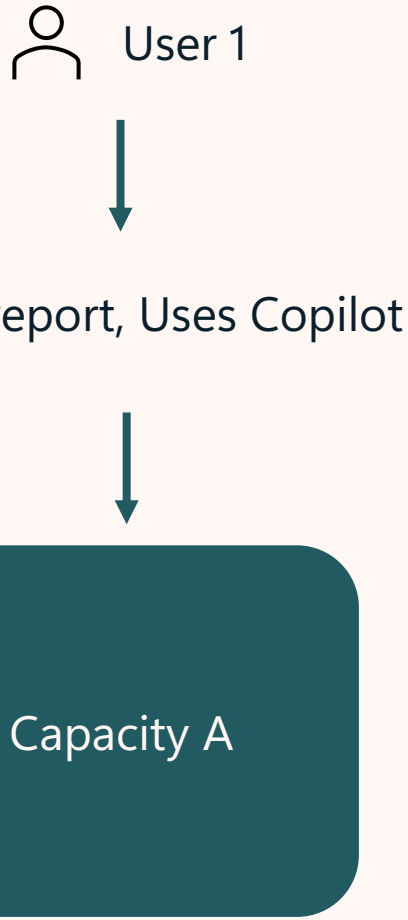
- Restrict who can configure
- "Capacities can be delegated .."



Fabric Copilot capacity

GA (Nov 2024)

Normally, copilot usage applies to the capacity the content is in.

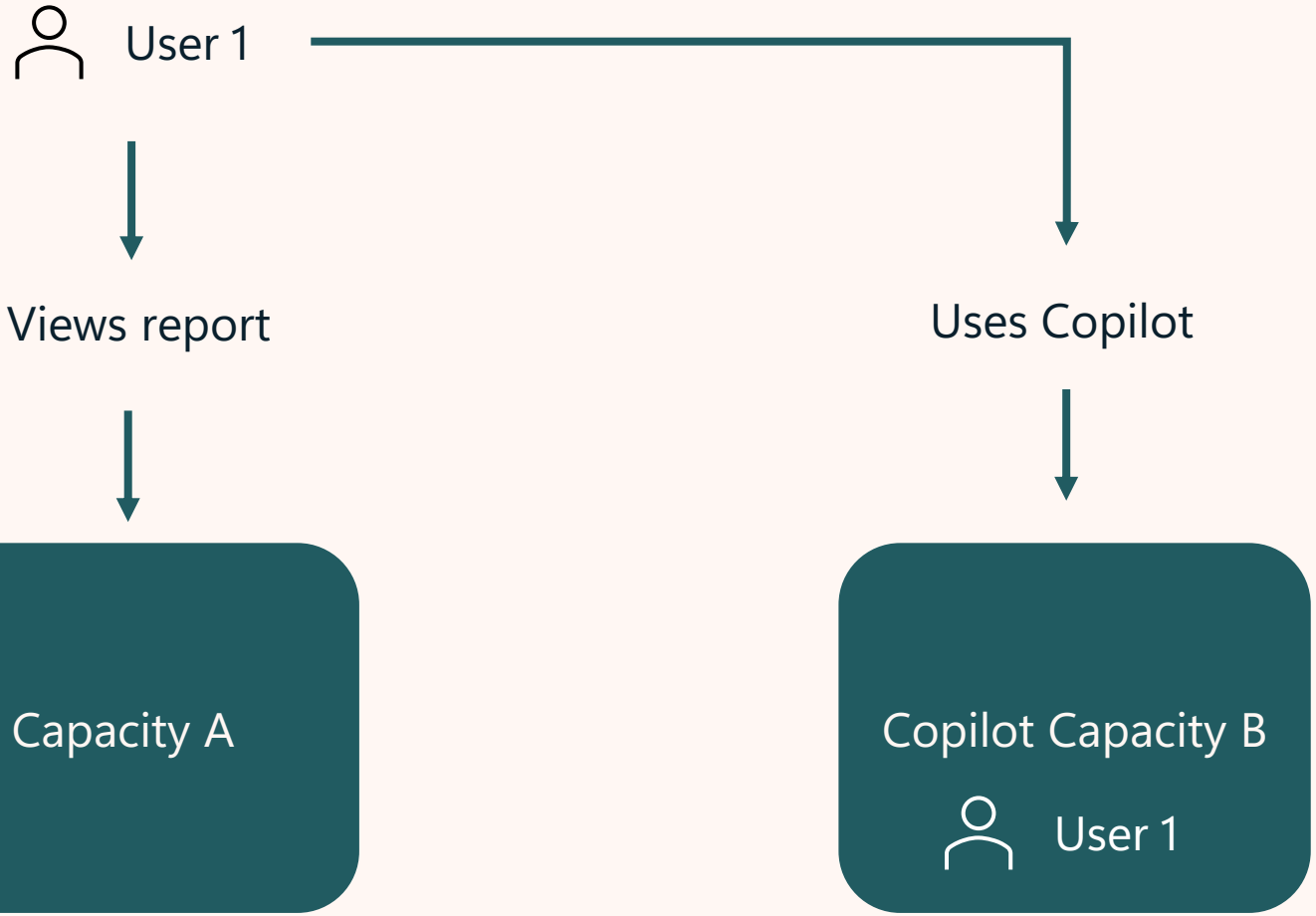


User 1

Fabric Copilot capacity

GA (Nov 2024)

After a user is added to a copilot capacity, the user's copilot usage is consumed from their copilot capacity



Protecting your capacity from Copilot usage

Copilot in Fabric operations are background, so no immediate spikes

- But when they do .. it has a 24h impact

Be mindful of who has access to Copilot skills and educate!

- Copilot requests trigger other operations too!

Track usage for Power BI Desktop, ensuring usage doesn't interfere with key workloads

- ℹ️ Daily check for "Power BI Session Desktop" item name in Metrics App
- Any workspace with permissions (Contributor), on Capacity that allows Copilot usage
- If user is assigned a Copilot Capacity, it automatically goes here

Track usage for Copilot in Service, ensuring usage doesn't interfere with key workloads

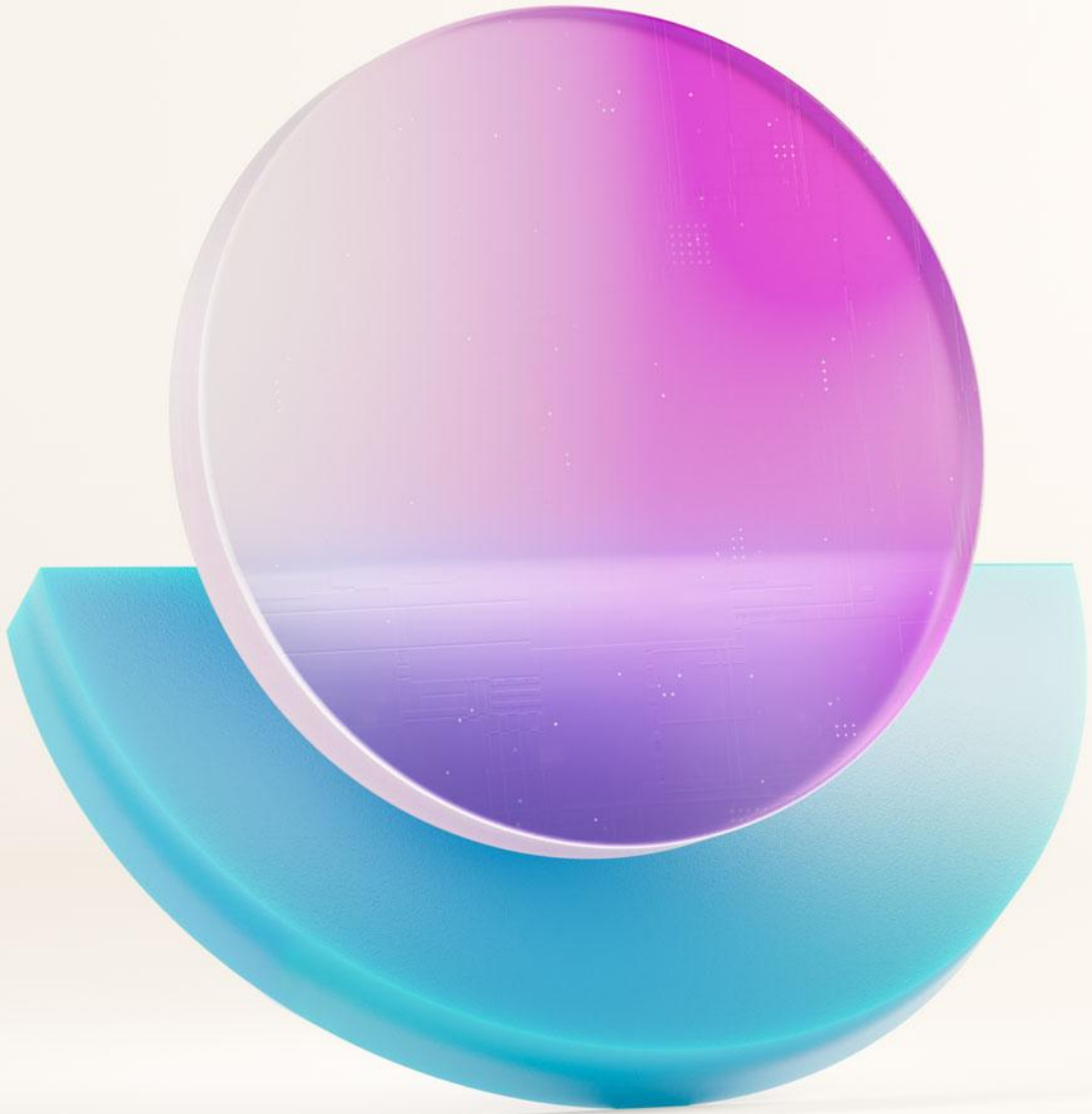
- ℹ️ Daily check for "Power BI Session Web" item name in Metrics App
- Any workspace with permissions (Viewer), on Capacity that allows Copilot usage
- If user is assigned a Copilot Capacity, it automatically goes here

Protecting your capacity from Copilot usage

Options to ensure the health of your capacity

- Fabric Copilot Capacity enabled for user base (by region, department, ..)
  - If possible, set up new Security Groups (avoid cross pollination)
- When Capacity throttles/rejects, Copilot no longer works
  - But everything else does .. 😼

Microsoft Fabric
Community Conference



Monitoring Capacities

Capacity Chargeback Reporting

Public Preview

Allocate costs to those who use your capacity (docs)

Helps allocate costs across your org

- Built-in turnkey reporting
- Rolls up usage per workspace / item / user

Focuses on % utilization

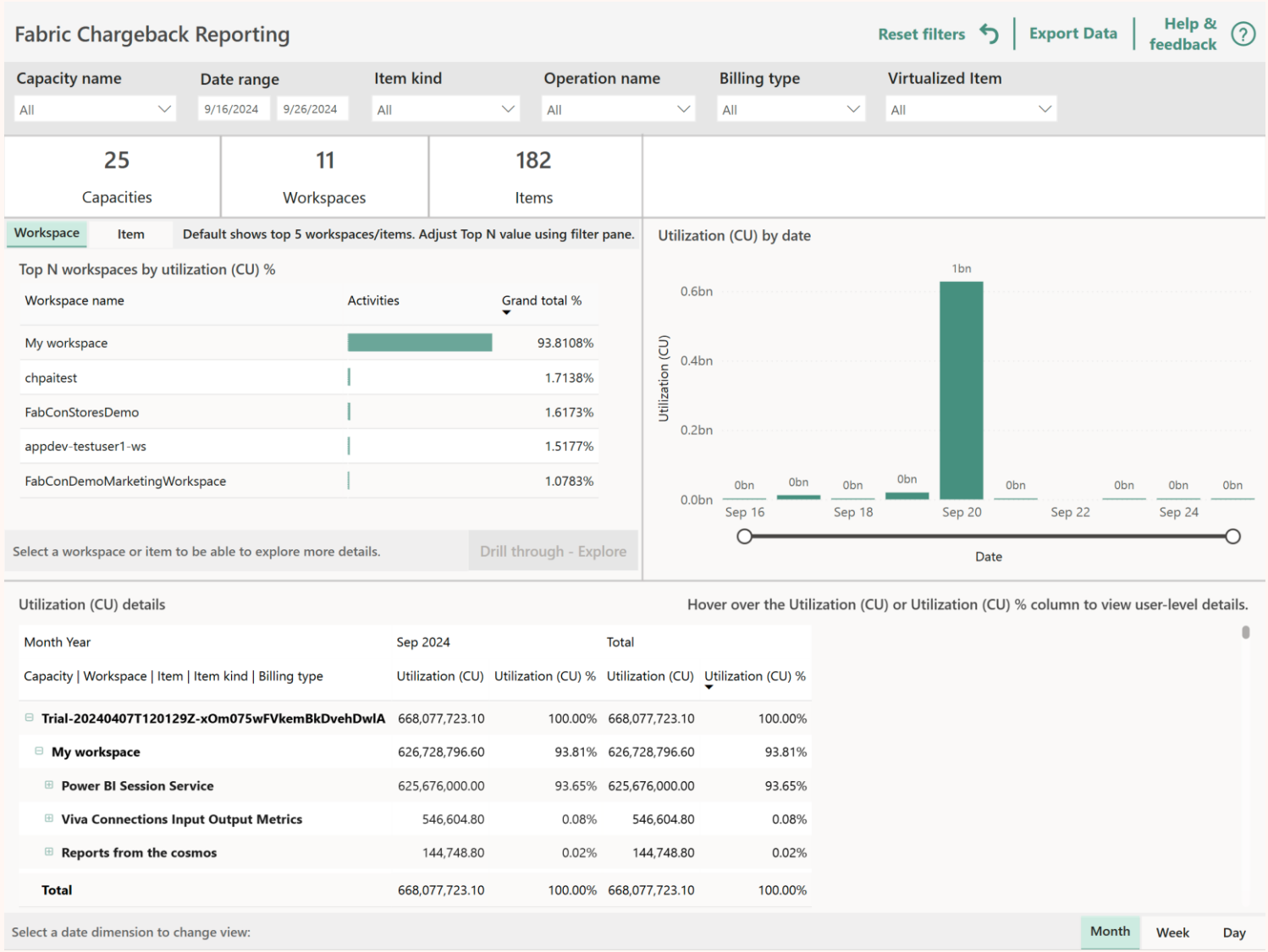
- Orgs need to look at cost in Azure billing and then allocate that cost to their content owners.

Impact

- Turn-key solution for Fin-ops with reduction in need to build custom solutions

Future

- Domain support
- Tags support



Capacity Platform Data in Realtime Hub (RTH)

GA + PuPr

Direct access to the same Capacity Platform data used in Capacity Metrics with flexible routing and alerting capabilities

Data Source(s)

- Platform Telemetry
- + many more..

Real-Time Intelligence

1. Ingest & Process
2. Analyze & Transform
3. Alert and Respond

Eventstream

Eventhouse

Reflex

KQL Queryset

Real-Time Hub

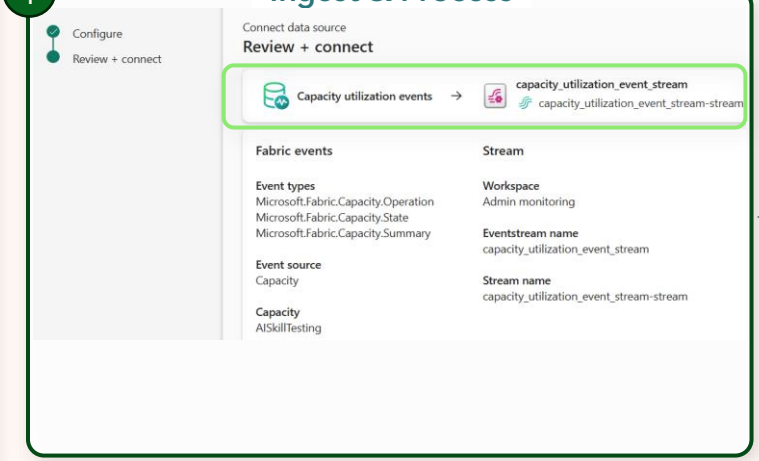
RT Dashboard

Power BI

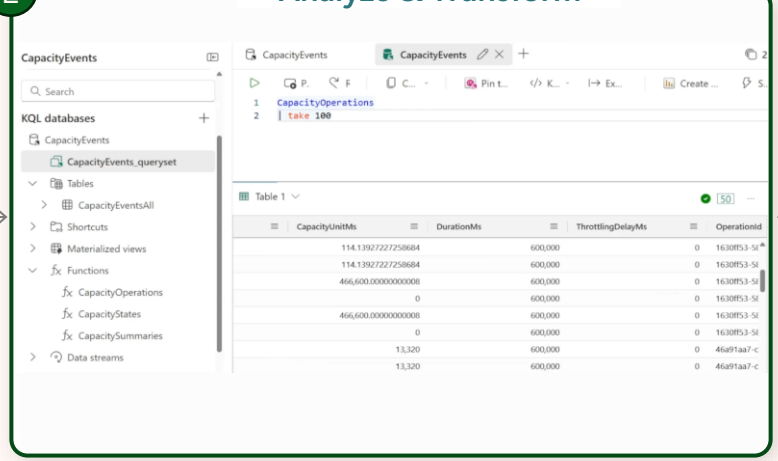
OneLake

AI

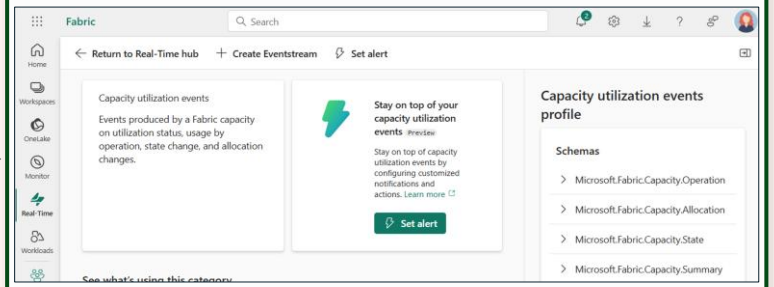
1 Ingest & Process



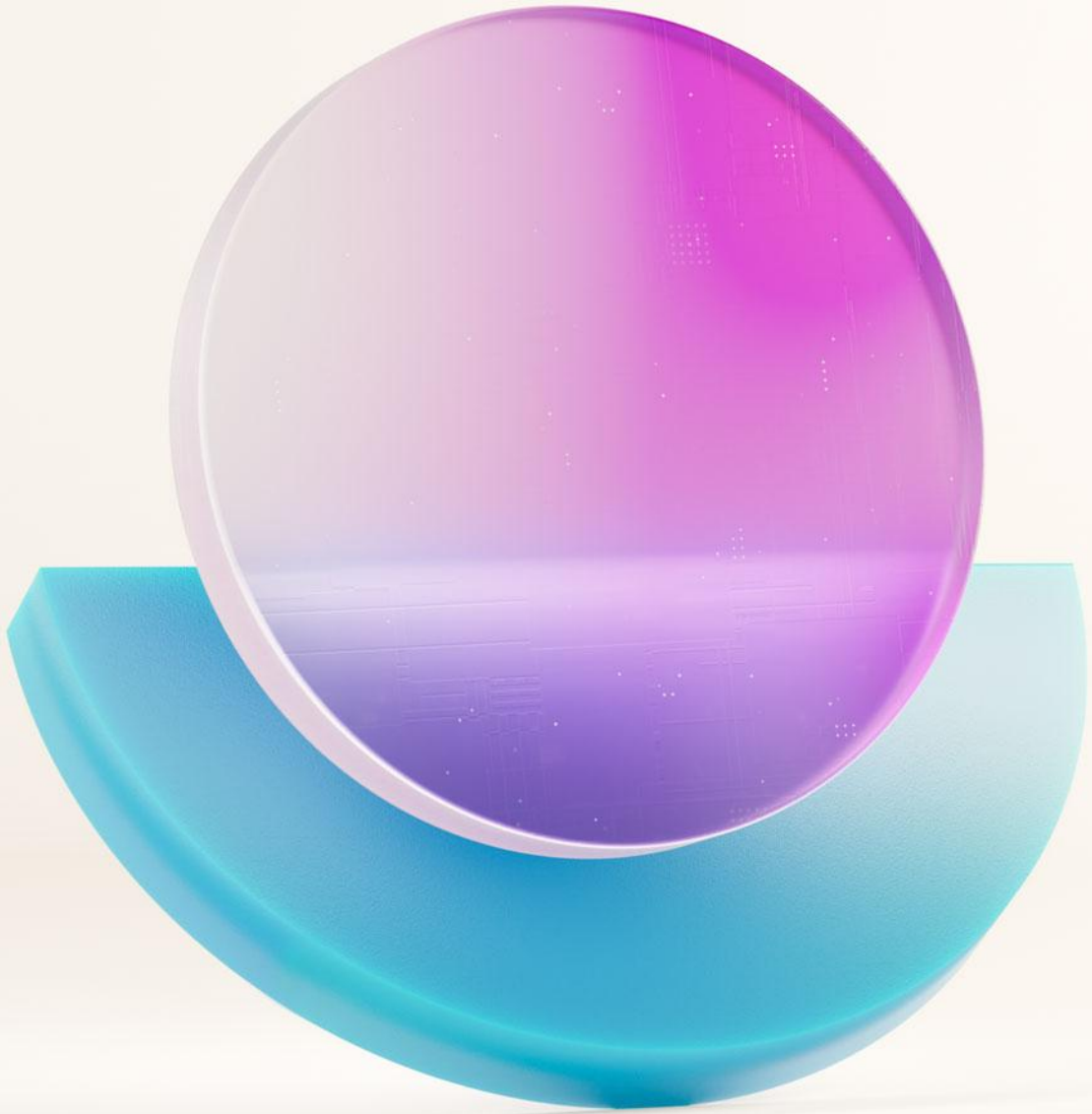
2 Analyze & Transform



3 Alert and Respond



Microsoft Fabric
Community Conference



Wrapping up

Best practices for deploying your capacities

Plan your capacities

Ensure your capacities are correctly sized

Use dedicated capacities to optimize quality of experience and costs

Isolate production, development, testing in separate capacities

Budget for variability

Manage Resources

Enable Surge Protection

Monitor usage using metrics app

Adjust workload limits like pools, memory, and timeouts

Share best practices with colleagues

Optimize experiences and costs

Consider Autoscale billing for Spark

Consider Fabric Copilot Capacities

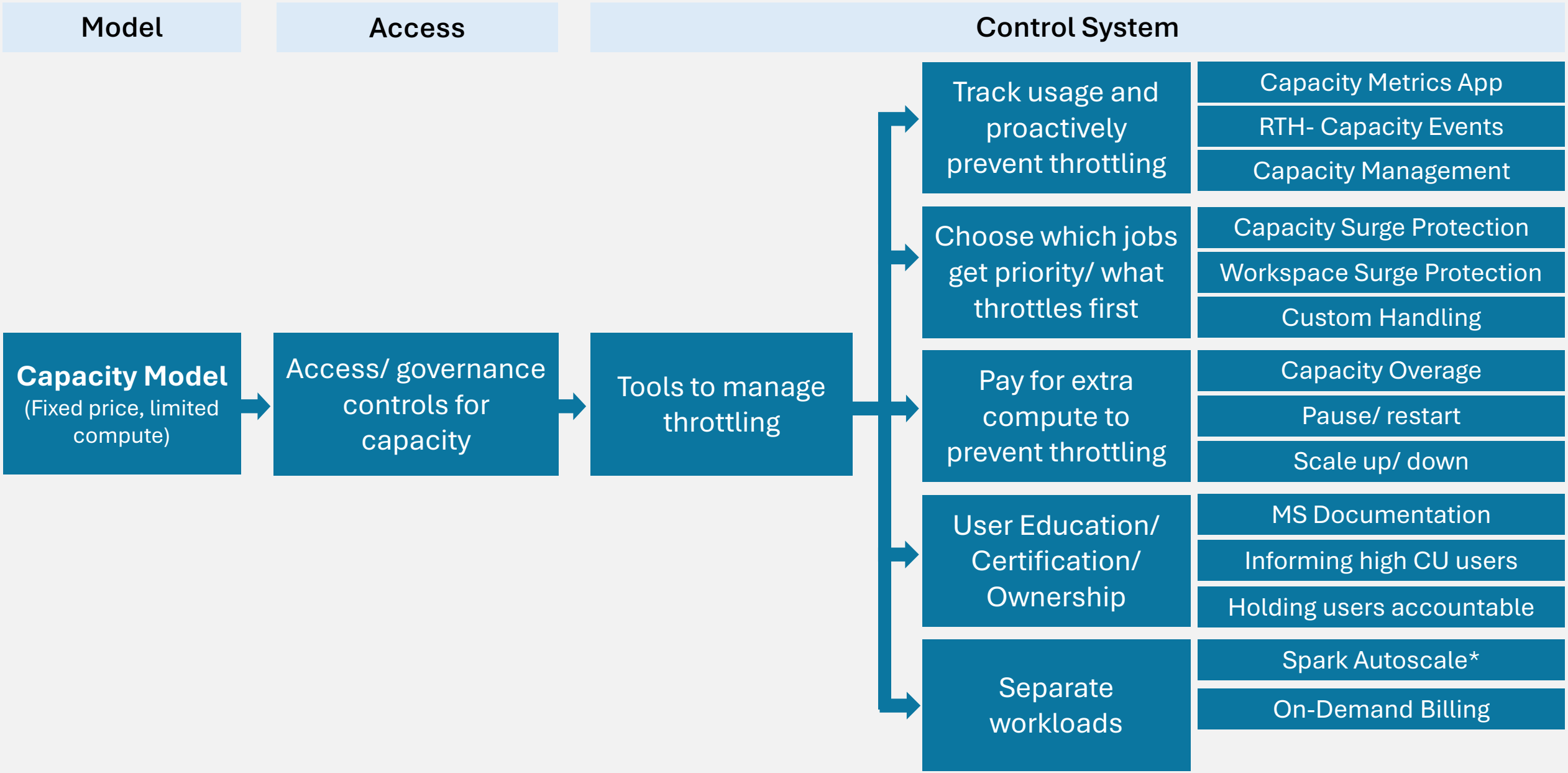
Leverage pause/resume **appropriately**

Resize capacities as needed

Move problematic content to rescue, time-out, or testing capacities

Capacity Management Overview

Controlling how much CU is consumed and where



How do you prevent overloading your capacities...

Use multiple capacities and strategies to operate your capacities

Capacity A

For general purpose compute needs

Sized for typical needs

Capacity B

For Self-Service Reporting workloads

Capacity C

For large periodic workloads

Paused when not needed

Capacity D

For testing newly built content

Small size to avoid large costs

Surge protection

Resize

Pause and Resume

Autoscale Billing for Spark, Copilot Capacity

OneLake Shortcuts

Rate This session

Deep Dive into Microsoft Fabric Capacities: Fundamentals, Concepts, and Advanced Strategies

